



Universidad
Internacional
de Valencia

Predicción de la demanda diaria de transporte público en Madrid mediante una arquitectura híbrida residual LSTM-XGBoost

Trabajo Fin de Máster

Septiembre-2026

Titulación:
Máster Universitario en
Big Data y Ciencia de
Datos

Alumno/a:
Aday Cuesta Correa

Director/a del TFT:
Gonzalo Surribas Sayago

Convocatoria:
Segunda Convocatoria
(2025-2026)

Curso académico
2025-2026

De:

 Planeta Formación y Universidades

Índice general

1	Introducción y objetivos	1
1.1	Objetivo general y específicos	2
1.2	Preguntas de investigación	3
1.3	Planificación temporal, cronograma y ciclo de vida	4
1.4	Estructura del documento	6
2	Predicción de la demanda de transporte público: dominio, técnicas y trabajos previos	7
2.1	Movilidad urbana y demanda de transporte	7
2.2	Modelado de series temporales y aprendizaje automático tabular	8
2.3	Arquitecturas híbridas y de ensamble	9
2.4	Análisis comparativo y diferenciación metodológica	11
3	Fuentes de datos, integración e ingeniería de características	14
3.1	Fuentes	14
3.2	Unificación y validación	16
3.3	Análisis exploratorio de datos	18
3.4	Ingeniería de características	20
3.5	Particiones y fuga de datos	23
4	Arquitectura híbrida residual LSTM-XGBoost para demanda diaria agregada	25
4.1	Diagnóstico previo: ¿tiene el residuo estructura de calendario?	26
4.2	Baselines	26
4.3	Etapas 1: LSTM	28
4.4	Esquema de validación fuera de muestra (OOF)	30
4.5	Etapas 2: XGBoost sobre el residuo	32
4.6	Modelos de control	33
4.7	Ensamble por combinación de pronósticos	34
5	Evaluación experimental del sistema predictivo	36
5.1	Tabla maestra de comparación	36
5.2	Ajuste temporal	38
5.3	Análisis de residuos	39
5.4	Error por tipo de día	41
5.5	Importancia de variables	44
5.6	Contraste central del TFM	47
5.7	Análisis de sensibilidad	48
5.8	Por qué gana el ensamble: descorrelación de errores	50
5.9	Anatomía del residuo de la etapa 1	51
5.10	Paridad informativa entre etapas: ablación y control LSTM	57

5.11	Desglose por subgrupos y periodos	61
5.12	Respuesta a las preguntas de investigación	66
6	Conclusiones y trabajos futuros	68
6.1	Conclusiones	68
6.2	Limitaciones	69
6.3	Trabajos futuros	71
6.4	Implicaciones para la planificación	72
6.5	Contribución a los Objetivos de Desarrollo Sostenible	72
A	Catálogo de variables	A1
B	Suite de tests y garantías anti-fuga	A3
C	Reproducibilidad y entorno	A6
D	Declaración de uso de inteligencia artificial generativa	A9
D.1	Herramientas empleadas y alcance	A9
D.2	Tareas no delegadas	A10
D.3	Referencia de las herramientas	A10
E	Cuadros diagnósticos extendidos	A11

Resumen

El transporte público metropolitano de Madrid, gestionado de forma coordinada por el Consorcio Regional de Transportes de Madrid (CRTM), integra bajo un mismo marco cuatro operadores de naturaleza dispar —Metro, EMT, autobús interurbano por carretera y Cercanías—, cuya demanda diaria agregada condiciona decisiones operativas reales de flota, personal y frecuencias de servicio. Este Trabajo de Fin de Máster evalúa si una arquitectura híbrida residual de dos etapas mejora la predicción de esa demanda frente a modelos consolidados de una sola etapa. Sobre una serie diaria continua de 1.310 observaciones (1 de enero de 2023 a 2 de agosto de 2026), sin huecos ni valores nulos, se unifican las fuentes de demanda del CRTM, la meteorología de Open-Meteo y el calendario laboral municipal, y se construye una matriz de 161 variables predictoras —retardos de demanda total y por operador, medias y desviaciones móviles con desplazamiento previo estricto, armónicos de Fourier deterministas, codificación exhaustiva de calendario y meteorología retardada— sujeta a un conjunto de guardias anti-fuga verificadas por una suite de 470 pruebas automáticas. La primera etapa es una red LSTM univariante cuyas predicciones de entrenamiento se generan íntegramente fuera de muestra (*out-of-fold*) mediante una validación cruzada expansiva de cinco bloques; la segunda es un modelo XGBoost que corrige el residuo de la primera a partir de las variables exógenas. El resultado central es negativo para la arquitectura propuesta: el híbrido no supera a un SARIMAX seleccionado por AIC ni a un XGBoost entrenado directamente sobre las mismas variables (MAE de test 234.223 frente a 163.627 y 156.121, respectivamente). Este hallazgo se audita a la luz de la teoría de combinación de pronósticos de Granger (1989): el híbrido secuencial incumple la condición de que sus componentes se basen en conjuntos de información distintos, pues ambas etapas observan en última instancia el mismo pasado de la serie. Los modelos más precisos del estudio son dos ensambles aritméticos de SARIMAX y XGBoost —sin ninguna red neuronal ni reentrenamiento—, prácticamente indistinguibles entre sí: 139.725 de MAE de test en la variante equiponderada 50/50 y 139.681 en la ponderada por el inverso del MAE, ambas con un R^2 de 0,9755. Se selecciona la equiponderada por no depender de ningún dato de validación para fijar sus pesos; su ganancia se explica de forma cuantitativa mediante la identidad de reducción de varianza de Bates y Granger (1969) aplicada a la correlación medida entre los errores de sus componentes. En cuanto a su relación con trabajos previos, este Trabajo de Fin de Máster no continúa ningún proyecto anterior propio ni ajeno, y toma de un trabajo previo de quien ejerce su dirección (Surribas Sayago et al.) únicamente inspiración de ingeniería de características —la construcción de retardos, los armónicos deterministas y la idea de una rama exógena—, sin replicar su arquitectura: ninguna de las cifras aquí presentadas procede de aquel trabajo.

Palabras clave: transporte público, predicción de demanda, series temporales diarias, híbrido residual LSTM-XGBoost, combinación de pronósticos, validación out-of-fold

Abstract

The metropolitan public transport system of Madrid, coordinated by the Regional Transport Consortium (CRTM), integrates four structurally different operators —the underground network, the municipal bus company (EMT), interurban coach concessions, and suburban rail (Cercanías)— whose aggregate daily demand shapes real operational decisions on fleet, staffing, and service frequency. This Master's Thesis evaluates whether a two-stage residual hybrid architecture improves the forecasting of that demand over consolidated single-stage models. On a continuous daily series of 1,310 observations (1 January 2023 to 2 August 2026), with no gaps and no missing values, the CRTM demand data, the Open-Meteo weather data, and the municipal working calendar are unified, and a matrix of 161 predictor variables is engineered —lags of total and per-operator demand, shift-first rolling means and standard deviations, deterministic Fourier harmonics, an exhaustive calendar encoding, and lagged weather— subject to a set of anti-leakage guards verified by a suite of 470 automated tests. The first stage is a univariate LSTM whose training-time predictions are generated strictly out-of-fold through an expanding five-block walk-forward cross-validation; the second is an XGBoost model that corrects the first stage's residual using the exogenous variables. The central result is negative for the proposed architecture: the hybrid beats neither an AIC-selected SARIMAX nor an XGBoost trained directly on the same variables (test MAE of 234,223 versus 163,627 and 156,121, respectively). This finding is audited through Granger's (1989) forecast-combination theory: the sequential hybrid violates the requirement that its components rest on genuinely different information sets, since both stages ultimately observe the same history of the series. The most accurate models in the study are two arithmetic ensembles of SARIMAX and XGBoost —with no neural network and no retraining— that are practically indistinguishable from one another: a test MAE of 139,725 for the 50/50 equal-weight variant and 139,681 for the inverse-MAE-weighted one, both with an R^2 of 0.9755. The equal-weight variant is the one selected, since it derives its weights from no validation data; its gain is quantitatively explained by the Bates and Granger (1969) variance-reduction identity applied to the measured correlation between the components' errors. The contribution of this work is therefore methodological rather than a gain in accuracy: a negative result established under a pre-registered protocol, with every verdict fixed in writing before the corresponding metric was observed, and audited against forecast-combination theory instead of being reported as an unexplained failure. As for its relation to prior work, this Master's Thesis continues no earlier project of its own or of others, and draws from a previous work by its supervisor (Surribas Sayago et al.) only feature-engineering inspiration —the construction of lags, the deterministic harmonics, and the idea of an exogenous branch—, without replicating its architecture: none of the figures reported here derives from that work.

Keywords: public transport, demand forecasting, daily time series, residual LSTM-XGBoost hybrid, forecast combination, out-of-fold validation

Agradecimientos

A la Universidad Internacional de Valencia y al profesorado del Máster Universitario en Big Data y Ciencia de Datos, por el marco formativo en el que se ha desarrollado este trabajo, y a la persona que ha ejercido su dirección, por la orientación metodológica y la revisión crítica a lo largo de todo el proceso.

Al Consorcio Regional de Transportes de Madrid y al proyecto Open-Meteo, cuyos datos abiertos han hecho posible el estudio empírico que sustenta esta memoria.

A mi familia y a las personas cercanas, por el apoyo sostenido durante el tiempo dedicado a este Trabajo de Fin de Máster.

1. Introducción y objetivos

El transporte público metropolitano de Madrid es una infraestructura crítica gestionada de forma coordinada por el Consorcio Regional de Transportes de Madrid (CRTM), que integra bajo un mismo marco tarifario y de gobernanza cuatro operadores de naturaleza muy distinta: Metro de Madrid, la red de superficie de la Empresa Municipal de Transportes (EMT), las concesiones de autobús interurbano por carretera y los servicios de Cercanías operados por Renfe. Anticipar con precisión la demanda diaria agregada de este sistema no es un ejercicio académico: alimenta decisiones operativas reales de asignación de flota y personal, de dimensionamiento de frecuencias y de planificación de contingencia ante episodios de demanda anómala (festivos, puentes, eventos meteorológicos extremos), y lo hace con una urgencia particular en el periodo que cubre este TFM, íntegramente posterior a la disrupción de movilidad provocada por la pandemia de COVID-19. Este proyecto adopta, precisamente, una perspectiva que la literatura revisada en el capítulo 2 aborda con mucha menor frecuencia de la que su relevancia práctica exigiría: en lugar de aislar una línea de autobús, una estación de metro o un operador concreto, se modela la demanda **sistémica** del conjunto de los cuatro operadores, preservando en todo momento el desglose por operador como variable de seguimiento y de ingeniería de características, sobre un horizonte temporal continuo de 1.310 días (1 de enero de 2023 a 2 de agosto de 2026) sin un solo hueco ni valor nulo verificado.

La motivación técnica de este TFM es, en última instancia, una pregunta de diseño de arquitecturas de predicción: ¿mejora la combinación secuencial de un modelo de dinámica temporal pura y un modelo de corrección exógena sobre residuos a los modelos de una sola etapa que la literatura ya emplea con éxito en este dominio? La arquitectura propuesta formaliza esa combinación como un híbrido residual de dos etapas: una red neuronal recurrente LSTM univariante en la primera etapa, que captura la dinámica temporal pura de la serie de demanda total, y un modelo XGBoost en la segunda etapa, que corrige el residuo $e_t = y_t - \hat{y}_t^{\text{LSTM}}$ a partir de una matriz de 161 variables exógenas de calendario y meteorología retardada, de modo que $\hat{y}^{\text{final}} = \hat{y}^{\text{LSTM}} + \hat{e}^{\text{XGBoost}}$.

Conviene adelantar aquí, con la misma honestidad con la que se presenta en el capítulo 5, el resultado central de este trabajo: la arquitectura híbrida, tal como queda diseñada e implementada, **no supera** a un modelo SARIMAX clásico ni a un XGBoost entrenado directamente sobre las mismas variables exógenas (MAE de test 234.223 frente a 163.627 y 156.121, respectivamente), y el modelo más preciso de todo el estudio resulta ser un ensamble simple de esos mismos dos componentes de una sola etapa: 139.725 de MAE de test en su variante equiponderada 50/50 y 139.681 en su variante ponderada por el inverso del MAE. Este TFM no oculta ese resultado negativo tras la presentación de su motivación inicial; al contrario, lo convierte en el objeto de estudio central, auditado con el mismo rigor metodológico con el que se evaluaría un resultado positivo: validación estrictamente cronológica, protocolo *out-of-fold* y compara-

ción exhaustiva contra controles.

1.1. Objetivo general y específicos

El **objetivo general** de este TFM es evaluar, bajo un protocolo de validación estrictamente cronológico y libre de fuga de información, si una arquitectura híbrida residual LSTM-XGBoost mejora la predicción de la demanda diaria agregada de transporte público en Madrid frente a baselines estadísticos consolidados y frente a modelos de aprendizaje automático de una sola etapa entrenados sobre las mismas variables exógenas, empleando para ello datos reales del CRTM en el periodo 2023-2026.

Este objetivo general se descompone en cinco objetivos específicos, cada uno de los cuales corresponde a una fase completa del pipeline desarrollado, aunque no todos se verifican del mismo modo. Esos cinco objetivos consolidan los diez enunciados en la propuesta inicial del trabajo: las etapas instrumentales que allí figuraban por separado, el análisis exploratorio y el análisis de importancia de variables, se integran aquí dentro del objetivo del que son medio y no fin, y se desarrollan en las secciones 3.3 y 5.5 respectivamente; ninguna se ha suprimido. OE2, OE4 y OE5 son objetivos *comparativos* (enfrentan modelos entre sí) y quedan respondidos de forma directa y explícita por las preguntas de investigación PI1-PI5 de la sección 1.2. OE1 y OE3, en cambio, son objetivos de **diseño e integridad estructural** del pipeline: construir una matriz libre de fuga y generar residuos genuinamente fuera de muestra. No admiten una comparación de desempeño entre modelos y, por tanto, no se responden mediante una pregunta de investigación, sino que se verifican de forma exhaustiva mediante la suite de tests y las garantías anti-fuga documentadas en el Anexo B, condición previa y necesaria para que las respuestas a PI1-PI5 sean fiables:

- **OE1 — Ingestión y matriz de características libre de fuga** [diseño e integridad; verificado en el Anexo B, sin PI asociada]. Construir un pipeline de ingestión que unifique las tres fuentes crudas del proyecto (CRTM, Open-Meteo y el calendario laboral municipal) en una serie diaria continua y verificada, y diseñar sobre ella una matriz de 161 variables predictoras —retardos de demanda total y por operador, medias y desviaciones móviles con desplazamiento previo estricto, armónicos de Fourier deterministas, codificación exhaustiva de calendario y meteorología retardada— sujeta a un conjunto de guardias anti-fuga verificadas por una suite de tests que crece con cada fase del proyecto (capítulo 3).
- **OE2 — Baselines estadísticos y SARIMAX seleccionado por AIC** [respondido por PI1, PI2]. Establecer una jerarquía de modelos de referencia —persistencia, persistencia estacional, medias móviles y un modelo SARIMAX con exógenas de calendario, seleccionado mediante rejilla de criterio de información de Akaike sobre 324 combinaciones de orden— evaluados bajo un protocolo de pronóstico *walk-forward* de un solo paso, de modo que la arquitectura híbrida se mida frente a un

adversario estadístico ya de por sí exigente y no frente a un baseline trivial (sección 4.2).

- **OE3 — Etapa 1: LSTM univariante con validación *out-of-fold*** [diseño e integridad; verificado en el Anexo B, sin PI asociada]. Entrenar una red LSTM univariante sobre la dinámica temporal pura de la demanda total y generar sus predicciones de entrenamiento mediante un esquema *walk-forward* expansivo de 5 bloques, de manera que ningún residuo empleado en la etapa siguiente proceda de una predicción *in-sample* (sección 4.4), en cumplimiento de una decisión de diseño que el documento de contexto técnico del proyecto fija por escrito como no negociable, con anterioridad a la obtención de cualquier resultado.
- **OE4 — Etapa 2: XGBoost sobre el residuo *out-of-fold*** [respondido por PI3, PI4]. Entrenar un modelo XGBoost sobre los residuos *out-of-fold* sanos de la etapa 1 utilizando la matriz completa de 161 variables exógenas, y contrastar el resultado del híbrido completo frente a dos controles diseñados específicamente para aislar si la etapa 1 aporta valor o introduce ruido: la propia etapa 1 en solitario y un XGBoost entrenado directamente sobre la demanda total con idéntica matriz de variables (secciones 4.5 y 4.6).
- **OE5 — Ensamble, sensibilidad e interpretación teórica del resultado** [respondido por PI5]. Explorar, mediante dos experimentos acotados y registrados por adelantado, si un repesado de los días de calendario irregular mejora al híbrido y si una combinación aritmética simple de los dos modelos de una sola etapa supera a ambos por separado, e interpretar el conjunto de resultados a la luz de la teoría de combinación de pronósticos de Granger [9], ya citada por Zhang [19] como condición necesaria para que una hibridación mejore a sus componentes (sección 5.7).

1.2. Preguntas de investigación

De los tres objetivos específicos de naturaleza comparativa (OE2, OE4 y OE5) se derivan cinco preguntas de investigación concretas, formuladas aquí con la misma redacción exacta con la que se responden en la sección 5.12:

- **PI1.** ¿Bate el híbrido a SARIMAX en validación y en test simultáneamente?
- **PI2.** ¿Bate el híbrido a `xgboost_alone`, entrenado sobre las mismas variables exógenas?
- **PI3.** ¿Reduce el híbrido específicamente el error en festivos y puentes frente a SARIMAX y `xgboost_alone`?
- **PI4.** ¿Funciona la etapa 2 como mecanismo de corrección residual, aunque el híbrido completo no supere a los modelos de una sola etapa?

- **PI5.** ¿Existe una combinación más simple que supere a ambos componentes de una sola etapa?

1.3. Planificación temporal, cronograma y ciclo de vida

El ciclo de vida seguido es una reconstrucción documentada del proceso realmente ejecutado y no un cronograma planificado a priori: el registro técnico del proyecto no anota fechas absolutas por fase, sino el cierre sucesivo de cada una verificado por una suite de tests que crece de forma monótona. El patrón responde al ciclo clásico de descubrimiento de conocimiento en bases de datos (KDD) —selección, integración, limpieza, transformación en variables predictoras, modelado, evaluación y despliegue—, aplicado de forma **incremental y verificada**: ninguna fase se dio por cerrada sin su suite en verde, y ninguna posterior reabrió una decisión anterior sin dejar constancia del motivo.

La figura 1.1 y el cuadro 1.1 documentan la secuencia con eje temporal deliberadamente relativo (Fase 0 a Fases 7-11), en coherencia con esa naturaleza reconstructiva. La última etapa del ciclo, el *despliegue*, se materializa en el cuadro de mando de la Fase 6, que reúne los cinco elementos de visualización comprometidos en la propuesta inicial —demanda real, demanda predicha, comparación entre modelos, error de predicción e importancia de variables—; el Anexo C documenta cómo ejecutarlo.

Tres bloques agrupan las once fases. La **preparación de datos** (Fases 0-2) cubre el andamiaje del repositorio, la ingestión y unificación de las tres fuentes crudas y la ingeniería de la matriz de 161 variables. El **modelado** (Fases 3-5b) concentra el grueso del trabajo experimental: baselines estadísticos, etapa 1 LSTM bajo esquema *out-of-fold*, arquitectura híbrida residual y los dos experimentos de sensibilidad acotados. La **explotación y documentación** (Fases 6-11) traduce esos artefactos en el panel interactivo, su refinamiento visual y esta misma memoria, trazada de principio a fin en `docs/LaTeX/REPORT_MAPPING.md`.

La columna de tests acumulados del cuadro 1.1 congela su recuento en el cierre de la Fase 6b (257 pruebas), último hito de modelado antes de la redacción; las suites que esta añade auditan la capa de presentación y no el pipeline, y se documentan por separado en el Anexo B.

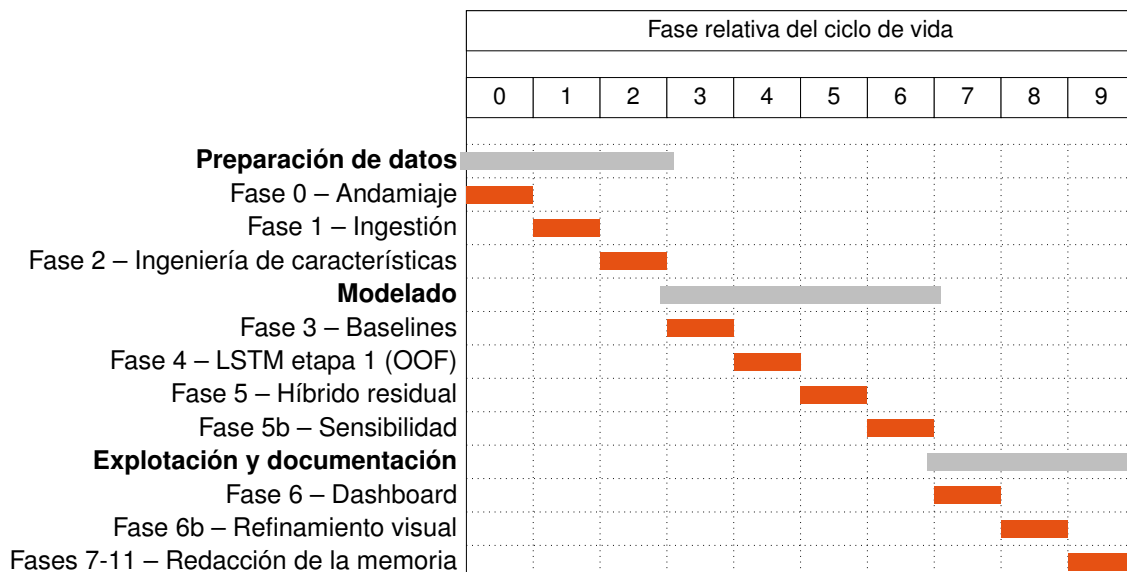


Figura 1.1: Ciclo de vida del proyecto por fases (eje relativo: 0 = Andamiaje, 9 = Redacción de la memoria; reconstrucción documentada, sin fechas de calendario).

Fase	Denominación	Entregable principal	Artefacto persistido	Tests acumulados
Fase 0	Andamiaje	Estructura src/, venv, requirements.txt, contexto técnico del proyecto	Documento de contexto técnico	0
Fase 1	Ingestión y unificación	unified_daily.parquet (1.310 x 31)	data/interim/unified_daily.parquet	30
Fase 2	Ingeniería de características anti-fuga	features_daily.parquet (1.310 x 87)	data/processed/features_daily.parquet	66
Fase 3	Particiones, promoción meteorológica y baselines	105 vars. meteorológicas retardadas, chronological_split, SARIMAX	data/processed/baseline_predictions.parquet	111
Fase 4	LSTM etapa 1 con validación OOF	Walk-forward expansivo 5 folds, lstm_stage1_final.keras	data/processed/lstm_oof_predictions.parquet	138
Fase 5	Híbrido residual	Conjunto residual 552 filas, xgboost_residual.joblib	data/processed/hybrid_predictions.parquet	160
Fase 5b	Análisis de sensibilidad pre-registrado	Experimento A (repesado 3x) y B (ensamble)	data/processed/sensitivity_comparison.parquet	177
Fase 6	Dashboard de resultados	Plotly + ipywidgets, 13 PNG	reports/figures/*.png	217
Fase 6b	Refinamiento visual	Tema VIU compartido, tablas copiables	src/evaluation/theme.py	257
Fases 7-11	Redacción de la memoria	Modularización LaTeX, figuras y tablas de memoria, REPORT_MAPPING.md	docs/LaTeX/	257

Cuadro 1.1: Fases del proyecto, entregable principal, artefacto persistido y tests acumulados (fuente: export_latex_tables.PHASES).

1.4. Estructura del documento

El resto de la memoria sigue el orden de los objetivos de la sección 1.1, en tres bloques. El primero fundamenta el trabajo: los capítulos 2 y 3 lo sitúan frente a la literatura y construyen la matriz de características (OE1). El segundo formaliza el diseño experimental de OE2 a OE5 (capítulo 4). El tercero evalúa y concluye (capítulos 5 y 6). Cierran la memoria el catálogo de variables (A), la suite de tests (B), la reproducibilidad del entorno (C), la declaración de uso de inteligencia artificial generativa (D) y los cuadros diagnósticos extendidos (E).

2. Predicción de la demanda de transporte público: dominio, técnicas y trabajos previos

Tres bloques temáticos trazan la línea argumental que justifica cada decisión de diseño tomada en el capítulo 4: el dominio de aplicación, las técnicas de modelado disponibles y las arquitecturas híbridas y de ensamble. De esa línea salen las respuestas a por qué se agregan los cuatro operadores del CRTM en lugar de una sola línea, por qué se elige una arquitectura híbrida secuencial y no un ensamble paralelo, y por qué el protocolo de validación exige un esquema *walk-forward out-of-fold* en lugar de un holdout único. La comparativa final (sección 2.4) frente a cinco trabajos de referencia sitúa con precisión el hueco exacto que ocupa este TFM.

2.1. Movilidad urbana y demanda de transporte

La demanda de transporte público en un área metropolitana como Madrid es, por naturaleza, un fenómeno multimodal: los viajeros se distribuyen entre Metro de Madrid, la Empresa Municipal de Transportes (EMT), la red de concesionarias de autobús interurbano (*carretera*) y Renfe Cercanías, y el Consorcio Regional de Transportes de Madrid (CRTM) es la única entidad que agrega y publica esta demanda a escala diaria para el conjunto del sistema. Sin embargo, la literatura que aborda la predicción de demanda en Madrid y en ciudades comparables tiende sistemáticamente a operar a una granularidad muy inferior a la de este TFM: una única línea de autobús [14], una única red de metro analizada estación por estación [6], o un conjunto multimodal tratado sin la perspectiva de demanda agregada de todo el sistema [17]. Esta elección de escala no es casual: cuanto más fina es la granularidad —una línea, una estación, una franja horaria—, más sensible es la serie al ruido idiosincrático de esa unidad concreta (una obra puntual, un desvío de servicio, un evento singular), mientras que la demanda total agregada de un sistema de transporte metropolitano completo, al sumar sobre miles de orígenes y destinos, expone con mucha mayor nitidez los patrones estructurales de fondo que interesan a un modelo predictivo: la estacionalidad semanal del ciclo laborable/fin de semana, la estacionalidad anual asociada al calendario académico y vacacional, y el efecto de los días festivos y “puente”.

Este TFM adopta precisamente esa perspectiva sistémica: modela la serie total del CRTM sin renunciar al seguimiento individual de sus cuatro operadores, que se conserva a lo largo de todo el pipeline de ingeniería de características (sección 3.4) tanto para el análisis exploratorio como para su uso como variables de retardo.

Los factores que gobiernan esta demanda agregada, caracterizados en detalle en

la sección 3.3, son de dos naturalezas bien diferenciadas y de peso muy desigual. Por un lado, la estacionalidad de calendario explica, como confirma la propia importancia de variables del capítulo 5, la inmensa mayoría de la varianza observada: el contraste laborable/sábado/domingo y la posición del día respecto a un festivo o un puente forman una señal determinista, conocida con años de antelación y sin incertidumbre de pronóstico. Por otro lado, las condiciones meteorológicas (temperatura, precipitación, viento) introducen un efecto de segundo orden, más difícil de aislar y sujeto además a la incertidumbre propia de cualquier pronóstico atmosférico, razón por la que este TFM excluye deliberadamente la meteorología del mismo día del conjunto de variables predictoras (sección 3.4.1). Ningún estudio de los citados en este capítulo combina ambos factores, calendario exhaustivo y física atmosférica retardada de forma anti-fuga, en el marco de una serie diaria agregada de todo un sistema metropolitano; esa combinación es, precisamente, uno de los ejes que la sección 2.4 formaliza como aporte diferencial.

2.2. Modelado de series temporales y aprendizaje automático tabular

Dos grandes familias de modelos sostienen la arquitectura de este TFM antes de considerar ninguna hibridación entre ellas. La primera es la de los modelos estadísticos clásicos de series temporales, fundada por la formulación ARIMA de Box y Jenkins [5]: una descomposición autorregresiva, de diferenciación e integración de medias móviles que, en su extensión estacional SARIMA y con variables exógenas (SARIMAX), sigue constituyendo siete décadas después una referencia robusta y difícilmente superable en series con estructura estacional fuerte y bien definida como la de este proyecto. El SARIMAX(2, 1, 2) $\times$ (0, 1, 2, 7) seleccionado por rejilla AIC en la sección 4.2 no es un baseline decorativo: como demuestra el capítulo 5, es uno de los dos modelos de una sola etapa más precisos de todo el proyecto, solo superado por las dos variantes de ensamble que los combinan. Entender su fuerza, el modelado explícito de la periodicidad semanal mediante diferenciación estacional de orden 7, es indispensable para interpretar por qué la arquitectura híbrida no logra desplazarlo.

La segunda familia es la del aprendizaje automático sobre datos tabulares con ingeniería de características explícita, y en particular los métodos de árboles potenciados por gradiente, cuyo exponente de referencia actual es XGBoost [8]: un algoritmo de *gradient boosting* regularizado que, a diferencia de una red neuronal, no requiere aprender representaciones latentes de las variables de entrada y resulta especialmente eficaz cuando, como en este TFM, la señal más informativa puede expresarse mediante un conjunto moderado de variables diseñadas a mano (retardos, medias móviles, términos de Fourier, indicadores de calendario). Es precisamente este algoritmo el que este proyecto emplea en dos papeles distintos: como corrector de residuos en la etapa 2 de la arquitectura híbrida (sección 4.5) y, de forma independiente, como modelo de control entrenado directamente sobre la demanda total (`xgboost_alone`, sección 4.6), cuyo desempeño resulta ser, junto con SARIMAX, el más sólido entre los modelos de una sola

etapa de todo el estudio.

Sea cual sea la familia de modelo empleada, la validez de su evaluación depende de un protocolo de partición y validación coherente con la naturaleza secuencial de los datos. Bergmeir y Benítez [4] y, más recientemente, Cerqueira et al. [7] demuestran empíricamente que los esquemas de validación cruzada aleatoria estándar, válidos en datos i.i.d., producen estimaciones de error sistemáticamente optimistas y no comparables entre modelos cuando se aplican a series temporales, precisamente porque permiten que información del futuro contamine, a través del reordenamiento de los folds, el ajuste de observaciones pasadas. Este resultado es la base teórica directa del esquema *walk-forward* expansivo en 5 bloques que genera las predicciones *out-of-fold* de la etapa 1 LSTM (sección 4.4) y de la partición estrictamente cronológica 70/15/15 que gobierna todo el proyecto (sección 3.5): ninguna de las dos decisiones es una convención arbitraria, sino la aplicación directa de un resultado metodológico contrastado en la literatura de evaluación de pronósticos.

Las métricas de error empleadas en todo el proyecto (MAE, RMSE, MAPE y R^2) y los criterios generales de buenas prácticas en la construcción de baselines siguen, asimismo, el tratamiento de referencia de Hyndman y Athanasopoulos [11]. Se reporta el RMSE en lugar del error cuadrático medio sin raíz por expresar la misma información en las unidades del propio objetivo y ser, por tanto, directamente comparable con el MAE.

2.3. Arquitecturas híbridas y de ensamble

Los antecedentes de combinar más de un modelo para predecir una misma serie temporal distinguen dos estrategias que la literatura trata a menudo como intercambiables y que este TFM mantiene deliberadamente separadas: la hibridación secuencial por corrección de residuos, y el ensamble paralelo de modelos independientes.

El antecedente directo de la primera estrategia es el trabajo seminal de Zhang [19], que formula una serie temporal como la suma de una componente lineal L_t y una componente no lineal residual N_t , de modo que $y_t = L_t + N_t$; ajusta un modelo ARIMA para estimar $\hat{L}_t$, calcula el residuo $e_t = y_t - \hat{L}_t$ y entrena una red neuronal para modelar e_t a partir de sus propios rezagos, combinando finalmente ambas predicciones como $\hat{y}_t = \hat{L}_t + \hat{N}_t$. Esta formulación es, sin ambigüedad, el precedente conceptual directo de la arquitectura híbrida evaluada en este TFM ($\hat{y}^{\text{final}} = \hat{y}^{\text{LSTM}} + \hat{e}^{\text{XGBoost}}$, sección 4), pero las diferencias de diseño respecto al esquema original de Zhang son sustanciales y deliberadas, no accidentales, y conviene enunciarlas con precisión en lugar de diluirlas en una afirmación genérica de similitud. Primero, el **orden de las etapas está invertido**: Zhang ajusta primero el componente lineal (ARIMA) y corrige después con una red neuronal [10]; este TFM ajusta primero el componente no lineal (LSTM) y corrige después con un modelo de árboles (XGBoost), una inversión motivada por el hecho de que en este proyecto la componente más difícil de modelar no es la dinámica lineal de la serie sino precisamente la estructura de calendario, que un modelo de árboles captura de forma nativa y una red recurrente univariante no puede ver en absoluto. Segundo, el **me-**

canismo de la etapa de corrección es distinto: en Zhang, la red neuronal modela el residuo a partir de sus propios rezagos, $e_t = f(e_{t-1}, \dots, e_{t-n})$, un esquema univariante; en este TFM, XGBoost modela el residuo a partir de 161 variables exógenas (calendario, meteorología retardada, armónicos de Fourier, rezagos de operador) y no de los rezagos del propio residuo, de modo que la etapa 2 introduce información nueva al sistema y no se limita a extrapolar la autocorrelación de su propio error.

Tercero, y más relevante para la interpretación de los resultados del capítulo 5: los residuos que alimentan la segunda etapa en Zhang son **residuos *in-sample*** del modelo ARIMA ya ajustado, mientras que este TFM impone la generación estrictamente *out-of-fold* de esos residuos mediante el esquema *walk-forward* de 5 folds descrito en la sección 4.4. Es una exigencia deliberadamente más estricta que tiene un coste medible: el 38 % de las filas de entrenamiento originales (889 a 552) se descartan por no disponer de una predicción genuinamente fuera de muestra. Esta tercera diferencia es la que impide, por diseño, cualquier comparación directa entre el resultado positivo que Zhang reporta para su híbrido y el resultado negativo de este TFM: son experimentos con un rigor de validación distinto sobre series de naturaleza distinta, y el capítulo 5 evalúa el suyo bajo el estándar más exigente de los dos.

La segunda estrategia de combinación, el ensamble de modelos independientes que finalmente resulta ser el mejor modelo de todo este TFM (sección 5.7.2), se apoya en un cuerpo teórico distinto: el marco general de generalización apilada (*stacked generalization*) de Wolpert [18], del que el ensamble de este proyecto es un caso particular extremadamente simple —un meta-modelo de combinación fijo (la media aritmética o ponderada) en lugar de un meta-modelo aprendido—, y la condición teórica precisa que determina cuándo esa combinación mejora a sus componentes, formulada por Granger [9] en su revisión de dos décadas de literatura de combinación de pronósticos: una combinación solo puede superar a sus partes cuando los modelos combinados son individualmente subóptimos y, de forma crucial, se basan en conjuntos de información distintos entre sí.

Bates y Granger [3] formalizan además la magnitud exacta de esa mejora: para dos pronósticos con varianza de error igual σ^2 y correlación ρ entre sus errores, la varianza del error de su media aritmética es $\sigma^2(1 + \rho)/2$, de modo que cuanto menor es ρ , cuanto menos correlacionados están los errores de los dos modelos combinados, mayor es la reducción de varianza obtenida, un resultado que Perrone y Cooper [15] extienden al contexto de comités de redes neuronales y que este TFM verifica empíricamente sobre sus propios datos en la sección 5.8.

Esta condición de Granger es exactamente la pieza teórica que conecta este capítulo con el hallazgo central negativo del capítulo 5 (sección 5.6.1): la arquitectura híbrida de este TFM encadena dos etapas que observan, en última instancia, el mismo conjunto de información, el pasado de la serie `total`, mientras que el ensamble ganador combina SARIMAX y `xgboost_alone`, dos modelos que fallan en subconjuntos de días sistemáticamente distintos (el primero en laborables ordinarios, el segundo en fin de semana y festivos) y que, por tanto, sí satisfacen la condición de información distinta que la hibri-

dación secuencial incumple. Makridakis et al. [12, 13], a partir de las competiciones M4 y M5 y de la constatación reiterada en ambas de que métodos estadísticamente simples y combinaciones de modelos superan con frecuencia a arquitecturas individuales más complejas (especialmente en muestras de tamaño moderado como las 1.310 observaciones diarias de este proyecto), proporcionan el contexto empírico a gran escala que hace de este patrón un resultado esperable y no una anomalía específica de esta implementación.

Por último, el diseño de la matriz de 161 variables exógenas de este TFM —en particular los rezagos multiescala de demanda y meteorología y los términos armónicos de Fourier para la estacionalidad semanal y anual (sección 3.4)— toma como inspiración metodológica el trabajo de Surribas-Sayago et al. [16] sobre predicción de radiación solar difusa mediante una arquitectura de fusión paralela CNN+LSTM+MLP con una rama auxiliar exógena. Es indispensable, sin embargo, no diluir esta cita en una afirmación de arquitectura compartida: tal y como queda fijado por escrito en el documento de contexto técnico del proyecto, ese trabajo se cita **exclusivamente** como fuente de inspiración para la construcción de retardos estructurados y de una base armónica de Fourier, nunca como la arquitectura replicada por este proyecto. La fusión paralela de ramas concurrentes de Surribas-Sayago et al. y la corrección secuencial de residuos de este TFM son diseños conceptualmente distintos que no deben confundirse ni en el código ni en esta memoria.

2.4. Análisis comparativo y diferenciación metodológica

La tabla 2.1 sitúa este TFM frente a los cinco trabajos discutidos en las secciones anteriores sobre cinco ejes comparables: ámbito y granularidad, horizonte temporal, matriz de variables exógenas, protocolo de validación y aporte empírico.

Estudio	(a) Ámbito y granularidad	(b) Horizonte temporal	(c) Matriz exógena	(d) Protocolo de validación	(e) Aporte empírico
Monje et al. (2022)	Una línea de autobús (EMT línea 1), solo franja de tarde	26 meses pre-pandemia (ene. 2015 - feb. 2017)	Festivo binario + calendario (mes, día de semana); cero meteorología	Holdout cronológico simple 80/20, una sola ejecución, sin CV	Reporta el ganador (LSTM $R^2=0,89$) sin MAE/RMSE/MAPE absolutos
Toqué et al. (2017)	Transporte multimodal por estación (París)	Logs de billeteaje, 2017	No especificada en la fuente disponible	No especificado en la fuente disponible	Comparativa de métodos de ML para flujos multimodales, sin arquitectura híbrida residual
Cardozo et al. (2012)	Entradas por estación del Metro de Madrid, corte transversal espacial	Sin dimensión temporal (regresión espacial)	Entorno construido; sin calendario ni clima	Ajuste espacial in-sample	Regresión espacial por estación; no aborda predicción temporal
Zhang (2003)	Serie univariantes ajenas al transporte (manchas solares, linceas, GBP/USD)	288 / 114 / 731 observaciones	Ninguna (residuo modelado a partir de sus propios rezagos)	Holdout único, residuos ARIMA in-sample (sin OOF)	El híbrido ARIMA+red neuronal bate a ambos componentes en las tres series
Surribas-Sayago et al. [ficha sin verificar]	Radiación solar difusa (dominio meteorológico, no transporte)	No verificado	Retardos y términos de Fourier (fuente de inspiración de la ingeniería de variables de este TFM, no de su arquitectura)	No verificado	Arquitectura paralela CNN+LSTM+MLP; citada solo por su ingeniería de características
Este TFM	Demanda diaria agregada de los cuatro operadores del CRTM (Madrid)	1.310 días continuos 2023-2026, cero huecos, cero nulos, integralmente post-pandemia	161 variables: 105 meteorológicas físicas retardadas (lags 1/2/3/7 + media móvil 7), 6 términos de Fourier, 55 días puente detectados; meteorología del mismo día explícitamente prohibida	Partición cronológica 70/15/15 de frontera única; walk-forward OOF en 5 bloques expansivos; SARIMAX walk-forward de un paso sin reestimación; escalado ajustado solo en train; 257 tests fijando las reglas anti-fuga	Auditoría del fallo: el híbrido no bate a SARIMAX (+70.597 MAE test) ni a XGBoost solo (+78.103); el mejor modelo es un ensamble 50/50 (MAE 139.725, -10,5% vs. el mejor componente)

Cuadro 2.1: Comparativa metodológica frente a la literatura previa, sobre cinco ejes (ámbito, horizonte, variables exógenas, protocolo de validación, aporte empírico).

Los cuatro primeros ejes se leen directamente en el cuadro y no se reproducen aquí uno a uno; conviene destacar de ellos lo que sitúa a este trabajo fuera del rango de los cinco estudios de referencia. En **ámbito** (a) es el único que agrega los cuatro operadores del CRTM en una serie de sistema completo, frente a una sola línea de autobús en Monje et al. [14], a una red de metro estación por estación en Cardozo et al. [6] o a series ajenas al transporte en Zhang [19]. En **horizonte** (b), sus 1.310 días continuos (2023-2026) son íntegramente posteriores a la pandemia, a diferencia de los 26 meses prepandemia de Monje et al., cuya estructura de demanda quedó alterada de forma permanente en 2020-2021. En **matriz exógena** (c), ningún otro combina una base meteorológica físicamente retardada de 105 variables con una codificación de calendario que detecta 55 días puente; la meteorología del mismo día queda excluida para no asumir un pronóstico atmosférico perfecto en inferencia, decisión resuelta antes de entrenar ningún modelo. En **protocolo de validación** (d) es el único que combina partición cronológica de frontera única, esquema *walk-forward out-of-fold* de 5 bloques expansivos para la etapa 1 y pronóstico SARIMAX *walk-forward* de un paso sin reestimación, frente al holdout simple de Monje et al. o los residuos *in-sample* de Zhang; las 257 pruebas que fijan las reglas anti-fuga del capítulo 3 (de un total de 470, Anexo B) no tienen equivalente declarado en ninguno de los cinco.

El eje decisivo es el **(e) aporte empírico**, porque es donde este TFM no reporta una mejora sino la auditoría deliberada de un fallo. A diferencia de Zhang, cuyo híbrido ARIMA-red neuronal supera a ambos componentes en las tres series que evalúa, la arquitectura híbrida LSTM-XGBoost de este proyecto **no** bate ni a SARIMAX ni a `xgboost_alone` (capítulo 5), y el modelo finalmente más preciso es un ensamble de esos mismos dos componentes de una sola etapa (MAE de test 139.725 en su variante equiponderada 50/50 y 139.681 en la ponderada por el inverso del MAE). Leído a la luz de la condición de Granger desarrollada en la sección 2.3, y no como una simple discrepancia frente a Zhang, ese contraste es en sí mismo la aportación: una auditoría con validación *out-of-fold* estricta de las condiciones bajo las cuales la hibridación secuencial de un componente no lineal y uno exógeno *no* mejora sobre sus partes, frente a aquellas bajo las cuales un ensamble paralelo de modelos con información parcialmente descorrelacionada sí lo hace.

Es precisamente la ausencia de un antecedente directo que audite un resultado híbrido negativo bajo un protocolo igualmente estricto, sobre una serie de demanda de transporte público real, agregada y multimodal, la que constituye el *research gap* que este TFM ocupa: no la ausencia de arquitecturas híbridas en la literatura —que existen, y Zhang es su fundamento teórico—, sino la ausencia de una evaluación de cuándo y por qué fallan frente a alternativas más simples.

De ese hueco se derivan las hipótesis de trabajo, que esta memoria no formula aquí por segunda vez: están enunciadas como las cinco preguntas de investigación PI1–PI5 de la sección 1.2, con la redacción exacta con la que se responden en la sección 5.12, y cada una lleva asociado un criterio de decisión fijado por escrito antes de observar la métrica correspondiente.

3. Fuentes de datos, integración e ingeniería de características

Este capítulo documenta la cadena completa que transforma tres ficheros crudos, heterogéneos en formato y en semántica, en la matriz de 161 variables sobre la que se entrenan los modelos de los capítulos 4 y 5. En cada etapa se hace explícita la regla anti-fuga que la gobierna, puesto que la integridad de esa cadena es la condición de validez de todo resultado reportado.

3.1. Fuentes

El proyecto integra tres fuentes crudas, todas ellas de frecuencia diaria y con un horizonte temporal común de 1.310 observaciones continuas, sin huecos y sin valores nulos, comprendido entre el 1 de enero de 2023 y el 2 de agosto de 2026. Los tres ficheros se conservan sin modificar en `data/raw/` y se resumen en la tabla 3.1.

La primera fuente procede del Consorcio Regional de Transportes de Madrid (CRTM), el organismo público responsable de la coordinación tarifaria e informativa del transporte colectivo en la Comunidad de Madrid, y reporta la evolución diaria de validaciones agregadas por operador: Metro de Madrid, EMT (autobús urbano), las concesionarias de transporte interurbano por carretera y Renfe Cercanías, junto con el total sumado de los cuatro. El fichero se distribuye como un libro Excel con dos hojas (mensual y diaria, de la que solo la segunda se emplea) cuyo maquetado real difiere del descrito en el brief original del proyecto: la cabecera no ocupa la primera fila, sino la segunda, de modo que una lectura ingenua con `header=2` en lugar de `header=1` consume silenciosamente la primera observación (2023-01-01) como si fuera parte de la cabecera.

La columna de fechas, además, carece de nombre propio en el fichero de origen, la primera columna contiene un marcador suelto sin significado analítico, la octava columna está vacía y la novena recoge una nota textual que advierte del carácter *provisional* de las cifras de demanda; esa advertencia se traslada íntegra a las limitaciones del proyecto (6.2). Las cuatro columnas de operador se conservan explícitamente en el conjunto unificado y en la matriz de características, aunque el objetivo de modelado sea exclusivamente la columna `total`: no se trata de variables redundantes a descartar, sino de una desagregación que sostiene el análisis exploratorio de la sección 3.3 y que podría alimentar líneas de trabajo futuras sobre demanda por modo de transporte.

La segunda fuente es una serie meteorológica diaria obtenida de Open-Meteo para la localización de Madrid (40,386642° N, 3,676086° O, 666 m de altitud), con 22 variables que cubren temperatura, temperatura aparente, precipitación, humedad relativa, viento, presión a nivel del mar, radiación de onda corta y código de tiempo (WMO). La propuesta

inicial preveía recurrir a la Agencia Estatal de Meteorología (AEMET); se optó finalmente por Open-Meteo porque su producto de reanálisis cubre la ventana completa 2023-2026 para un único punto geográfico sin huecos ni cambios de estación de medida, lo que evita tener que imputar o empalmar series. Esa operación, sobre variables que después se retardan y promedian, habría introducido una fuente de error difícil de acotar. La misma familia de API se retoma en la sección 6.3 como vía natural para incorporar pronóstico operativo.

El fichero incorpora dos filas de metadatos de cabecera seguidas de una línea en blanco antes de la fila con los nombres de columna reales, y esos metadatos declaran una zona horaria Europe/Berlin en lugar de Europe/Madrid; ambas comparten horario de verano centroeuropeo (CEST) en la práctica totalidad del año, por lo que el efecto sobre los agregados diarios se consideró y descartó como fuente de sesgo relevante durante la ingestión, sin darlo por supuesto a priori. Las unidades de medida viajan incrustadas en el propio nombre de cada columna (por ejemplo, `temperature_2m_mean` (°C) o `shortwave_radiation_sum` (MJ/m²)) y se normalizan mediante una expresión regular que las separa del nombre canónico de la variable.

La tercera fuente es el calendario laboral de referencia para la Comunidad de Madrid, distribuido como un CSV separado por punto y coma y codificado en UTF-8 con marca de orden de bytes (BOM), con las fechas expresadas en formato dd/mm/aaaa y analizadas mediante un patrón explícito en lugar de una inferencia automática de orden día/mes, para evitar ambigüedades sistemáticas. El calendario distingue el día de la semana (en minúsculas y sin tildes, de modo que `miercoles` o `sabado` no deben compararse contra sus formas acentuadas), el tipo de día (`laborable/sabado/domingo/festivo`) y, cuando procede, el tipo y el nombre de la festividad. El propio nombre de la columna de tipo de día declara una quinta categoría, `domingo festivo`, que en la práctica no se observó en ninguna de las 1.310 filas disponibles; esta ausencia se verificó de forma explícita durante la ingestión, en lugar de asumirse por la mera lectura del brief, y la categoría se mantiene declarada en la codificación para que, si apareciera en una actualización futura del calendario, se codifique correctamente en vez de degradarse silenciosamente a un valor nulo.

La figura 3.1 muestra la serie completa de demanda total tal como resulta de esta primera fuente, antes de cualquier transformación: establece la escala del problema (entre 1,3 y 6,6 millones de viajes diarios aproximadamente) y deja ya visible la textura semanal de sierra y la caída estacional recurrente cada mes de agosto, patrones que se analizan en detalle en la sección 3.3.

Fichero	Contenido	Formato	Origen
CRTM_Evolucion_demanda_diaria.xlsx	Demanda diaria por operador (metro, EMT, carretera, cercanías) y total	Excel, hoja 'diaria', cabecera en fila 2	https://datos.crtm.es/documents/crtm::crtm-evolucion-demanda-diaria/about
open-meteo-40.39N3.68W666m.csv	22 variables meteorológicas diarias (temperatura, precipitación, viento, presión, radiación)	CSV, cabecera real en línea 4	https://open-meteo.com/en/docs/historical-weather-api
300082-1-calendario_laboral-csv.csv	Día de la semana, tipo de día (laborable/sábado/domingo/festivo) y festividad	CSV separado por ';', UTF-8 con BOM	https://datos.madrid.es/dataset/300082-0-calendario_laboral

Cuadro 3.1: Fuentes de datos crudas (*data/raw/*), 2023-01-01 a 2026-08-02.

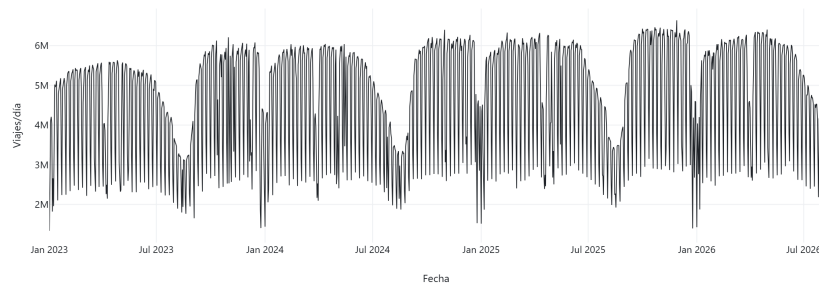


Figura 3.1: Demanda diaria total de transporte público en Madrid, serie completa.

3.2. Unificación y validación

Las tres fuentes se combinan mediante una unión interna (*inner join*) sobre la columna de fecha. Se prefiere esta estrategia frente a una unión externa por la izquierda porque esta última fabricaría filas exógenas enteramente nulas allí donde una fuente tuviera un hueco, contaminando artificialmente el indicador de calidad “cero nulos” del conjunto unificado. La contrapartida es que una unión interna oculta, por construcción, cualquier hueco de cobertura: si una fecha faltara simultáneamente en las tres fuentes, el problema no se manifestaría como una fila nula sino como una fila que sencillamente no llega a existir. Por ello la unificación no se limita a ejecutar el *join* y confiar en el resultado, sino que aplica cuatro comprobaciones de integridad temporal en cadena: en primer lugar, una detección de fechas duplicadas dentro de cada fuente individual, antes de fusionar nada; en segundo lugar, una comparación del número de filas resultante contra el número de filas de *cada* fuente por separado, de modo que ante cualquier discrepancia el proceso interrumpe la ejecución señalando explícitamente las fechas concretas que se perderían, en lugar de limitarse a reportar un recuento agregado; en tercer lugar, una

reindexación del resultado contra un rango de fechas diario completo entre el mínimo y el máximo observados, que es la única comprobación capaz de detectar un hueco común a las tres fuentes simultáneamente, precisamente el caso que la comparación anterior no puede ver; y en cuarto lugar, una verificación final de ausencia de duplicados y de valores nulos sobre el conjunto ya unificado.

Estas cuatro comprobaciones se someten además a pruebas negativas que inyectan huecos sintéticos en conjuntos de datos de prueba en memoria y confirman que cada una de ellas interrumpe la ejecución cuando corresponde, de manera que no puedan degradarse silenciosamente en verificaciones inertes con el tiempo.

El resultado se persiste en `data/interim/unified_daily.parquet` como una tabla de 1.310 filas por 31 columnas, resumidas por grupos temáticos en la tabla 3.2: la fecha, las cinco columnas de demanda (el objetivo `total` y los cuatro operadores), las veintiuna variables meteorológicas y las cuatro columnas de calendario. Sobre esta tabla se verifica, y se mantiene como invariante durante todo el proyecto, que la suma de las cuatro columnas de operador reproduce exactamente la columna `total` en las 1.310 filas, salvo por un margen de tolerancia de 10^{-6} que absorbe el ruido de redondeo introducido por fórmulas de Excel en el fichero de origen (por ejemplo, un valor de `carretera` igual a 847904,00000001 en la fila del 7 de julio de 2026, con una desviación del orden de 10^{-8} respecto al entero esperado); cualquier desviación genuinamente fraccionaria, por encima de esa tolerancia, se trata como un error de datos y detiene el proceso, ya que ensancharla sin más para acallar un futuro fallo eliminaría la capacidad de detectar una ruptura real de esa identidad.

Por último, los valores nulos que el calendario de origen deja en `holiday_type` y `holiday_name` para los días sin festividad se sustituyen deliberadamente por el valor centinela `'none'`: de este modo la invariante de “cero nulos” del conjunto unificado es honesta —cualquier nulo que aparezca en adelante señala un defecto real, no la ausencia ordinaria de festividad— y la siguiente fase de ingeniería de características recibe un nivel categórico limpio en lugar de un hueco que codificar de forma ad hoc.

Grupo	N.º columnas	Ejemplo
Fecha	1	<code>date</code>
Demanda (target + operadores)	5	<code>total</code> , <code>metro</code> , <code>emt</code> , <code>carretera</code> , <code>cercanias</code>
Meteorología	21	<code>temperature_2m_mean</code> , <code>precipitation_sum</code> , <code>wind_speed_10m_max</code>
Calendario	4	<code>day_of_week_es</code> , <code>day_type</code> , <code>holiday_type</code> , <code>holiday_name</code>

Cuadro 3.2: Esquema unificado (`data/interim/unified_daily.parquet`), 1.310 x 31.

La figura 3.2 descompone la serie total por operador y hace visible la jerarquía de escala entre ellos: Metro de Madrid concentra el volumen mayor, seguido de EMT, mientras que las concesionarias por carretera y Renfe Cercanías aportan magnitudes

sensiblemente menores pero con una textura semanal igualmente marcada. Esta descomposición no participa en el modelado del capítulo 4, cuyo único objetivo es `total`, pero justifica por qué las cuatro columnas de operador se conservan íntegras a través de todo el pipeline: son la base de este análisis exploratorio secundario y quedan disponibles para un trabajo futuro de predicción desagregada por modo de transporte.

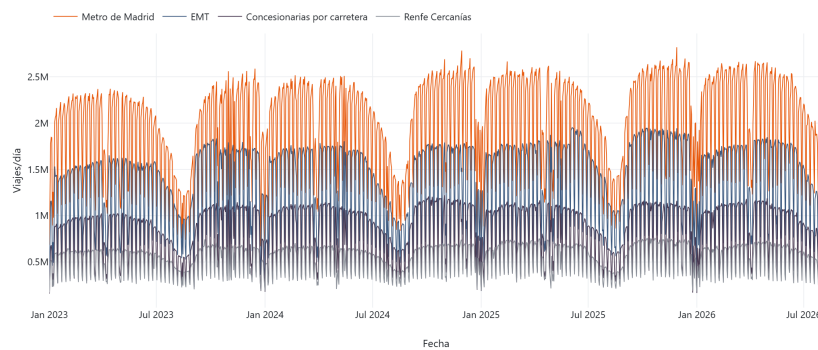


Figura 3.2: Demanda diaria por operador del CRTM (Metro, EMT, carretera, Cercanías).

3.3. Análisis exploratorio de datos

El análisis exploratorio persigue un objetivo acotado: caracterizar la estructura de estacionalidad y de dispersión de la demanda antes de diseñar sobre ella la ingeniería de características de la sección 3.4, no agotar aquí todas las posibles lecturas de los datos. La propia serie completa (figura 3.1) ya deja ver, a simple vista y de forma repetida en cada uno de los tres años y medio cubiertos, dos regularidades anuales: un desplome pronunciado y sostenido durante el mes de agosto —coincidente con el periodo vacacional de mayor intensidad en Madrid, en el que buena parte de la actividad laboral y educativa que sostiene la demanda de días laborables se suspende— y una recuperación progresiva durante septiembre que culmina en un máximo relativo hacia octubre y noviembre, una vez restablecida por completo la actividad laboral y escolar tras el verano.

Ese patrón se cuantifica en la figura 3.4, donde la demanda media de agosto se sitúa en torno a los 3 millones de viajes diarios, sensiblemente por debajo de cualquier otro mes del año, mientras que octubre alcanza el máximo del calendario anual con una media próxima a los 5,2 millones; los meses de invierno y primavera se mantienen en una meseta intermedia, entre 4,5 y 5 millones, con una dispersión (barras de error de desviación típica) que es sistemáticamente mayor en los meses que combinan más irregularidades de calendario —festivos nacionales y autonómicos, semana de Navidad, Semana Santa de fecha variable— que en los meses centrales del curso.

La segunda regularidad, de periodicidad semanal y de magnitud mayor que la anual, se recoge en la figura 3.3. Los cinco días laborables presentan medianas muy

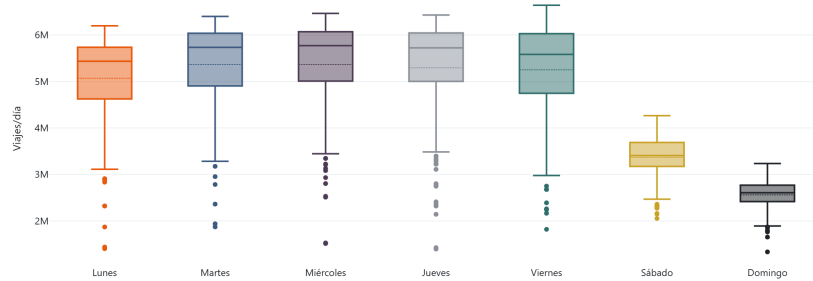


Figura 3.3: Estacionalidad semanal de la demanda total (diagrama de caja por día de la semana).

próximas entre sí, en el entorno de los 5 a 5,5 millones de viajes diarios, con una cola inferior de valores atípicos que corresponde a los festivos que caen en día laborable; el sábado desciende a una mediana cercana a los 3,4 millones y el domingo, el día de menor demanda de la semana, se sitúa alrededor de los 2,6 millones. La transición entre el régimen laborable y el de fin de semana implica, por tanto, un salto de aproximadamente 2,5 millones de viajes diarios dos veces por semana, una magnitud que resulta clave para interpretar correctamente los baselines del capítulo 4: un modelo de persistencia ingenuo ($\hat{y}_t = y_{t-1}$) queda sistemáticamente mal calibrado en cada transición viernes-sábado y domingo-lunes, precisamente porque ignora esta estructura semanal tan pronunciada.

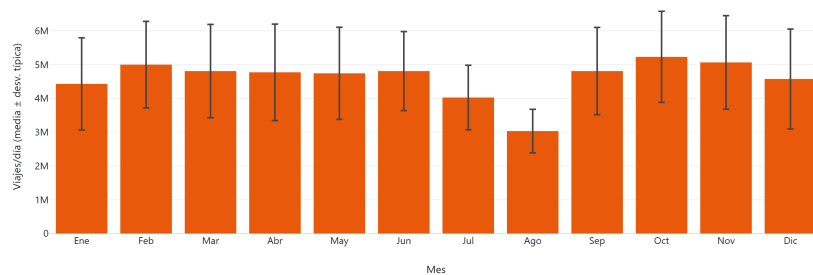


Figura 3.4: Estacionalidad anual de la demanda total (media mensual, 2023-2026).

Por último, la figura 3.5 cruza la demanda total con la temperatura media diaria, coloreando cada observación por tipo de día. El resultado es informativo precisamente por lo que no muestra: dentro de cada franja de tipo de día la nube de puntos se mantiene aproximadamente horizontal, sin una relación monótona clara entre temperatura y demanda, mientras que la separación vertical entre franjas (laborable frente a sábado, domingo y festivo) domina visualmente el gráfico. Esto sugiere que el **calendario, no la meteorología, es el factor exógeno de primer orden** sobre la demanda agregada, y que la meteorología, si aporta señal, lo hace como un efecto secundario que matiza el nivel de cada régimen de calendario más que como un determinante independiente. Esta

lectura motiva la decisión, desarrollada en la sección 3.4.1, de incorporar la meteorología como bloque de variables exógenas retardadas y nunca como sustituto del calendario.

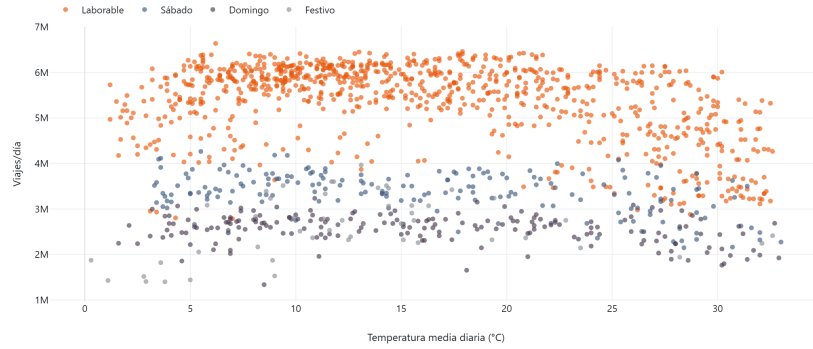


Figura 3.5: Demanda total frente a temperatura media diaria, por tipo de día (EDA; la meteorología del mismo día no entra en el modelo).

3.4. Ingeniería de características

A partir del esquema unificado de la sección 3.2 se construye una matriz de 161 variables de modelado, todas ellas identificadas mediante el prefijo `feat_` y recuperables mecánicamente mediante la función `feature_columns()` en lugar de mantenerse en una lista mantenida a mano. Es una decisión deliberada para que el conjunto de variables no pueda derivar silenciosamente respecto del código que las genera. La tabla 3.3 resume los seis grupos resultantes, cuyo diseño matemático se detalla a continuación.

El primer grupo son los retardos de la propia demanda total, $\text{lag}_k(\text{total})_t = \text{total}_{t-k}$ para $k \in \{1, 2, 3, 7, 14, 21, 28\}$: siete variables que cubren desde la inercia de un día para otro hasta la memoria calendario de cuatro semanas, seleccionada porque el lag 28 recoge el mismo día de la semana cuatro veces atrás, bajo el mismo régimen semanal que la sección 3.3 identificó como dominante. El mismo esquema de siete retardos se aplica a cada una de las cuatro columnas de operador, aportando 28 variables adicionales: la reparto modal de ayer es información legítimamente disponible en el momento de la predicción y conserva señal real, a diferencia del reparto modal del propio día que se discute más abajo. A estas se suman cuatro estadísticos móviles de la demanda total (media y desviación típica sobre ventanas de 7 y 28 días), y seis términos de Fourier deterministas que codifican la estacionalidad semanal ($k = 1$) y anual ($k = 1$ y $k = 2$) mediante pares seno-coseno:

$$\sin\left(\frac{2\pi kt}{T}\right), \quad \cos\left(\frac{2\pi kt}{T}\right),$$

con $T = 7$ para el término semanal y $T = 365,25$ para los dos términos anuales, y t medido en días desde una época fija, `FOURIER_EPOCH = 2023-01-01`, deliberadamente

distinta de la fecha mínima observada en cada ejecución: fijar la época evita que la llegada de datos futuros modifique retroactivamente la codificación de una fecha ya existente.

El último grupo interno, de once variables, cubre el calendario mediante *one-hot encoding* exhaustivo del tipo de día y del tipo de festividad, incluyendo explícitamente el nivel centinela 'none' y la categoría de cero observaciones domingo festivo, de modo que una fila con todo ceros sería un error de codificación detectable y no una ambigüedad entre "categoría base" y "fallo silencioso". Se añaden un indicador binario de fin de semana y un indicador de "día puente": un día laborable inmediatamente anterior o posterior a un festivo o domingo festivo, construido observando también el calendario del día siguiente. Esta última mirada hacia adelante no constituye fuga de información porque el calendario oficial de festivos español se conoce con más de un año de antelación y es, por tanto, dato disponible en el momento de la predicción; el procedimiento detectó 55 días puente en todo el periodo, todos ellos laborables por definición.

Grupo	N.º variables	Descripción
Retardos de la demanda total	7	lags 1, 2, 3, 7, 14, 21, 28
Retardos de operadores	28	4 operadores x 7 retardos
Medias/desv. móviles de la demanda	4	media y desv. típica, ventanas 7 y 28, shift-first
Términos de Fourier	6	semanal k=1, anual k=1 y k=2
Calendario	11	day_type (5), holiday_type (4), is_weekend, is_bridge_day
Meteorología retardada	105	21 variables x (lags 1/2/3/7 + media móvil 7)
Total	161	

Cuadro 3.3: Grupos de variables del modelo, 161 en total (`feature_columns()`).

Dos reglas anti-fuga gobiernan la construcción de este bloque de variables y se verifican mediante pruebas automatizadas específicas. La primera exige que todo retardo o estadístico móvil derivado de `total` o de cualquier columna de operador aplique un desplazamiento de un día, `shift(1)`, *antes* de cualquier operación de ventana móvil: es decir, $\text{roll}_w(s.\text{shift}(1))_t$ y nunca $\text{roll}_w(s)_t.\text{shift}(1)$ como corrección a posteriori. Para una ventana móvil trailing ambos órdenes de operación son matemáticamente equivalentes, de modo que la regla no corrige aquí ningún error numérico observado; se exige de todos modos porque su corrección no depende de esa equivalencia particular, que deja de cumplirse en cuanto la ventana se centra (`center=True`), y porque una ventana sin desplazar en absoluto sí constituye una fuga real y directa: en dicho caso, la media móvil en el instante t incorporaría el propio valor y_t que se pretende predecir.

La segunda regla es un guardián de fuga contemporánea sobre las columnas de operador: puesto que $\text{metro}_t + \text{emt}_t + \text{carretera}_t + \text{cercanias}_t = \text{total}_t$ de forma exacta (sección 3.2), cualquier columna de operador del *mismo* día no es una variable predictiva, sino una reformulación aritmética del propio objetivo, y su presencia en la matriz de

modelado haría que un modelo alcanzara un ajuste casi perfecto en entrenamiento para colapsar en inferencia, momento en el que el reparto modal del día en curso no puede conocerse todavía. Este guardián se aplica dos veces: sobre la salida de cada módulo de retardos individualmente y de nuevo sobre el fichero final ensamblado, que es el punto donde una columna mal etiquetada terminaría aterrizando realmente en la matriz.

Los términos de Fourier constituyen, dentro de este esquema, la única excepción autorizada a la prohibición general de calcular ingeniería de características sobre el conjunto completo antes de la partición: a diferencia de una media móvil o de un escalador, cuyo valor depende de observaciones de y y, por tanto, estima un parámetro que solo debería aprenderse en entrenamiento, el valor de un término de Fourier en la fecha t es una función cerrada de t que no observa el objetivo en ningún momento, de modo que calcularlo sobre el rango completo es idéntico, variable a variable, a calcularlo por separado sobre cada partición.

Promoción meteorológica: solo variables retardadas

La sección 3.3 dejó abierta, más que resuelta, la pregunta de si la meteorología debía incorporarse a la matriz de modelado observada en tiempo real o bajo alguna forma de retardo. Este proyecto resuelve la cuestión de manera conservadora: las 21 variables meteorológicas del esquema unificado se promocionan a variables de modelado exclusivamente en forma retardada, combinando los retardos de 1, 2, 3 y 7 días con una media móvil de 7 días (también construida con desplazamiento previo, por la misma regla anti-fuga de la sección anterior), lo que produce $21 \times 5 = 105$ variables nuevas y eleva la matriz de 56 a 161 columnas. La meteorología del propio día de la predicción no entra nunca en el modelo, ni siquiera como variable adicional junto a las retardadas, y esta prohibición se verifica mediante un guardián dedicado (`_assert_no_same_day_weather`) que se reejecuta sobre el conjunto final ensamblado. La razón es que la meteorología observada del mismo día no es, en rigor, un dato disponible en el momento operativo de la predicción, sino una observación posterior a esta; utilizarla sin retardo equivaldría a asumir de forma implícita un pronóstico meteorológico perfecto, un sesgo optimista que además se concentraría precisamente en los días atípicos (temporales, olas de calor) en los que un pronóstico meteorológico real es menos fiable y en los que el efecto sobre la demanda sería, presumiblemente, mayor.

El conjunto de retardos elegido para la meteorología, $\{1, 2, 3, 7\}$, es deliberadamente más corto que el empleado para la demanda total, $\{1, 2, 3, 7, 14, 21, 28\}$, porque ambas series obedecen a mecanismos de memoria distintos: la autocorrelación de la demanda a 28 días está gobernada por el calendario (el mismo día de la semana, cuatro semanas atrás, bajo un régimen semanal recurrente), mientras que la autocorrelación atmosférica carece de un mecanismo equivalente y decae en la práctica a lo largo de una escala sinóptica de tres a siete días; extender los retardos meteorológicos a 14, 21 o 28 días habría añadido 63 columnas prácticamente redundantes con los términos de Fourier anuales, que ya capturan de forma más limpia la componente puramente estacional del

clima. Queda como línea de trabajo pendiente, y explícitamente fuera del alcance de los resultados principales de este TFM (6.2), la construcción de una variante de sensibilidad con meteorología del mismo día sin retardar, que solo tendría sentido reportar como una cota superior bajo el supuesto idealizado de un pronóstico meteorológico perfecto, y nunca como modelo de referencia.

3.5. Particiones y fuga de datos

La totalidad de los modelos entrenados en los capítulos 4 y 5 (baselines estadísticos, LSTM, XGBoost y sus combinaciones) comparten una única partición cronológica 70/15/15, calculada por una sola función, `chronological_split`, que actúa como fuente única de verdad: ningún modelo recalcula sus propias fronteras temporales de forma independiente, precisamente para que un desacuerdo de un solo día entre dos modelos sobre dónde empieza el conjunto de test no pueda invalidar silenciosamente su comparación. La partición resultante, resumida en la tabla 3.4, reserva 917 días (70,0 %) para entrenamiento, desde el 1 de enero de 2023 hasta el 5 de julio de 2025; 196 días (15,0 %) para validación, hasta el 17 de enero de 2026; y 197 días (15,0 %) para test, hasta el final de la serie disponible, el 2 de agosto de 2026.

El cálculo de estas fronteras exige dos precauciones que, de omitirse, introducirían errores sutiles y difíciles de detectar a posteriori. La primera es puramente numérica: la expresión $0,70 \times 1310$ evalúa en aritmética de punto flotante a 916,9999... (esto es, 916,9 hasta la precisión de la máquina), de modo que un truncamiento ingenuo mediante `int()` produciría un conjunto de entrenamiento de 916 días en lugar de los 917 que resultan de $\lfloor 0,70 \times 1310 \rfloor$; la implementación añade una tolerancia $\varepsilon = 10^{-9}$ antes de truncar, precisamente para blindarse frente a este tipo de error de redondeo binario. La segunda precaución es de diseño: las fronteras se resuelven sobre fracciones acumuladas ($\lfloor 0,70n \rfloor$ para el final de entrenamiento, $\lfloor 0,85n \rfloor$ para el final de validación) en lugar de aplicar el porcentaje de forma independiente a cada partición, de modo que los tres bloques resultantes encajen exactamente, sin solapamientos ni huecos de un día entre ellos que un redondeo partición por partición sí podría introducir.

Partición	Inicio	Fin	Días	Porcentaje
Train	2023-01-01	2025-07-05	917	70,0
Val	2025-07-06	2026-01-17	196	15,0
Test	2026-01-18	2026-08-02	197	15,0

Cuadro 3.4: Partición cronológica 70/15/15 (`src.utils.splits.chronological_split`).

Sobre esta partición actúa, además, una política de calentamiento (*warm-up*) derivada directamente de la ingeniería de características de la sección 3.4: las 28 primeras filas del conjunto de entrenamiento (2023-01-01 a 2023-01-28) contienen valores nulos en las variables que dependen del retardo o de la ventana móvil más larga, 28 días, y se descartan del entrenamiento, reduciéndolo de 917 a 889 filas efectivas; los conjun-

tos de validación y de test, al situarse muy por delante de esa ventana inicial, no se ven afectados y conservan sus 196 y 197 días íntegros. Esta política se aplica mediante una función, `apply_warmup_policy`, que *interrumpe* la ejecución si el calentamiento necesario llegara a alcanzar la partición de validación o de test, en lugar de recortar silenciosamente el periodo de evaluación: un conjunto de evaluación acortado sin aviso produciría métricas que dejarían de ser comparables entre modelos y entre reejecuciones del pipeline, un riesgo que se considera preferible convertir en un fallo explícito y temprano.

4. Arquitectura híbrida residual LSTM-XGBoost para demanda diaria agregada

Este capítulo describe la arquitectura de modelado propuesta y el protocolo experimental que la sustenta, sobre la matriz de 161 variables construida en el capítulo 3. La propuesta central es un híbrido residual secuencial de dos etapas: una red LSTM univariante (etapa 1) que modela la dinámica temporal de la propia demanda, y un modelo XGBoost (etapa 2) que corrige, a partir de variables exógenas de calendario y meteorología, el residuo que la etapa 1 deja sin explicar. Formalmente, si $\hat{y}_t^{\text{LSTM}}$ denota la predicción de la etapa 1 en el instante t , el residuo que alimenta la etapa 2 se define como

$$e_t = y_t - \hat{y}_t^{\text{LSTM}},$$

y la predicción final del sistema combina ambas etapas por suma directa,

$$\hat{y}_t^{\text{final}} = \hat{y}_t^{\text{LSTM}} + \hat{e}_t^{\text{XGBoost}}.$$

Conviene precisar en qué difiere este diseño del esquema híbrido ARIMA-red neuronal de Zhang [19], que es su referente más directo en la literatura y al que se volverá al interpretar el resultado central del capítulo 5 (5.6.1). En primer lugar, el orden de las etapas está invertido: aquí es la red neuronal la que modela primero la componente no lineal de la serie y un modelo basado en árboles el que corrige después, no al revés. En segundo lugar, la etapa 2 de este TFM no modela el residuo como una serie temporal autorregresiva sobre sus propios retardos, sino como una función de 161 variables exógenas (calendario, meteorología retardada, términos de Fourier) construidas de forma independiente de la dinámica del residuo mismo. Y en tercer lugar, y de manera no negociable para la validez del diseño, las predicciones de la etapa 1 que se emplean para calcular ese residuo de entrenamiento se generan siempre fuera de muestra (*out-of-fold*, OOF), mediante un esquema de validación cruzada expansivo hacia adelante (4.4) y nunca a partir de un único ajuste *in-sample*: un residuo calculado sobre predicciones que el propio modelo ya ha visto en entrenamiento está optimistamente sesgado hacia cero y es estructuralmente distinto del residuo que la etapa 2 enfrentará realmente en inferencia, de modo que aprender a corregirlo no transferiría a producción.

4.1. Diagnóstico previo: ¿tiene el residuo estructura de calendario?

Antes de comprometerse con la arquitectura de dos etapas descrita más arriba conviene comprobar, con evidencia y no por intuición, que existe una estructura de error sistemática que una segunda etapa no lineal pueda efectivamente corregir: si un modelo lineal ya captura todo el error explicable, ni el híbrido residual ni el repesado por calendario de la sección 5.7.1 tendrían ningún margen de mejora posible. Este diagnóstico exploratorio se calcula sobre el error absoluto de SARIMAX (el baseline estadístico de la sección 4.2, disponible ya en esta fase del proyecto) agregado por régimen de calendario sobre las 1.282 observaciones posteriores al recorte de calentamiento de 28 días (sección 3.5), y es deliberadamente exploratorio: se calcula sobre el periodo completo train+val+test y no sobre una partición fuera de muestra, porque los días festivos son demasiado raros (48 en total) para dividirlos de forma fiable en tres particiones y seguir teniendo un tamaño de grupo interpretable.

El resultado confirma la premisa de la arquitectura híbrida: el error absoluto medio de SARIMAX es 492.092 en festivos (20,50 % de error porcentual medio) y 262.194 en sábado, frente a 212.383 en laborables en general, y el sesgo con signo es sistemáticamente negativo en festivos ($-145,250$), es decir, SARIMAX **sobreestima** la demanda festiva de forma consistente sin saber cuantificar correctamente su magnitud.

El contraste más nítido, sin embargo, aísla el efecto de los días puente del efecto de fin de semana comparando únicamente dentro del régimen laborable: los 54 días laborables de tipo puente presentan un error absoluto medio de 518.634 frente a 192.191 en los 819 laborables ordinarios, es decir, **los días puente concentran 2,70 veces el error absoluto medio de un laborable corriente**. Esta es exactamente la estructura no lineal y condicionada al calendario (grande en magnitud, concentrada en un subconjunto pequeño y bien identificable de días) que la etapa 2 de la arquitectura híbrida (sección 4.5) está diseñada para aprender a partir de variables exógenas, y la que motiva directamente, más adelante, tanto el repesado de días irregulares del experimento A (sección 5.7.1) como la lectura del desglose de error por tipo de día del capítulo 5 (sección 5.4). Que el mecanismo exista de forma medible en este diagnóstico exploratorio, y que aun así el híbrido resultante no supere a sus componentes de una sola etapa, es precisamente el contraste que sostiene el hallazgo central de este TFM: el problema no es que no haya señal de calendario que corregir, sino que la etapa 1 de la que parte esa corrección es demasiado débil (sección 5.6.1).

4.2. Baselines

Antes de evaluar cualquier arquitectura de aprendizaje automático es necesario fijar el listón frente al que se mide: un conjunto de modelos de referencia sencillos que establezcan cuánta señal puede explicarse sin recurrir a redes neuronales ni a variables exógenas. El primero, la persistencia $\hat{y}_t = y_{t-1}$, resulta ser un hombre de paja particu-

larmente débil sobre esta serie: como se documentó en la sección 3.3, la demanda salta en torno a 2,5 millones de viajes diarios en cada transición entre régimen laborable y régimen de fin de semana, una transición que ocurre dos veces por semana, y la persistencia ignora por construcción cualquier estructura de calendario. El resultado es un coeficiente de determinación negativo tanto en entrenamiento como en test, lo que indica que ni siquiera predecir la media de la serie sería peor. Por ello, el baseline ingenuo relevante en este trabajo es la persistencia estacional $\hat{y}_t = y_{t-7}$, que sí incorpora el ciclo semanal al comparar cada día con el mismo día de la semana anterior; superar a la persistencia estacional, y no a la persistencia simple, es la barra honesta que cualquier modelo posterior debe rebasar para justificar su complejidad adicional.

El baseline estadístico fuerte de esta familia es un modelo SARIMAX, seleccionado mediante una búsqueda en rejilla de 324 combinaciones de órdenes $(p, d, q) \times (P, D, Q, s)$, con $p, q, P, Q \in \{0, 1, 2\}$, $d, D \in \{0, 1\}$ y periodo estacional fijo $s = 7$, el ciclo semanal identificado en el análisis exploratorio. La selección se realiza exclusivamente por el criterio de información de Akaike (AIC) calculado sobre el conjunto de entrenamiento, deliberadamente sin consultar en ningún momento el error de validación: hacerlo convertiría la validación en un conjunto de ajuste adicional y contaminaría la comparación honesta que se plantea en el capítulo 5. El orden seleccionado, resumido en la tabla 4.1, es SARIMAX(2, 1, 2) $\times$ (0, 1, 2, 7), con un AIC de entrenamiento de 24.616,89; las cinco combinaciones mejor puntuadas se sitúan dentro de 0,8 puntos de AIC entre sí, de modo que el orden concreto elegido debe leerse como *una* especificación razonable de esta complejidad, no como una verdad estructural descubierta de forma inequívoca.

Orden (p,d,q)	Orden estacional (P,D,Q,s)	AIC	N.º variables exógenas
(2,1,2)	(0,1,2,7)	24.616,89	5

Cuadro 4.1: Orden SARIMAX seleccionado por AIC sobre 324 combinaciones.

El vector de variables exógenas se limita deliberadamente a cinco columnas de calendario (los indicadores de fin de semana y de día puente, y las tres categorías reales de tipo de festividad, excluyendo `feat_holiday_type_none` como nivel de referencia para evitar la trampa de la variable ficticia frente al intercepto de la regresión), sin incorporar ningún retardo ni estadístico móvil de la propia demanda: el componente $(p, d, q) \times (P, D, Q, s)$ del modelo ya captura la autorregresión y la diferenciación estacional de la serie [5], y añadir retardos hechos a mano expresaría la misma dependencia dos veces a través de dos mecanismos en competencia.

Los parámetros del modelo se estiman una única vez sobre el conjunto de entrenamiento; a partir de ahí, la predicción sobre validación y test es un pronóstico *walk-forward* de un paso mediante el filtro de Kalman (`.filter()`, sin reestimación de parámetros) con `get_prediction(dynamic=False)`, de modo que cada predicción en el instante t condici-
ciona exclusivamente sobre las observaciones reales hasta $t - 1$. Se prefiere este esquema a un único pronóstico estático a 390 días porque este último decaería hacia la media

de la serie y mediría, en la práctica, una distancia de extrapolación en lugar de la calidad real del modelo; se verificó además que los residuos no presentan autocorrelación de primer orden relevante (aproximadamente $-0,03$) y que la predicción sigue de cerca el valor real del propio día ($MAE = 163,627$) y no el del día anterior ($MAE = 909,082$), descartando así un desfase temporal accidental en la implementación. Con este protocolo, SARIMAX alcanza un MAPE de test del 3,85 % y un R^2 de 0,9672 (tabla 5.1, sección 5.1), constituyéndose en un baseline exigente que cualquier arquitectura de aprendizaje automático de este TFM debe superar para justificar su mayor coste de desarrollo.

4.3. Etapa 1: LSTM

La primera etapa del híbrido es una red LSTM [10] **univariante por diseño**: recibe únicamente la ventana de las W observaciones pasadas de `total` y debe inferir a partir de esa dinámica (sin ningún indicador de día de la semana, festividad o puente) toda la estructura de calendario que la sección 3.3 mostró dominante. Esta restricción no es una limitación accidental, sino la pregunta que la etapa 1 está diseñada para responder: cuánta de esa estructura puede recuperarse de la memoria temporal de la serie por sí sola, dejando a la etapa 2 la tarea explícita de aportar la información de calendario y meteorología que la etapa 1 estructuralmente no puede ver.

La longitud de la ventana W se fijó mediante una comparación empírica, no una búsqueda exhaustiva, entre tres valores candidatos, $W \in \{14, 28, 60\}$, entrenando en cada caso un modelo final completo y evaluándolo únicamente sobre la partición de validación; el conjunto de test se mantuvo deliberadamente fuera de esta decisión, de modo que la elección de W no pudiera beneficiarse, ni siquiera indirectamente, de información sobre la partición de evaluación final. Los resultados se recogen en la tabla 4.2: $W = 28$ resulta ganador en las cuatro métricas de validación simultáneamente, con una brecha de MAE de aproximadamente el 11 % frente a sus dos vecinos, una diferencia lo bastante amplia como para no atribuirse al azar de una única ejecución, aunque el propio diseño del experimento, una ejecución por valor de W , obliga a presentarla como evidencia documentada y no como el resultado de una búsqueda sistemática de hiperparámetros.

W	Secuencias train	Épocas ejecutadas	Mejor época	MAE (val)	RMSE (val)	MAPE (val, %)	R^2 (val)
14	875	154	144	459.813	721.182	13,28	0,7607
28	861	164	154	411.142	655.479	11,81	0,8023
60	829	87	77	456.063	692.825	12,89	0,7792

Cuadro 4.2: Comparación de longitud de ventana W (evidencia, no búsqueda exhaustiva: una ejecución final por W , test nunca consultado en la elección).

Conviene señalar, con la misma transparencia con la que se presentan las demás decisiones metodológicas de este capítulo, que W se eligió observando precisamente la partición de validación: por tanto, las métricas de validación de toda la familia LSTM

(`lstm_alone`, `hybrid`, `hybrid_weighted`) son levemente optimistas respecto de lo que cabría esperar de una evaluación totalmente independiente, mientras que sus métricas de test, nunca consultadas durante esta elección, constituyen la evaluación estrictamente limpia y son las que deben citarse como comparación principal frente a SARIMAX y XGBoost.

La arquitectura final, entrenada sobre $W = 28$, es deliberadamente compacta: una capa LSTM de 32 unidades con *dropout* de 0,2, seguida de una capa densa de salida lineal de una única neurona. Se optimiza con Adam ($\text{lr} = 10^{-3}$) bajo error cuadrático medio, en lotes de 32 secuencias, con parada anticipada de paciencia 10 y restauración de los mejores pesos observados (`restore_best_weights`). El corte de la parada anticipada se decide sobre una cola interna de las propias secuencias de *entrenamiento* (las últimas 129, correspondientes al periodo 2025-02-27 a 2025-07-05) y nunca sobre la partición de validación oficial: emplear la validación oficial en ese punto convertiría la parada anticipada en un mecanismo de ajuste sobre validación, sesgando de forma optimista y silenciosa cada cifra de validación que se reporte más adelante en la tabla maestra de comparación (5.1).

Entrenada de este modo sobre las 889 filas de entrenamiento posteriores al calentamiento, la etapa 1 en solitario queda claramente por debajo de SARIMAX en ambas particiones de evaluación. Y, más revelador todavía, no llega a superar ni siquiera a la persistencia estacional cuando ambas se comparan sobre las mismas filas fuera de muestra generadas por el esquema de la sección 4.4 (MAE OOF de 690.460 frente a 461.087). Esto no constituye un defecto de la implementación sino la respuesta honesta a la pregunta que la etapa 1 plantea: la dinámica temporal univariante, por sí sola, no basta para igualar a un modelo que sí observa el calendario de forma explícita. El detalle numérico completo de esta comparación, junto con el resto de modelos de una sola etapa, se presenta en la tabla maestra del capítulo 5 (5.1).

Dos decisiones de ajuste, verificadas empíricamente y no asumidas de antemano, merecen constar. Primero, el número máximo de épocas se elevó de 200 a 500 tras observar que el fold 3 del esquema OOF alcanzaba las 200 épocas sin que la parada anticipada hubiera actuado todavía —es decir, el límite artificial de épocas, no la convergencia del modelo, era la restricción activa—, lo que habría entrenado folds de tamaño distinto hasta presupuestos de cómputo efectivamente distintos y los habría hecho incomparables entre sí; al elevar el límite, el fold 3 llegó a 228 épocas y su MAE mejoró de 288.194 a 282.122. Segundo, se probó y se descartó el barajado de los lotes de entrenamiento: aunque barajar secuencias ya construidas no constituiría en sí misma una fuga de información —cada secuencia es autocontenida y la cola de parada anticipada se separa cronológicamente antes de cualquier barajado—, medido sobre el fold 3 el barajado empeoró el resultado (MAE OOF de 288.000 a 309.000 aproximadamente), por lo que se mantiene `shuffle=False` por evidencia, no por precaución teórica.

4.4. Esquema de validación fuera de muestra (OOF)

El esquema descrito en esta sección es, con diferencia, el componente metodológico más crítico del proyecto: la decisión de diseño de la que depende la validez de toda la etapa 2. Como se anticipó en la introducción del capítulo, las predicciones de la etapa 1 que alimentan el cálculo del residuo de entrenamiento de la etapa 2 deben generarse siempre fuera de muestra. La alternativa sería ajustar un único modelo LSTM sobre todo el entrenamiento y calcular el residuo directamente sobre esas mismas predicciones *in-sample*, y produciría un residuo artificialmente pequeño y con una distribución distinta de la que la etapa 2 encontrará en producción, precisamente porque el modelo ya ha memorizado, en algún grado, las observaciones sobre las que se evalúa; la etapa 2 aprendería entonces a corregir una distribución de error que no existe fuera de muestra, lo que se traduciría en un desempeño peor del esperado en inferencia real. Este esquema de validación fuera de muestra para series temporales sigue la práctica establecida en la literatura de evaluación de modelos de pronóstico [4, 7], adaptada aquí a la generación de predicciones de *stacking* [18] en dos etapas.

Concretamente, se emplea una validación cruzada expansiva hacia adelante (*walk-forward*) de cinco folds sobre el periodo de entrenamiento posterior al calentamiento, 2023-01-29 a 2025-07-05 (889 filas). El fold k entrena un modelo LSTM completo sobre el bloque $[0, s_k)$ y genera predicciones únicamente sobre el bloque inmediatamente siguiente, $[s_k, s_{k+1})$, que nunca ha visto durante su entrenamiento; cada fold reajusta además su propio escalador de normalización exclusivamente sobre su propia porción de entrenamiento, sin reutilizar en ningún caso un escalador ajustado con información de un bloque posterior. La tabla 4.3 recoge las métricas de cada uno de los cinco folds, y la figura 4.1 muestra sus predicciones fuera de muestra frente al valor real.

Fold	Épocas ejecutadas	Mejor época	Loss train (final)	Loss val (final)	MAE	RMSE
1	15	5	0,1195	0,0788	1.461.905	1.682.893
2	153	143	0,0271	0,0509	659.425	882.524
3	228	218	0,0207	0,0281	282.122	397.512
4	131	121	0,0208	0,0098	548.081	839.318
5	81	71	0,0251	0,0331	506.148	718.174

Cuadro 4.3: Métricas por fold del esquema OOF *walk-forward* expansivo (5 folds).

Las 200 primeras filas de entrenamiento (2023-01-29 a 2023-08-16) no reciben ninguna predicción OOF, puesto que evaluarlas exigiría un modelo entrenado con menos observaciones que la ventana mínima inicial que el esquema considera aceptable; estas filas se conservan en el fichero de salida marcadas explícitamente como `has_oof=False`, con su motivo de exclusión indicado, en lugar de eliminarse sin más. La cobertura efectiva resultante es de 689 de las 889 filas de entrenamiento (77,5 %), con un MAE OOF agregado de 690.460 y un RMSE de 997.520.

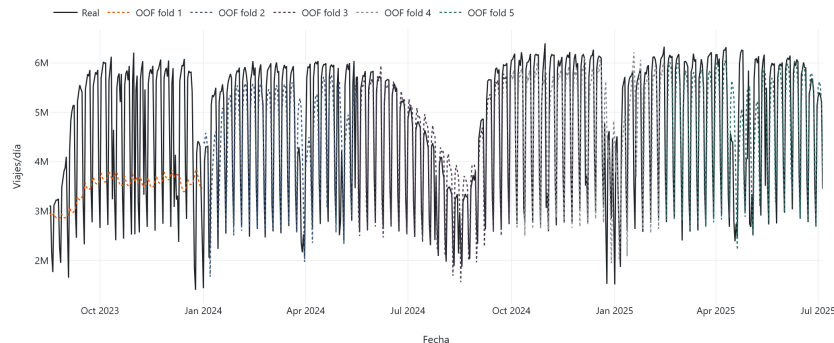


Figura 4.1: Predicción LSTM fuera de muestra (OOF) por fold del walk-forward expansivo.

El fold 1 merece un análisis detallado porque su comportamiento es **degenerado** y la decisión que se toma sobre él condiciona directamente el conjunto de entrenamiento de la etapa 2. Entrenado sobre apenas 200 observaciones (172 secuencias, reducidas a 147 tras separar la cola de parada anticipada), sus predicciones fuera de muestra presentan una desviación típica de solo el 0,199 de la de los valores reales y una correlación de apenas 0,300 con ellos: el modelo, con tan poco historial disponible, nunca llega a escapar de predecir un valor próximo a una constante, y la parada anticipada se activa ya en la época 15 sobre una señal de validación interna construida con solo 25 secuencias, demasiado ruidosa para guiar el entrenamiento de forma fiable. La figura 4.2 contrasta visualmente esta degeneración con el fold 3, sano por comparación (correlaciones entre 0,79 y 0,95 en los folds 2 a 5, frente al 0,300 del fold 1).

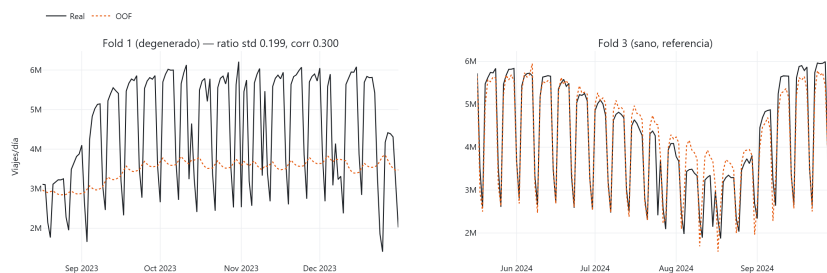


Figura 4.2: Fold 1 del esquema OOF es degenerado (predicción casi constante) frente al fold 3 (sano, referencia).

La consecuencia práctica es que las 137 filas del fold 1 se **excluyen** del conjunto de entrenamiento de la etapa 2 (4.5): sus residuos están dominados por el fallo estructural de un modelo con historial insuficiente, no por la estructura de calendario que se pretende que XGBoost aprenda, e incorporarlos introduciría ruido de una naturaleza distinta a la del resto del conjunto residual. Esta exclusión no se decide por intuición, sino que se somete a una comprobación explícita de cobertura de categorías: para cada nivel categórico presente en el fold 1 (tipo de día, tipo de festividad, indicador de día puente)

se verifica que ese mismo nivel reaparece con observaciones suficientes en los folds 2 a 5 restantes. Ninguna categoría queda huérfana tras la exclusión, de modo que eliminar el fold 1 retira redundancia de cobertura y no información única e insustituible; de haberse detectado una categoría con cobertura exclusiva en el fold 1, el procedimiento habría detenido la exclusión en lugar de proceder de todos modos.

4.5. Etapa 2: XGBoost sobre el residuo

Tras retirar las 200 filas sin predicción OOF y las 137 filas del fold 1 degenerado, el conjunto de entrenamiento de la etapa 2 queda formado por 552 filas, desde el 2024-01-01 hasta el 2025-07-05, con las 161 variables exógenas de la sección 3.4 y sin ningún valor nulo. La variable objetivo de esta etapa no es la demanda sino el residuo definido en la introducción del capítulo, $e_t = y_t - \hat{y}_t^{\text{LSTM}}$, calculado exclusivamente a partir de las predicciones fuera de muestra de la sección anterior; su signo se fija de modo que la corrección de la etapa 2 se *suma* a la predicción de la etapa 1. El residuo presenta una media de 109.601 y una desviación típica de 726.669 sobre un rango de $[-3,128,729, +1,899,895]$: una dispersión del mismo orden de magnitud que la propia señal de demanda, lo que anticipa que corregirlo es, en sí mismo, un problema de predicción exigente.

XGBoost [8] se entrena sobre este conjunto residual con los hiperparámetros de la tabla 4.4, seleccionados mediante una rejilla de 54 combinaciones evaluada sobre un *holdout* interno cronológico (el último 15 % de las fechas del propio conjunto de entrenamiento del modelo, nunca la partición de validación oficial) y reajustados después sobre la totalidad de los datos disponibles para ese modelo. La configuración ganadora combina árboles relativamente profundos ($\text{max_depth} = 5$) con una tasa de aprendizaje baja (0,01) y un número elevado de estimadores (300), junto con una regularización moderada por tamaño mínimo de nodo ($\text{min_child_weight} = 3$); esta combinación resulta coherente con un conjunto de entrenamiento reducido (552 filas) y una señal de calendario dispersa y de bajo peso relativo frente al ruido introducido por la propia etapa 1.

Modelo	Filas entrenamiento	max_depth	n_estimators	learning_rate	min_child_weight
xgboost_residual	552	5	300	0,01	3
xgboost_alone	889	5	100	0,05	10

Cuadro 4.4: Hiperparámetros seleccionados por grid search de 54 puntos (idéntico para ambos modelos XGBoost) sobre un *holdout* interno cronológico.

La variable individual con mayor ganancia en este modelo es *feat_day_type_festivo* (ganancia 0,125; tabla 5.8, capítulo 5), lo que confirma empíricamente que la etapa 2 sí aprende, a partir de las variables exógenas, precisamente la señal de calendario que la etapa 1 no puede ver por diseño: el mecanismo que motiva la arquitectura

híbrida es real y medible, con independencia de que el resultado agregado del capítulo 5 termine siendo, o no, favorable a la arquitectura completa.

Conviene dejar constancia, en este punto y no como un descubrimiento tardío en la discusión de resultados, de una limitación estructural del esquema de *stacking* OOF empleado. Los cinco modelos LSTM que generan los residuos de entrenamiento se ajustan, según el fold, sobre entre 337 y 748 filas, mientras que el modelo LSTM final que se emplea en inferencia, el descrito en la sección 4.3, se entrena sobre las 889 filas completas y es, previsiblemente, algo mejor que cualquiera de los modelos parciales que generaron los residuos de entrenamiento. En consecuencia, la etapa 2 aprende a corregir una distribución de residuo ligeramente más ancha que la que el sistema realmente enfrenta en inferencia. Este desajuste es inherente a cualquier esquema de *stacking* fuera de muestra —la alternativa, usar residuos *in-sample* del modelo final, es estrictamente peor por las razones expuestas en la sección 4.4— y se acepta aquí como el coste asumido de evitar una fuga de información mucho más grave.

4.6. Modelos de control

Para poder atribuir el desempeño del híbrido a su arquitectura, y no simplemente al hecho de disponer de variables exógenas, es necesario compararlo contra un control que utilice exactamente las mismas 161 variables sin pasar por ninguna etapa LSTM previa. Este control, denominado `xgboost_alone`, entrena un modelo XGBoost directamente sobre la demanda total, con idéntico procedimiento de ajuste de hiperparámetros que el modelo residual de la sección anterior y sobre la misma matriz de características. Su función es aislar si la etapa 1 aporta valor incremental a la predicción final o si, por el contrario, introduce ruido que un modelo de una sola etapa evitaría directamente.

Una decisión que podría parecer, a primera vista, una inconsistencia entre ambos modelos es deliberada: `xgboost_alone` se entrena sobre las 889 filas completas del conjunto de entrenamiento posterior al calentamiento, no sobre las 552 filas del conjunto residual de la etapa 2. La razón es que `xgboost_alone` no depende en ningún momento de las predicciones OOF de la etapa 1, de modo que ni la exclusión de las 200 filas sin cobertura OOF ni la exclusión del fold 1 degenerado le son aplicables: imponérselas de todos modos, en nombre de una supuesta igualdad de condiciones, habría privado al control de un tercio de su historial de entrenamiento disponible por una razón que le es completamente ajena. La matriz de variables y el procedimiento de ajuste de hiperparámetros son, por tanto, idénticos entre ambos modelos XGBoost; el tamaño del conjunto de entrenamiento, legítimamente, no lo es.

La propuesta inicial de este TFM contemplaba, junto a XGBoost, un segundo representante de la familia de aprendizaje automático tabular (*random forest*). Se ha prescindido de él de forma deliberada: ambos son métodos de conjunto sobre árboles de decisión y comparten la misma capacidad de representación sobre esta matriz de características, de modo que un segundo representante habría añadido una fila más a la comparación maestra sin poner a prueba ninguna hipótesis distinta. El esfuerzo de con-

trol se ha destinado, en su lugar, a los experimentos de *paridad informativa* de la sección 5.10 (ablación de bloques de variables y LSTM con acceso a la matriz exógena completa), que sí discriminan entre las explicaciones rivales del resultado central de este trabajo.

Como complemento a este control directo, la sección 5.7 del capítulo 5 introduce una variante adicional de la etapa 2, `hybrid_weighted`, que no constituye un modelo nuevo, sino una reponderación del mismo modelo residual: idéntica en las 552 filas de entrenamiento, las 161 variables, la semilla aleatoria y los hiperparámetros (cargados directamente del modelo de la sección 4.5 en lugar de reajustarse de forma independiente, precisamente para que ambas variantes sean comparables sin introducir una segunda fuente de variación), y que difiere únicamente en que asigna un peso de muestra triple a las filas correspondientes a días de calendario irregular (festivos y días puente), un 8,2 % del conjunto residual. El objetivo de esta variante es comprobar si concentrar la capacidad del modelo en los días donde la etapa 1 falla con mayor intensidad (festivos y puentes, según el diagnóstico de la sección 4.1) mejora la corrección precisamente en esos días; su evaluación completa, con un criterio de éxito fijado antes de observar el resultado, se desarrolla en la sección 5.7.1 y no se anticipa aquí.

4.7. Ensamble por combinación de pronósticos

El segundo control de la sección anterior, SARIMAX, y el modelo `xgboost_alone` no solo se emplean por separado como adversarios del híbrido: sus predicciones, ya generadas y persistidas, permiten formular sin ningún coste de entrenamiento adicional un tercer candidato —una combinación lineal de ambos— cuya justificación conviene formalizar aquí, como parte del diseño experimental, y no introducir por primera vez al presentar sus resultados en el capítulo 5. Dados los pronósticos puntuales de ambos modelos en el instante t , $\hat{y}_t^{\text{SARIMAX}}$ y $\hat{y}_t^{\text{XGB}}$, el ensamble se define como la combinación lineal convexa

$$\hat{y}_t = w_1 \hat{y}_t^{\text{SARIMAX}} + w_2 \hat{y}_t^{\text{XGB}}, \quad w_1 + w_2 = 1, \quad w_1, w_2 \geq 0,$$

una operación puramente aritmética sobre predicciones ya calculadas, sin reentrenar ningún modelo ni introducir ningún parámetro adicional que deba ajustarse por descenso de gradiente.

La justificación de que una combinación tan simple pueda superar a ambos componentes individuales no es heurística: descansa en la identidad de reducción de varianza de Bates y Granger y en la condición de Granger sobre conjuntos de información distintos, ambas expuestas en la sección 2.3. Lo que importa fijar aquí es que SARIMAX y `xgboost_alone` sí satisfacen esa condición: el primero modela explícitamente la autorregresión y la estacionalidad semanal mediante su componente $(p, d, q) \times (P, D, Q, s)$, mientras que el segundo aprende relaciones no lineales directamente sobre variables exógenas de calendario y meteorología, sin ningún término autorregresivo. El híbrido

secuencial de este TFM, en cambio, la *incumple*, porque ambas etapas observan en el fondo el mismo pasado de la serie `total` (sección 5.6.1).

Se contemplan, por diseño y antes de observar ningún resultado, dos estrategias de ponderación. La primera, `ensemble_equal`, fija $w_1 = w_2 = 0,5$: no requiere ningún dato adicional para determinarse y actúa como el punto de referencia neutro de la combinación. La segunda, `ensemble_inverse_mae`, pondera cada componente en proporción inversa a su MAE sobre la partición de validación, de modo que el modelo históricamente más preciso recibe mayor peso; sus coeficientes se derivan exclusivamente de la validación y nunca de la partición de test, preservando la separación estricta entre selección de configuración y evaluación final que gobierna todo este TFM. Ambas variantes se evalúan en el capítulo 5 (secciones 5.7 y 5.8), donde se cuantifica empíricamente la correlación de error ρ entre SARIMAX y `xgboost_alone` y se contrasta la reducción de varianza observada frente a la que la identidad de Bates y Granger predice; se adelanta únicamente que ese contraste sitúa a `ensemble_equal` y `ensemble_inverse_mae` como los dos modelos más precisos de todo el estudio (MAE de test 139.725 y 139.681, respectivamente), prácticamente indistinguibles entre sí, lo que hace preferible la variante equiponderada por no depender de ningún dato de validación para fijar sus pesos.

5. Evaluación experimental del sistema predictivo

Este capítulo es el núcleo empírico del TFM: presenta y discute los resultados de los once modelos descritos o derivados en el capítulo 4, evaluados sobre las particiones de validación y test definidas en la sección 3.5. Ninguna cifra se teclea a mano: todas llegan al documento desde los artefactos persistidos por las fases 3 a 5b, por la cadena que detalla el Anexo C. La respuesta al contraste central del TFM (5.6) se somete a tres controles diagnósticos independientes: la anatomía del residuo de la etapa 1 (5.9), la paridad informativa entre etapas (5.10) y el desglose por subgrupos y periodos con corrección por comparaciones múltiples (5.11).

5.1. Tabla maestra de comparación

Las tablas 5.1 y 5.2 reúnen los once modelos de este TFM (cuatro baselines ingenuos, dos modelos de una sola etapa, tres variantes de la familia LSTM y dos variantes de ensamble) ordenados de mejor a peor por MAE sobre las particiones de test y de validación respectivamente. El coeficiente de determinación se reporta a cuatro decimales y no a dos, una asimetría deliberada frente al resto de métricas: a dos decimales, `ensemble_equal` (0,9755), `xgboost_alone` (0,9690) y `sarimax` (0,9672) colapsarían en 0,98/0,97/0,97, borrando exactamente la distinción que esta comparación existe para establecer. El MAPE, en cambio, no sufre ese problema a esta escala y se mantiene a dos decimales.

La jerarquía resultante es nítida y consistente entre ambas particiones, como confirma la figura 5.1. En primer lugar, las dos variantes de ensamble (combinaciones puramente aritméticas de SARIMAX y `xgboost_alone`, sin entrenamiento adicional) lideran ambas particiones: `ensemble_inverse_mae` obtiene el mejor MAE de test, **139.681**, con un R^2 de 0,9755, seguido a menos de un 0,05 % de distancia por `ensemble_equal` (139.725). En segundo lugar, los dos modelos de una sola etapa ocupan el siguiente escalón sin solaparse con el anterior: `xgboost_alone` (MAE test 156.121, $R^2 = 0,9690$) y SARIMAX (MAE test 163.627, $R^2 = 0,9672$) mejoran de forma clara a toda la familia LSTM que seguirá describiéndose en las secciones siguientes, pero quedan por detrás del ensamble en aproximadamente un 11 % y un 17 % de MAE respectivamente. En tercer lugar, la familia LSTM (`hybrid_weighted`, `hybrid` y `lstm_alone`, en ese orden) se sitúa sistemáticamente peor que ambos modelos de una sola etapa, un resultado que se analiza en detalle en la sección 5.6.

Por último, los cuatro baselines ingenuos colapsan frente a cualquiera de los modelos anteriores: la persistencia estacional, el mejor de ellos, supera en torno a un 10 % en MAE al peor miembro de la familia LSTM, y la persistencia simple y las dos medias

Modelo	Familia	n	MAE	RMSE	MAPE (%)	R ²
ensemble_inverse_mae	Ensemble	197	139.681	208.339	3,11	0,9755
ensemble_equal	Ensemble	197	139.725	208.464	3,13	0,9755
xgboost_alone	Una etapa	197	156.121	234.154	3,30	0,9690
sarimax	Una etapa	197	163.627	241.147	3,85	0,9672
hybrid_weighted	Familia LSTM	197	220.286	319.447	5,17	0,9424
hybrid	Familia LSTM	197	234.223	328.131	5,49	0,9392
lstm_alone	Familia LSTM	197	280.952	431.714	6,62	0,8948
seasonal_naive	Baseline ingenuo	197	309.817	690.480	7,16	0,7308
persistence	Baseline ingenuo	197	900.570	1.390.444	21,79	-0,0916
moving_average_7	Baseline ingenuo	197	1.093.679	1.260.492	28,13	0,1029
moving_average_28	Baseline ingenuo	197	1.119.987	1.301.939	28,74	0,0430

Cuadro 5.1: Comparación de modelos, partición TEST (n=197, 2026-01-18 a 2026-08-02).

Modelo	Familia	n	MAE	RMSE	MAPE (%)	R ²
ensemble_inverse_mae	Ensemble	196	226.418	323.348	5,62	0,9519
ensemble_equal	Ensemble	196	227.588	324.752	5,69	0,9515
xgboost_alone	Una etapa	196	244.843	355.027	5,68	0,9420
sarimax	Una etapa	196	273.467	404.153	7,31	0,9248
hybrid_weighted	Familia LSTM	196	276.347	401.456	6,94	0,9258
hybrid	Familia LSTM	196	280.738	415.562	7,09	0,9205
lstm_alone	Familia LSTM	196	411.142	655.479	11,81	0,8023
seasonal_naive	Baseline ingenuo	196	460.615	883.627	12,63	0,6408
persistence	Baseline ingenuo	196	914.119	1.387.393	23,72	0,1144
moving_average_7	Baseline ingenuo	196	1.068.961	1.237.156	28,86	0,2958
moving_average_28	Baseline ingenuo	196	1.137.759	1.357.281	31,12	0,1524

Cuadro 5.2: Comparación de modelos, partición VAL (n=196, 2025-07-06 a 2026-01-17).

móviles presentan R^2 próximos a cero o directamente negativos, confirmando sobre el conjunto completo de modelos la advertencia ya adelantada en el capítulo 4: superar a la persistencia simple es un listón trivial en esta serie, y el resultado interesante de este capítulo se juega enteramente entre los siete modelos restantes.

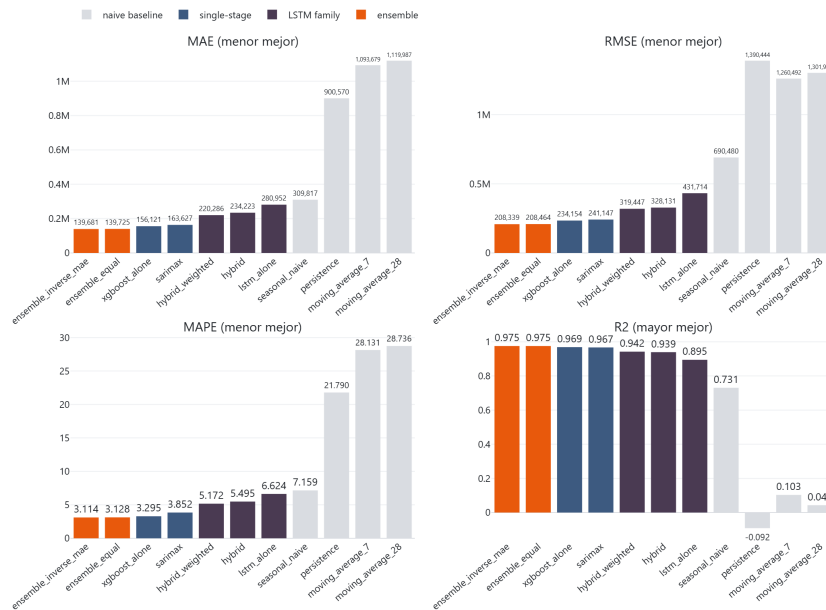


Figura 5.1: Comparación de modelos en la partición test. Barras agrupadas de MAE, RMSE, MAPE y R^2 , ordenadas de mejor a peor por MAE. El color codifica la familia del modelo.

Ninguna cifra de la partición de entrenamiento se presenta junto a estas dos tablas como si fuera una estimación honesta de desempeño: train es, por construcción, una muestra *in-sample* para todo modelo que se ajusta sobre ella, y su inclusión aquí induciría a compararla incorrectamente con val o test, un error que la sección 5.3 evita explícitamente al tratar el desempeño en train como diagnóstico de sobreajuste y no como un resultado más de la tabla maestra.

5.2. Ajuste temporal

La figura 5.2 muestra la serie real de demanda total superpuesta a la predicción del modelo ganador, `ensemble_equal`, sobre la partición de test; la de validación reproduce el mismo patrón. La predicción sigue de cerca tanto la oscilación semanal (el diente de sierra laborable/fin de semana de la sección 3.3) como las transiciones asociadas a festivos aislados, sin el retraso de fase que delataría un modelo que en realidad predijera el valor del día anterior. Las mayores desviaciones puntuales se concentran en los días de calendario irregular, festivos y puentes, desglosados en la sección 5.4; el resto de la serie se sigue con una fidelidad coherente con el MAPE de test del 3,13 % de la tabla 5.1.

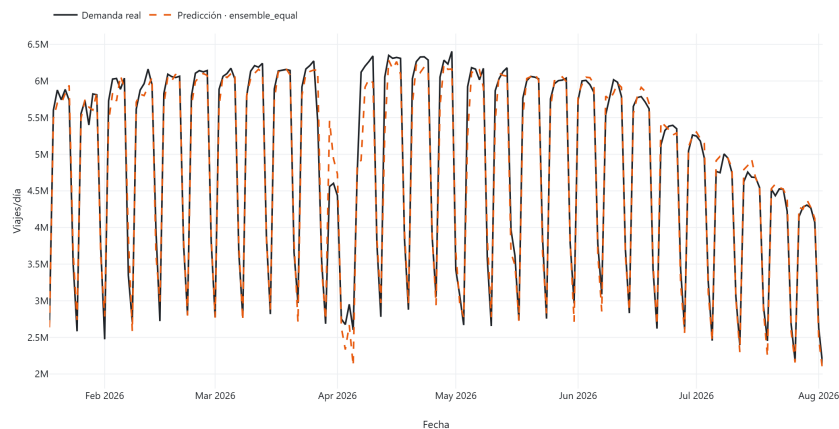


Figura 5.2: Demanda diaria real y predicha por el modelo `ensemble_equal` en la partición test. Serie observada y predicha superpuestas.

5.3. Análisis de residuos

El análisis de residuos permite distinguir un modelo que generaliza de uno que memoriza. El caso más claro de esta distinción en todo el proyecto es `xgboost_alone`: su coeficiente de determinación en entrenamiento alcanza 0,9929, un ajuste casi perfecto, frente a un MAE de test de 156.121; esa brecha entre un R^2 de entrenamiento prácticamente unitario y un error de test sustancial es la evidencia de que el modelo memoriza una parte relevante de su conjunto de entrenamiento en lugar de limitarse a capturar la estructura genuinamente generalizable de la demanda. La figura 5.3 hace visible esta brecha comparando las distribuciones de error en train y en test: la primera es marcadamente más estrecha y centrada que la segunda, un patrón de sobreajuste clásico en un modelo de árboles con capacidad suficiente para ajustar el ruido de una muestra de apenas 889 observaciones diarias. Es importante subrayar que la cifra que debe citarse en cualquier afirmación sobre el desempeño de este modelo es siempre la de test, nunca la de entrenamiento como si fuera indicativa de generalización.

Los modelos derivados de la etapa 1 LSTM, `hybrid` entre ellos, no disponen de esta misma comparación train-test, puesto que la fase 4 únicamente persistió predicciones del modelo final sobre las particiones de validación y de test, no sobre entrenamiento; no se fuerza aquí una comparación que los datos disponibles no permiten hacer honestamente. La figura 5.4 muestra, en su lugar, la distribución del error de `hybrid` en test sin más: una distribución visiblemente más dispersa que la de `xgboost_alone`, coherente con el MAE de test casi 50 % mayor que se cuantificó en la sección 5.1.

Por contraste, la figura 5.5 muestra la distribución de error del modelo ganador, `ensemble_equal`: una distribución sensiblemente más estrecha y mejor centrada en torno a cero que la de cualquiera de sus dos componentes individuales por separado. Este estrechamiento no es un artefacto cosmético, sino la manifestación directa, en la distribución del error, del mecanismo de reducción de varianza por descorrelación de errores

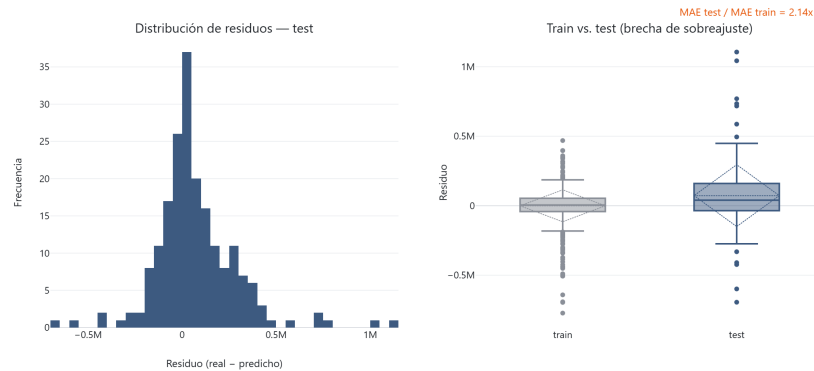


Figura 5.3: Distribución del error de predicción del modelo `xgboost_alone` en la partición test. La brecha train-test es la evidencia visual del sobreajuste (R^2 train=0,9929 vs. MAE test=156.121).

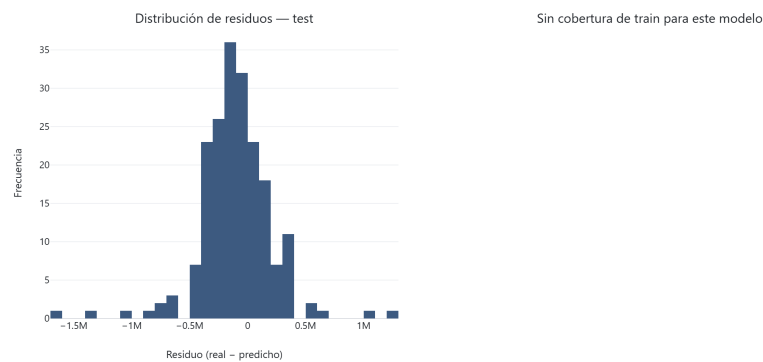


Figura 5.4: Distribución del error de predicción del modelo `hybrid` en la partición test.

que se cuantifica formalmente en la sección 5.8: promediar dos modelos cuyos errores no están perfectamente correlacionados produce un error agregado con menor dispersión que cualquiera de los dos errores originales, incluso sin que ninguno de los dos componentes cambie en absoluto.

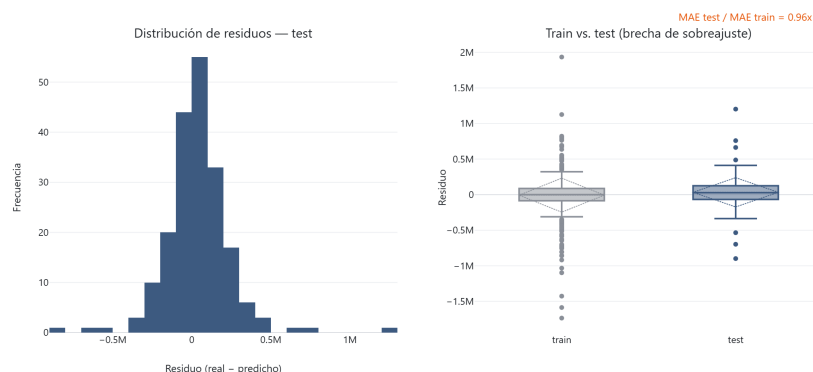


Figura 5.5: Distribución del error de predicción del modelo `ensemble_equal` (ganador) en la partición test.

5.4. Error por tipo de día

Las tablas 5.3 y 5.4 descomponen el MAE de cada modelo por régimen de calendario (laborable ordinario, laborable con puente, sábado, domingo y festivo), y la figura 5.6 lo representa gráficamente, atenuando visualmente los grupos con menos de diez observaciones y anotando su tamaño muestral bajo cada barra. La sección 5.11 retoma esta descomposición de forma sistemática (quince subgrupos de calendario y periodo fijados por adelantado, con réplica en validación y corrección por comparaciones múltiples) para responder a si el híbrido mejora en algún régimen concreto.

Tipo de día	n	sarimax	xgboost_alone	hybrid	hybrid_weighted	ensemble_equal
Laborable ordinario	133	148.297	169.488	211.581	200.326	138.628
Laborable	136	150.493	169.845	215.729	205.438	140.352
Sábado	27	203.910	129.381	247.793	185.918	138.805
Domingo	29	169.129	100.200	233.778	243.096	122.591
Festivo	5	271.410	251.553	666.572	677.429	227.037
Laborable, puente	3	247.865	185.671	399.637	432.066	216.768

Cuadro 5.3: MAE por tipo de día, partición test. $n=5$ (festivo) y $n=3$ (laborable, puente): indicadores, no concluyentes. La fila “Laborable” ($n = 136$) es la suma de “Laborable ordinario” (133) y “Laborable, puente” (3); no es un grupo adicional independiente.

Esta descomposición revela la razón mecánica por la que el ensamble supera a ambos modelos de una sola etapa que lo componen, y no solo el hecho de que lo hace. Sobre la partición test, SARIMAX es claramente mejor que `xgboost_alone` en los días laborables ordinarios (MAE 148.297 frente a 169.488): su componente autorregresivo estacional captura con precisión la dinámica de la inmensa mayoría de los días del año,

Tipo de día	n	sarimax	xgboost_ alone	hybrid	hybrid_ weighted	ensemble_ equal
Laborable ordinario	121	211.578	263.817	242.993	240.264	205.597
Laborable	133	238.197	285.078	286.849	280.851	231.011
Sábado	26	435.186	132.334	278.593	245.120	246.171
Domingo	28	245.694	128.658	245.450	253.547	162.840
Festivo	9	413.907	336.750	306.409	370.935	324.760
Laborable, puente	12	506.603	499.452	729.068	690.105	487.269

Cuadro 5.4: MAE por tipo de día, partición val. La fila “Laborable” ($n = 133$) es la suma de “Laborable ordinario” (121) y “Laborable, puente” (12); no es un grupo adicional independiente.

que son precisamente laborables ordinarios. `xgboost_alone`, en cambio, es netamente superior en fin de semana: en domingo su MAE es 100.200 frente a los 169.129 de SARIMAX, y en sábado 129.381 frente a 203.910, casi 40 % y 37 % mejor, respectivamente. **Ningún modelo domina al otro de forma global**; cada uno acierta mejor en un subconjunto distinto y complementario de los días del año, y es exactamente esa complementariedad la que hace mecánicamente ventajoso promediarlos, como se formaliza en la sección 5.8.

El híbrido y su variante reponderada no participan de esta complementariedad: son, con la única excepción de la comparación frente a `lstm_alone`, peores que SARIMAX y que `xgboost_alone` en prácticamente todos los grupos de la tabla 5.3. El hallazgo parcialmente positivo, y el único genuinamente favorable a la arquitectura híbrida en todo ese desglose, aparece en festivos: la etapa 2 reduce el error de su propia etapa 1 de 1.585.388 (fila festivo, columna `lstm_alone` de la tabla 5.3) a 666.572, una mejora del 58 %, que confirma de nuevo que el mecanismo de corrección funciona como está diseñado. Aun así, ese 666.572 sigue siendo 2,5 veces peor que el MAE de SARIMAX en el mismo grupo (271.410) y notablemente peor que `xgboost_alone` (251.553): la corrección funciona, pero parte de una etapa 1 demasiado débil como para que el resultado corregido alcance a los modelos de una sola etapa.

Es indispensable acompañar toda esta lectura de una nota de cautela obligatoria: en la partición test, el grupo de festivos contiene únicamente cinco observaciones y el de laborables con puente solo tres. Con muestras de ese tamaño, un único día atípico puede desplazar el MAE del grupo entero en cientos de miles de viajes, de modo que estas dos filas deben leerse como indicativas y en ningún caso como concluyentes; para afirmaciones robustas sobre *dónde* se concentra estructuralmente el error de un modelo univariante frente al calendario, la base más sólida sigue siendo el diagnóstico exploratorio de la fase 4 sobre el periodo completo (48 festivos y 54 puentes; la codificación de calendario detecta 55 días puente sobre el total de 1.310 días, de los que 54 quedan fuera de los 28 días de calentamiento excluidos del entrenamiento), no las particiones

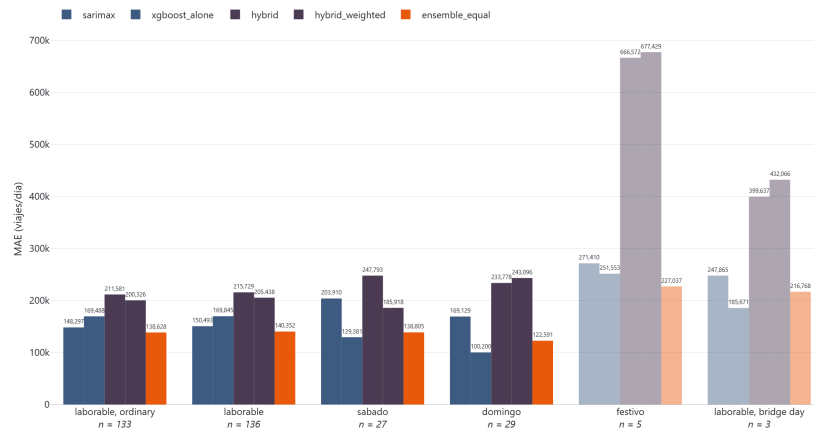


Figura 5.6: MAE por tipo de día en la partición test. Los grupos con $n < 10$ se dibujan con opacidad reducida y su n anotado bajo el eje: son indicativos, no concluyentes.

reducidas de test y validación empleadas para esta comparación final.

5.5. Importancia de variables

Las figuras 5.7 y 5.8, junto con las tablas 5.5 y 5.7, agregan la ganancia de cada uno de los dos modelos XGBoost del proyecto por grupo temático de variable, y las tablas 5.6 y 5.8 descienden al detalle de las quince variables individuales más influyentes de cada modelo. El contraste entre ambos modelos es la evidencia más directa de todo el capítulo sobre qué tipo de información resuelve cada problema de predicción.

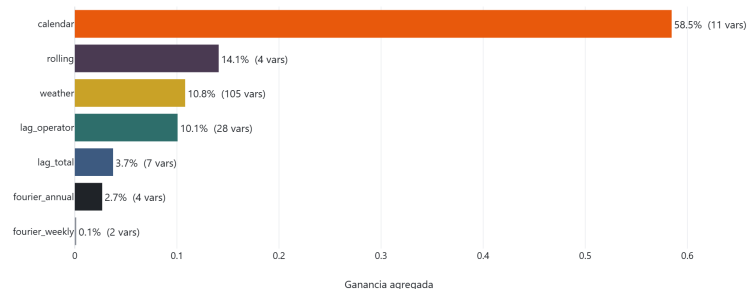


Figura 5.7: Importancia agregada por grupo de variables del modelo `xgboost_alone`, medida como ganancia.

En `xgboost_alone`, el grupo de calendario, solo 11 de las 161 variables disponibles, concentra el 58,5 % de la ganancia total del modelo (tabla 5.5), y su variable individual más influyente con diferencia, `feat_day_type_laborable`, aporta por sí sola una ganancia de 0,561 (tabla 5.6): al predecir la demanda directamente, saber si el día es laborable resuelve la práctica totalidad de la varianza semanal identificada en la sección 3.3, y el resto de variables (retardos de operador, meteorología retardada, términos de Fourier) se reparten un papel claramente secundario de matiz fino sobre ese nivel base.

Grupo	N.º variables	Ganancia	Peso (%)
calendar	11	0,5846	58,46
rolling	4	0,1409	14,09
weather	105	0,1081	10,81
lag_operator	28	0,1007	10,07
lag_total	7	0,0375	3,75
fourier_annual	4	0,0268	2,68
fourier_weekly	2	0,0014	0,14

Cuadro 5.5: Importancia por grupo de variables, `xgboost_alone`.

El modelo residual de la etapa 2, `xgboost_residual`, invierte por completo esta jerarquía de grupos (figura 5.8, tabla 5.7): la meteorología retardada, con sus 105 variables, concentra ahora el 51,6 % de la ganancia, mientras que el calendario desciende al 17,3 %, prácticamente empatado con los retardos de operador (17,2 %). La lectura inmediata de esta inversión, y la que se adoptó al observarla por primera vez, es coherente

#	Variable	Grupo	Ganancia
1	feat_day_type_laborable	calendar	0,5609
2	feat_total_roll_std_7	rolling	0,1321
3	feat_carretera_lag_1	lag_operator	0,0209
4	feat_total_lag_1	lag_total	0,0157
5	feat_day_type_sabado	calendar	0,0156
6	feat_metro_lag_1	lag_operator	0,0152
7	feat_metro_lag_14	lag_operator	0,0134
8	feat_fourier_annual_k2_sin	fourier_annual	0,0127
9	feat_relative_humidity_2m_mean_lag_2	weather	0,0111
10	feat_emt_lag_1	lag_operator	0,0110
11	feat_apparent_temperature_min_lag_1	weather	0,0086
12	feat_total_lag_3	lag_total	0,0075
13	feat_is_bridge_day	calendar	0,0070
14	feat_relative_humidity_2m_mean_roll_mean_7	weather	0,0064
15	feat_fourier_annual_k1_cos	fourier_annual	0,0060

Cuadro 5.6: Top-15 variables individuales por ganancia, *xgboost_alone*.

con la propia definición de la variable objetivo de esta etapa: el residuo $e_t = y_t - \hat{y}_t^{\text{LSTM}}$ ya no contendría la componente de calendario más gruesa, la distinción laborable/fin de semana, porque una parte relevante de esa componente ya habría sido absorbida, con mayor o menor acierto, por la propia dinámica temporal de la etapa 1; lo que quedaría por explicar en el residuo sería, en consecuencia, un matiz más fino, y ese matiz estaría más ligado a las condiciones meteorológicas retardadas que al calendario grueso.

Conviene subrayar que esto es una *hipótesis* sugerida por un ranking de importancia, no un resultado medido, y que las fases diagnósticas la someten a prueba con dos análisis independientes y la **rechazan**: la sección 5.9.2 no encuentra ninguna correlación parcial significativa entre el residuo y las 105 variables meteorológicas retardadas, y la ablación de la sección 5.10.1 muestra que retirar ese bloque *mejora* el modelo. La sección 5.10.3 reconcilia esa aparente contradicción y explica por qué una cuota de ganancia elevada es perfectamente compatible con la ausencia de señal aprovechable.

Grupo	N.º variables	Ganancia	Peso (%)
weather	105	0,5157	51,57
calendar	11	0,1726	17,26
lag_operator	28	0,1723	17,23
lag_total	7	0,0592	5,92
rolling	4	0,0405	4,05
fourier_annual	4	0,0320	3,20
fourier_weekly	2	0,0077	0,77

Cuadro 5.7: Importancia por grupo de variables, *xgboost_residual*.

Sin embargo, esta relectura por grupo no contradice el hallazgo mecanístico más

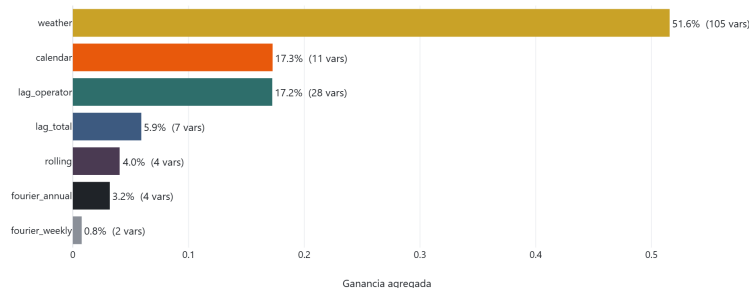


Figura 5.8: Importancia agregada por grupo de variables del modelo `xgboost_residual` (etapa 2 del híbrido).

importante de la etapa 2, visible únicamente al bajar al nivel de variable individual (tabla 5.8): pese a que el calendario pesa menos como grupo agregado, la variable individual con mayor ganancia en todo el modelo residual sigue siendo `feat_day_type_festivo`, con una ganancia de 0,125, muy por delante de la segunda variable de la lista. Esto confirma, al nivel más granular posible, que la etapa 2 sí aprende exactamente la señal de calendario (en concreto, la irregularidad de los festivos) que la etapa 1 univariante no puede ver por construcción: el mecanismo que justifica teóricamente la arquitectura híbrida es real, medible y localizado en la variable exacta que cabría esperar, con independencia de que el veredicto agregado de la sección 5.6 termine siendo negativo para la arquitectura completa.

#	Variable	Grupo	Ganancia
1	<code>feat_day_type_festivo</code>	calendar	0,1254
2	<code>feat_apparent_temperature_max_lag_7</code>	weather	0,0759
3	<code>feat_temperature_2m_mean_lag_3</code>	weather	0,0337
4	<code>feat_carretera_lag_3</code>	lag_operator	0,0319
5	<code>feat_day_type_laborable</code>	calendar	0,0305
6	<code>feat_total_lag_7</code>	lag_total	0,0249
7	<code>feat_total_roll_std_28</code>	rolling	0,0217
8	<code>feat_apparent_temperature_min_lag_3</code>	weather	0,0200
9	<code>feat_apparent_temperature_max_lag_3</code>	weather	0,0191
10	<code>feat_temperature_2m_min_roll_mean_7</code>	weather	0,0189
11	<code>feat_weather_code_roll_mean_7</code>	weather	0,0152
12	<code>feat_metro_lag_7</code>	lag_operator	0,0152
13	<code>feat_shortwave_radiation_sum_roll_mean_7</code>	weather	0,0149
14	<code>feat_total_lag_28</code>	lag_total	0,0137
15	<code>feat_temperature_2m_max_roll_mean_7</code>	weather	0,0128

Cuadro 5.8: Top-15 variables individuales por ganancia, `xgboost_residual`. `feat_day_type_festivo` encabeza con ganancia 0,125 – confirma que la etapa 2 aprende la señal de calendario que la etapa 1 no puede ver.

5.6. Contraste central del TFM

Con la evidencia de las secciones anteriores ya establecida, esta sección responde de forma explícita e inequívoca a la pregunta que estructura todo el proyecto: ¿mejora la arquitectura híbrida residual LSTM-XGBoost, tal como se ha diseñado e implementado en este TFM, la predicción de demanda frente a SARIMAX y frente a un XGBoost entrenado directamente sobre las mismas variables exógenas? La respuesta, sostenida por la tabla 5.9, es **no, en ninguna de las dos comparaciones y en ninguna de las dos particiones de evaluación.**

Partición	Comparación	Delta MAE	Veredicto
val	hybrid - sarimax	7.270	hybrid peor
val	hybrid - xgboost_alone	35.895	hybrid peor
test	hybrid - sarimax	70.597	hybrid peor
test	hybrid - xgboost_alone	78.103	hybrid peor

Cuadro 5.9: Diferencia de MAE del híbrido frente a SARIMAX y XGBoost-alone (Delta MAE = hybrid - competidor; positivo = híbrido peor). Generada, no tecleada a mano.

Sobre la partición test, el híbrido supera en MAE a SARIMAX en 70.597 viajes diarios y a `xgboost_alone` en 78.103; sobre validación las diferencias son menores en magnitud absoluta, pero mantienen el mismo signo, 7.270 y 35.895 respectivamente. La diferencia frente a `xgboost_alone` es, en ambas particiones, mayor que la diferencia frente a SARIMAX: el híbrido no solo pierde frente al baseline estadístico clásico, sino que pierde por un margen todavía más amplio frente a un modelo de aprendizaje automático que utiliza exactamente la misma información exógena sin pasar por ninguna etapa LSTM. Este segundo resultado es, si cabe, el más informativo de los dos: si el problema fuera únicamente que SARIMAX es un modelo particularmente bien ajustado a esta serie, cabría esperar que el híbrido, con acceso a variables no lineales y a una capa de aprendizaje profundo, cerrara al menos parte de esa distancia frente a otro modelo de aprendizaje automático; en su lugar, la distancia se amplía.

Por qué pierde el híbrido: lectura a la luz de Granger (1989)

La teoría clásica de combinación de pronósticos ofrece un marco preciso para interpretar este resultado, en lugar de limitarse a documentarlo. Granger [9], revisando dos décadas de literatura sobre combinación de modelos, formula la condición que determina cuándo un pronóstico híbrido o combinado puede superar a sus componentes: la mejora solo es posible cuando los modelos que se combinan son individualmente subóptimos y, de manera crucial, se basan en conjuntos de información distintos entre sí. Si dos modelos observan exactamente la misma información y solo difieren en la forma funcional con la que la procesan, combinarlos secuencialmente no introduce nada que uno de los dos no pudiera ya haber aprendido por sí mismo con una arquitectura suficientemente

flexible.

La arquitectura híbrida de este TFM **incumple precisamente esa segunda condición**. Ambas etapas observan, en última instancia, el mismo objeto: el pasado de la serie `total`. La etapa 1 lo observa directamente, en su forma univariante y sin calendario; la etapa 2 lo observa de forma indirecta, a través de un residuo que es función de ese mismo pasado más un conjunto de variables exógenas que, como muestra la sección 5.5, sí aportan información adicional genuina —de ahí que la etapa 2 mejore de forma sustancial a su propia etapa 1—, pero que no constituyen un conjunto de información *independiente* de la demanda pasada, sino un complemento sobre la misma serie. El resultado es exactamente el que la teoría de Granger predeciría: una mejora real y medible de la etapa 2 sobre la etapa 1 (documentada en la sección 5.4 y de nuevo en la 5.5), pero insuficiente para que la combinación secuencial supere a un modelo de una sola etapa que parte directamente de una base más sólida: SARIMAX, con su propio mecanismo autorregresivo y estacional, o `xgboost_alone`, que accede a las mismas 161 variables exógenas sin arrastrar el ruido de una etapa 1 estructuralmente débil.

Esta lectura se refuerza, además, por un factor puramente cuantitativo: la etapa 1 univariante deja un residuo con una desviación típica de 726.669 viajes diarios (4.5), del mismo orden de magnitud que la propia señal que se intenta predecir. Cuanto mayor es la varianza residual que la segunda etapa debe corregir, más difícil resulta que esa corrección alcance, y menos aún supere, a un modelo que ataca el problema completo desde el principio con la información exógena disponible. Este patrón —hibridar puede mejorar sustancialmente a un componente débil sin llegar a igualar a un componente fuerte de una sola etapa cuando ambos comparten información— no es exclusivo de este proyecto: es consistente con lo observado de forma sistemática en las competiciones de pronóstico a gran escala M4 y M5 [12, 13], donde los métodos híbridos y de combinación que finalmente destacan lo hacen precisamente al combinar familias de modelos con mecanismos e información de entrada genuinamente distintos, como la propia sección 5.8 demuestra para el ensamble de este TFM, y no al encadenar dos etapas sucesivas sobre el mismo conjunto de información. El resultado negativo del híbrido residual de este TFM debe leerse, por tanto, como consistente con la teoría publicada y con la evidencia empírica a gran escala, no como una anomalía específica de esta implementación.

5.7. Análisis de sensibilidad

Las dos secciones siguientes documentan dos experimentos acotados y registrados por adelantado, con su criterio de éxito fijado antes de observar el resultado, diseñados para explorar el margen de mejora del híbrido y para establecer un punto de comparación adicional, respectivamente. Ninguno de los dos reabre la conclusión central de la sección anterior ni constituye una búsqueda de una configuración que la revierta.

Experimento A: repesado de días irregulares (3x)

El primer experimento pregunta si concentrar la capacidad de ajuste de la etapa 2 precisamente en los días de calendario irregular —festivos y puentes, el subconjunto donde la sección 5.4 documentó el error relativo más alto de todo el sistema— mejora el desempeño del híbrido en esos mismos días, a cambio de un coste acotado en el desempeño global. La variante `hybrid_weighted`, descrita en la sección 4.6, asigna un peso de muestra triple a esas filas (un 8,2 % del conjunto residual) sin modificar ningún otro elemento del modelo. El criterio de éxito, fijado antes de observar el resultado y evaluado mecánicamente en código, exige el cumplimiento simultáneo de dos condiciones: que el MAE en festivos y puentes mejore, y que el MAE global de test no empeore más de un 5 % frente al híbrido original de la sección 5.6.

Criterio	hybrid	hybrid_weighted	Resultado
MAE en días irregulares (festivo + puente), n-ponderado	566.472	585.418	No mejora
MAE global test (techo +5 % sobre 234.223)	234.223	220.286	Dentro del margen
Veredicto del criterio pre-registrado (ambas condiciones)	—	—	NO CUMPLIDO

Cuadro 5.10: Veredicto del criterio pre-registrado del experimento A (repesado 3x de días irregulares). **CRITERIO NO CUMPLIDO:** el presupuesto global se cumplió pero el error en días irregulares NO mejoró.

El veredicto de la tabla 5.10 es que el criterio **no se cumple**: la condición de presupuesto global sí se satisface, con un MAE de test que incluso mejora un 5,95 % respecto al híbrido original (234.223 a 220.286), pero el MAE ponderado por tamaño de muestra en festivos y puentes conjuntamente *empeora*, de 566.472 a 585.418. El aspecto más interesante de este resultado nulo no es que el repesado fallara, sino *dónde* recayó la mejora que sí se produjo: no en los días que se pretendía corregir, sino en sábados (mejora del 25 %) y en laborables ordinarios (mejora del 5 %). Con solamente 45 filas repesadas frente a 161 variables de entrada, el peso adicional no parece comprar un mejor ajuste específicamente en esas filas; en su lugar, perturba el ajuste global del modelo, y esa perturbación resulta beneficiar, por las características propias del conjunto de datos, a los grupos más numerosos en lugar de a los grupos objetivo. Se reporta como lo que es: un dato sobre la fragilidad del repesado en muestras pequeñas, no como evidencia a favor ni en contra del esquema de pesos como estrategia general.

Experimento B: ensamble SARIMAX + XGBoost-alone

El segundo experimento no está sujeto a un criterio de éxito binario, sino que constituye un punto de comparación que cualquier evaluación honesta de este proyecto debe

incluir: ¿existe una combinación más sencilla que los propios componentes de una sola etapa? La respuesta se construye sin ningún reentrenamiento adicional, combinando de forma puramente aritmética las predicciones de SARIMAX y de `xgboost_alone` ya calculadas en las secciones anteriores, bajo dos esquemas de ponderación (tabla 5.11): un promedio simple 50/50 (`ensemble_equal`) y un promedio ponderado por el inverso del MAE de validación de cada componente (`ensemble_inverse_mae`).

Variante	Peso SARIMAX	Peso XGBoost-alone
<code>ensemble_equal</code>	0,5000	0,5000
<code>ensemble_inverse_mae</code>	0,4724	0,5276

Cuadro 5.11: Pesos de los dos componentes en cada variante del ensamble.

Como ya adelantó la tabla 5.1, ambas variantes no solo son competitivas, sino que **superan a los dos componentes individuales**, y lo hacen por un margen que una simple media de errores independientes no explicaría por sí sola (el mecanismo exacto de esa ganancia se desarrolla en la sección 5.8). Los dos esquemas de ponderación producen resultados casi idénticos entre sí (una diferencia de MAE de test de apenas el 0,03 %), lo que no es sorprendente dado que los pesos óptimos por inverso del MAE, 0,4724 y 0,5276, se sitúan muy próximos a un reparto igualitario. Por ello, y pese a que `ensemble_inverse_mae` obtiene el MAE de test nominalmente más bajo de todo el proyecto, se recomienda `ensemble_equal` como modelo de referencia para la memoria: ofrece un rendimiento prácticamente indistinguible sin necesitar ningún paso de ajuste que justificar y, a diferencia de la variante ponderada por inverso del MAE —cuyos pesos se derivan del propio MAE de validación y hacen, por tanto, que su cifra de validación sea levemente optimista—, no depende en ningún grado de la partición de validación para fijar sus coeficientes.

5.8. Por qué gana el ensamble: descorrelación de errores

La razón mecánica por la que el ensamble supera a ambos componentes individuales, y no solo el hecho empírico de que lo hace, se puede hacer explícita observando directamente la relación entre los errores de SARIMAX y de `xgboost_alone` en cada día de la partición test (figura 5.9).

La correlación entre ambos residuos es de 0,576: positiva, como cabría esperar de dos modelos que atacan el mismo fenómeno subyacente y comparten, por tanto, una parte de su señal de error, pero claramente inferior a la unidad. Esa distancia respecto a la correlación perfecta es, precisamente, la que hace mecánicamente rentable promediar los dos modelos. Para dos pronósticos con varianza de error igual σ^2 y correlación ρ entre sus errores, la varianza del error de su media aritmética es

$$\text{Var}\left(\frac{\hat{e}_1 + \hat{e}_2}{2}\right) = \frac{\sigma^2(1 + \rho)}{2},$$

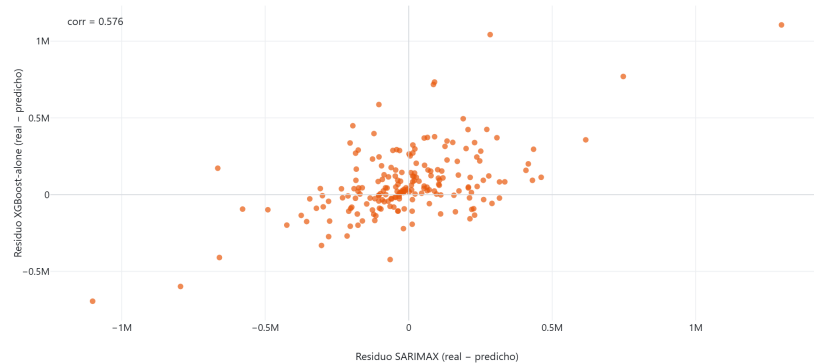


Figura 5.9: Dispersión de residuos SARIMAX vs. XGBoost-alone en la partición test – evidencia mecánica de la reducción de varianza del ensamble (correlación débil entre ambos errores).

la expresión clásica de la combinación de pronósticos de Bates y Granger [3]: con $\rho = 1$, errores perfectamente correlacionados, promediar no reduce la varianza en absoluto, mientras que con $\rho = 0,576$ la varianza del error combinado se reduce a aproximadamente el **79 %** de la varianza individual, incluso bajo el supuesto simplificador de que ambos componentes tuvieran exactamente el mismo error cuadrático medio. Esta reducción de varianza por promediado de predictores parcialmente descorrelacionados es, además, el mismo principio que sustenta el uso de comités o *ensembles* de modelos en el aprendizaje automático en general [15], aplicado aquí no a una colección de redes con la misma arquitectura sino a dos familias de modelos deliberadamente distintas.

La tabla 5.3 explica además por qué esa correlación es precisamente 0,576 y no un valor más cercano a la unidad: SARIMAX y `xgboost_alone` no cometen simplemente errores de magnitud distinta sobre los mismos días, sino errores grandes en *días distintos* (el primero en laborables ordinarios comparativamente, el segundo en fin de semana y festivos), de modo que su correlación de error refleja una complementariedad estructural de calendario y no solo ruido estadístico independiente. La ganancia observada, un MAE de test de 139.681 para `ensemble_inverse_mae`, un 10,5 % por debajo del mejor componente individual (`xgboost_alone`, 156.121), es mayor de lo que un simple promedio de errores sin ninguna estructura común explicaría, lo que confirma que la ganancia procede de esta descorrelación genuina y no de un artefacto de redondeo en la comparación.

5.9. Anatomía del residuo de la etapa 1

La sección 5.6.1 sostuvo, sobre una base teórica, que el híbrido pierde porque su etapa 1 es un cuello de botella: una LSTM univariante sobre unas 860 observaciones diarias no puede ver el calendario, y la etapa 2 arranca entonces desde un residuo cuya desviación típica, 555.385 viajes diarios sobre la serie contigua de validación y test, es del orden de la propia señal. Esta sección deja de afirmarlo y lo mide. Es un análisis

sis **confirmatorio, no exploratorio**: que una red que nunca observa el calendario deje estructura de calendario en su residuo es casi seguro de antemano; la aportación es cuantificar *cuánta*, con criterios de aceptación fijados en el código (constantes de módulo de `stage1_residual_anatomy.py`) antes de observar ningún resultado. Esta sección y las dos siguientes (5.10, 5.11) son **controles diagnósticos**: ninguna de sus cifras entra en la tabla maestra de comparación, en `dashboard_data.MODEL_NAMES` ni en `models/`, y ninguna compite por ser el mejor modelo; una prueba por fase (`test_phase7/8/9_adds_no_model_to_the_master_comparison`) lo fija mecánicamente. En esta, además, no se entrena, reajusta ni re-predice ningún modelo, con la única excepción declarada de la curva de aprendizaje de la sección 5.9.4.

Estructura que retiene el residuo

La tabla E.1 y la figura 5.10 recogen la autocorrelación del residuo de la etapa 1 hasta cinco ciclos semanales. La memoria temporal es **débil y ambigua**: el mayor valor absoluto de la ACF en los retardos 1 a 35 es de solo 0,166 (retardo 28), y aunque cinco de los treinta y cinco retardos quedan fuera de la banda de Bartlett, frente a los dos que cabría esperar por azar al 5 %, ninguna autocorrelación alcanza el umbral de 0,20 fijado como evidencia de estructura persistente. El estadístico de Ljung-Box rechaza la hipótesis de ruido blanco en los tres retardos examinados ($p < 0,01$ en 7, 14 y 28), pero esa prueba es **anticonservadora** en este contexto —su corrección de grados de libertad no está definida para una LSTM sin recuento de parámetros significativo, y sobre un residuo visiblemente autocorrelacionado rechaza con facilidad—, de modo que ninguna afirmación de esta sección descansa en su valor p : el criterio primario es siempre el tamaño del efecto.

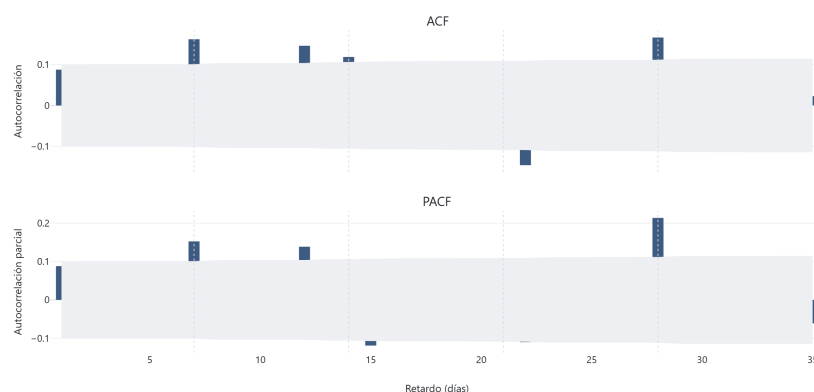


Figura 5.10: ACF y PACF del residuo de la etapa 1 hasta el retardo 35, con la banda de Bartlett sombreada y los retardos semanales (7, 14, 21, 28) marcados. El mayor valor absoluto de la ACF es 0,166 en el retardo 28.

El periodograma (figura 5.11) matiza esta lectura para la estacionalidad semanal. La cuota de potencia espectral concentrada exactamente en el bin de periodo 7 días

es solo 1,4 veces la cuota media, por debajo del factor 3 fijado como umbral, pero esto se debe en buena medida a la fuga espectral: con 393 observaciones no existe un bin de Fourier exactamente en 7,0 días, y la potencia semanal se reparte entre los bins adyacentes (6,4 y 6,7 días) y, sobre todo, hacia el armónico de 7/3 días, que es el pico individual más alto de todo el periodograma. La ACF en el retardo 7 (0,162) y, muy especialmente, el perfil por día de la semana apuntan, por tanto, a una estructura semanal real que la prueba de bin único infravalora.

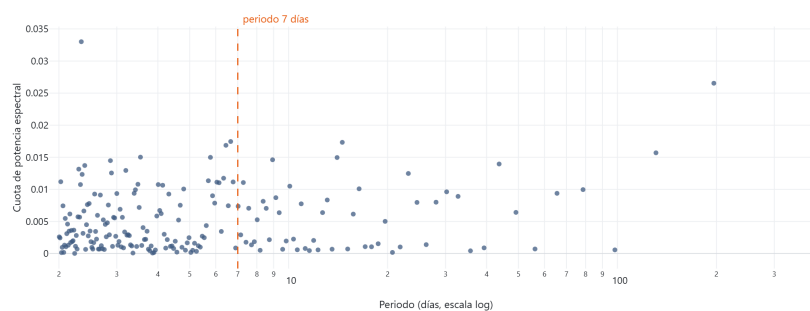


Figura 5.11: Periodograma del residuo de la etapa 1: cuota de potencia espectral frente al periodo en días (escala logarítmica). El periodo de 7 días está anotado; la potencia semanal se dispersa entre bins adyacentes y el armónico de 7/3 días.

La tabla E.2 y la figura 5.12 muestran ese perfil por día de la semana. La dispersión entre la media de residuo más alta y la más baja equivale a 0,60 veces la desviación típica del residuo, muy por encima del umbral de $0,30 \sigma$ para considerar que hay estructura: la etapa 1 sobrepredice sistemáticamente los sábados (residuo medio $-190,065$, es decir $-0,34 \sigma$) y los jueves ($-0,31 \sigma$), e infrapredice los lunes y los viernes ($+0,26$ y $+0,23 \sigma$). Como la prueba de significación entre grupos de día viola la independencia (los residuos están autocorrelacionados), es este tamaño de efecto, y no un valor p , el que sostiene la conclusión de que persiste un sesgo semanal sistemático que la etapa 1 no elimina.

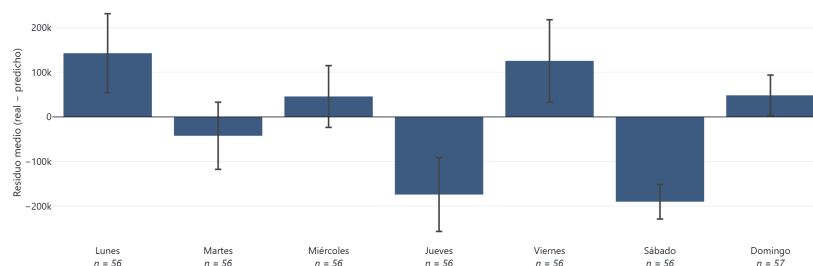


Figura 5.12: Residuo medio de la etapa 1 por día de la semana, con barras de ± 1 error típico y el tamaño muestral bajo cada etiqueta. Una barra positiva indica infrapredicción media de ese día.

Donde el residuo sí retiene estructura de forma inequívoca es en el calendario irregular (tabla E.3). El error absoluto medio de la etapa 1 en los días festivos, 1.818.580 viajes, es **6,52 veces** el de los días laborables ordinarios (279.107), y en los días laborables con puente es 2,96 veces, muy por encima del factor 2 fijado como umbral de concentración. Esta es exactamente la señal que la sección 5.5 identificó del lado de la etapa 2, donde `feat_day_type_festivo` es la variable de mayor ganancia del modelo residual, y coincide con el diagnóstico exploratorio de la fase 4 sobre el periodo completo. Es también el único de los cuatro criterios de aceptación pre-registrados que cae con claridad en el lado de “la estructura persiste”: la etapa 1 univariante deja sin explicar, precisamente, la irregularidad de calendario que por construcción no puede ver.

Señal meteorológica retardada en el residuo

Cabría preguntarse si, además del calendario, el residuo de la etapa 1 retiene señal meteorológica que la etapa 2 no haya sabido extraer. La tabla E.4 y la figura 5.13 responden que **no de forma detectable**. De las 105 variables meteorológicas retardadas (lags de 1, 2, 3 y 7 días y media móvil de 7 días; la meteorología del mismo día está excluida por diseño y la prueba lo reverifica), ninguna mantiene una correlación de Spearman *parcial* estadísticamente significativa con el residuo tras controlar por los cuatro términos anuales de Fourier y aplicar la corrección de Benjamini-Hochberg al 10 %. La brecha entre la correlación bruta y la parcial es, en sí misma, el hallazgo: la humedad relativa máxima retardada un día y la temperatura máxima retardada dos días presentan correlaciones brutas de 0,28 en valor absoluto, pero al partializar por el ciclo anual, que ambas comparten con la demanda, colapsan a 0,11 y 0,09, y no sobreviven a la corrección por comparaciones múltiples.

La lectura correcta de este resultado nulo, a la luz de la objeción metodológica registrada para esta fase, no es que la arquitectura híbrida debiera haber funcionado: la etapa 2 *recibió* esas 105 variables —de hecho, la sección 5.5 mostró que la meteorología retardada concentra el 51,6 % de su ganancia— y aun así el híbrido pierde. Lo que este análisis descarta es que quedara sobre la mesa una palanca exógena meteorológica sin explotar; lo que el residuo retiene es estructura de calendario, no de clima. Que un bloque de variables acapare la mitad de la ganancia por impureza y a la vez no mantenga ninguna correlación parcial detectable con el residuo es solo aparentemente contradictorio; la sección 5.10.3 reconcilia ambos hechos con el tercero que aporta la fase siguiente, la ablación del bloque completo.

La cadena de error, cuantificada

La tabla E.5 y la figura 5.14 cuantifican cuánto de la distancia entre una LSTM univariante y un XGBoost directo cierra realmente la etapa residual. No es una descomposición (`lstm_alone`, `hybrid` y `xgboost_alone` son tres modelos distintos, no sumandos de un mismo error), sino una cadena de tres puntos evaluados con el mismo `metrics.all_metrics` que la tabla maestra, de modo que no puede contradecirla. Sobre la partición

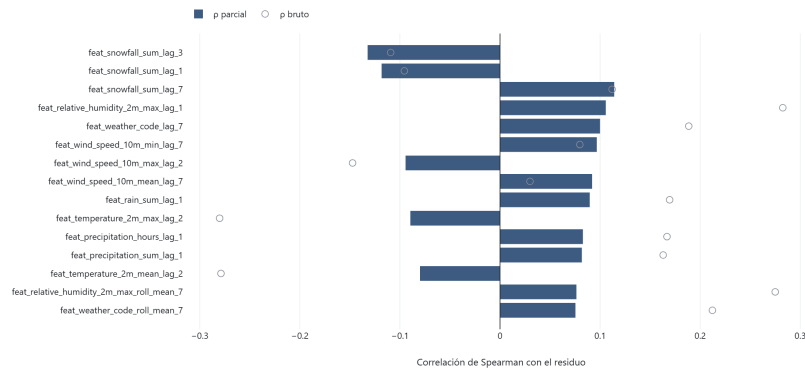


Figura 5.13: Correlación de Spearman del residuo de la etapa 1 con las 15 variables meteorológicas retardadas de mayor $|\rho|$ parcial. El marcador hueco muestra la ρ bruta: la distancia entre ambos es el efecto del ciclo anual compartido.

test, la distancia completa entre `lstm_alone` (MAE 280.952) y `xgboost_alone` (156.121) es de 124.831 viajes diarios. La etapa 2 recorta 46.729 de esa distancia, el **37,4 %** del hueco, dejando todavía 78.103 (el 62,6 %) de separación entre el híbrido corregido y un XGBoost directo sobre las mismas 161 variables. Sobre validación la corrección es proporcionalmente mucho mayor (130.405, el 78,4 % de un hueco más ancho porque `lstm_alone` rinde bastante peor en esa partición), pero los 35.895 viajes restantes siguen favoreciendo a `xgboost_alone`. La conclusión es la misma en ambas particiones y coincide con lo anticipado en la sección 5.6: la etapa residual es una corrección real y sustancial, pero parte de tan atrás que la corrección, por grande que sea, no alcanza a un modelo de una sola etapa.

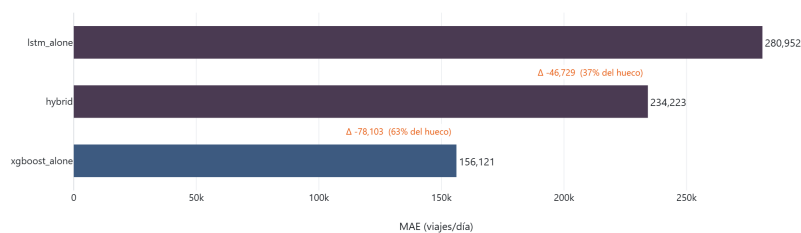


Figura 5.14: Barras escalonadas del MAE de test de los tres modelos de la cadena, con el paso entre barras consecutivas anotado en MAE absoluto y como cuota del hueco total. Deliberadamente no es una cascada con conectores.

Contexto de tamaño muestral

Queda por sustentar con evidencia la afirmación de que la debilidad de la etapa 1 es, en lo esencial, un problema de tamaño de muestra. Un primer indicio, solo un indicio, procede de los cinco folds de validación *out-of-fold*. El MAE por fold no decrece

de forma monótona al crecer el tamaño de entrenamiento: cae de 1.461.905 (fold 1, 200 filas) a 282.122 (fold 3, 474 filas) y vuelve a subir a unos 500.000 en los folds 4 y 5, porque cada fold predice además un periodo distinto y de dificultad distinta. Expresar el error de cada fold como *ratio de habilidad* frente a *seasonal_naive* evaluado sobre exactamente el mismo bloque de fechas atenúa en parte ese factor de confusión: los folds 3 y 5 alcanzan la paridad con el baseline estacional (ratios de 0,95 y 0,98), el fold 1 permanece degenerado (2,67) y los folds 2 y 4 quedan por encima de 1. Con cinco puntos, no monótonos y confundidos con la dificultad del periodo evaluado, esto es contexto indicativo y no una curva de aprendizaje.

La evidencia decisiva sobre el papel del tamaño de muestra procede de una curva de aprendizaje propiamente dicha (tabla E.6 y figura 5.15), el único experimento de esta fase que reentrena. Se ajusta una LSTM de etapa 1, con la arquitectura de la fase 4, sobre ventanas de entrenamiento de tamaño creciente, en seis fracciones del bloque de entrenamiento post-calentamiento (de 222 a 889 filas), con tres semillas por fracción para poder leer la mediana y la banda mín-máx en lugar de una única ejecución ruidosa. El conjunto de evaluación se mantiene fijo (siempre la partición de validación completa) y la partición de test no se consulta en ningún momento. La ventana de entrenamiento es **descendente**, anclada en *train_end* y creciendo hacia atrás: esta elección es deliberadamente conservadora para un experimento de tamaño de muestra, porque las fracciones más pequeñas entrenan sobre las filas temporalmente más próximas a validación y disfrutan, por tanto, de una ventaja de recencia; el diseño está sesgado *en contra* de encontrar un efecto de tamaño de muestra, de modo que un efecto que sobreviva a ese sesgo es más creíble y un resultado plano es correspondientemente más débil de lo que parecería a primera vista.

El resultado es una historia de tamaño de muestra **parcial**, y se reporta como tal con la misma prominencia que tendría un resultado positivo. Por un lado, el tamaño de muestra importa, y mucho, en el extremo bajo: con 222 filas de entrenamiento el modelo **colapsa** (las tres semillas producen ajustes degenerados, con un cociente de desviaciones típicas entre 0,30 y 0,43 y una correlación con la serie real en torno a 0,45, es decir, predicciones cuasi-constantes), exactamente el mismo modo de fallo documentado para el fold 1 en la sección 4.4. Pasar de 222 a 356 filas reduce el MAE de validación mediano de 1.167.187 a 515.603, y a partir de ahí el descenso continúa, pero se aplana: 502.449 con 489 filas, 490.773 con 622, y ya prácticamente estable en torno a 411.000-416.000 con 756 y 889 filas, dentro del ruido entre semillas.

Por otro lado, y esto es lo que la curva zanja, esa meseta se sitúa muy por encima de ambas referencias de una sola etapa: el mejor punto LSTM de toda la curva, unos 411.000 de MAE de validación, sigue estando aproximadamente 166.000 viajes por encima de *xgboost_alone* (244.843) y 138.000 por encima de SARIMAX (273.467), y las tres fracciones superiores no muestran ninguna tendencia a acercarse a esas líneas. La lectura correcta, por tanto, no es que “con más datos la vía LSTM ganaría”: el tamaño de muestra explica por qué la etapa 1 es *muy* débil en lugar de solo débil —el colapso por debajo de unas 350 filas y la fuerte ganancia inicial son reales—, pero el techo de

una arquitectura univariante sobre esta serie se encuentra, incluso con todos los datos disponibles y bajo un diseño de ventana que la favorece, claramente por encima de los modelos de una sola etapa a los que el híbrido aspira a superar.

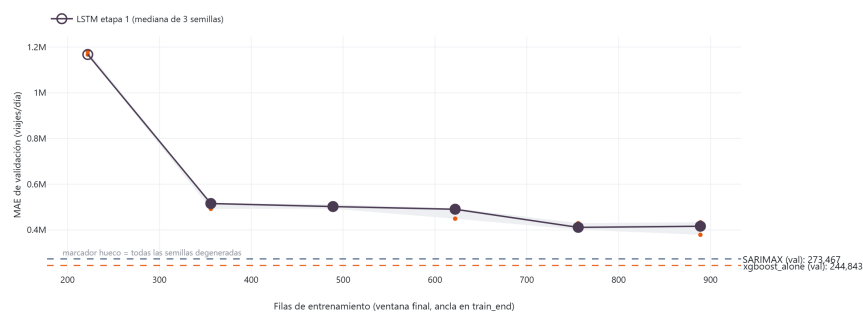


Figura 5.15: MAE de validación de la LSTM de etapa 1 frente a las filas de entrenamiento (ventana descendente anclada en `train_end`): mediana y banda mín-máx de 3 semillas (5 en la fracción 1,00), con las líneas de referencia de SARIMAX y `xgboost_alone` en validación. El marcador hueco indica la fracción cuyas tres semillas degeneraron. La curva se aplana muy por encima de ambas referencias.

5.10. Paridad informativa entre etapas: ablación y control LSTM

La sección 5.9 midió *qué* retiene el residuo de la etapa 1; esta sección se pregunta algo distinto: si la partición del problema en dos etapas está *informativamente* justificada. El análisis anterior deja dos cuestiones abiertas. La primera (Q1): la etapa 1 es univariante mientras que XGBoost ve 161 variables desde el principio, así que ambas etapas no son equivalentes en información; ¿pierde la etapa 1 por ser una *LSTM* o por ser *univariante*? La segunda (Q2): ¿integra ya el XGBoost directo los efectos temporales y exógenos de forma simultánea a través de las variables construidas, de modo que separar el problema en dos etapas añade complejidad sin añadir información predictiva? Esta sección deja de conjeturar y lo mide con dos controles. Como la sección 5.9, es un análisis **confirmatorio** y un control diagnóstico bajo el régimen ya declarado allí; sus dos controles sí reentrenan, pero no persisten ningún artefacto bajo `models/`.

Ablación de bloques de variables sobre `xgboost_alone`

Se reentrena `xgboost_alone` sobre subconjuntos restringidos de variables, manteniendo idénticos la partición de entrenamiento (889 filas tras el descarte de calentamiento), la rejilla de 54 puntos, el *holdout* cronológico interno y el protocolo de evaluación; lo único que cambia es qué bloques son visibles. Los subconjuntos —`completo` (161 variables), `solo_temporal` (39: retardos de `total` y de operadores más medias móviles), `solo_exogeno` (122: calendario, Fourier y meteorología), `sin_meteo` (56) y `solo_`

calendario (17)— se fijaron *antes* de ver ningún resultado, y el criterio de materialidad (un bloque aporta si retirarlo eleva el MAE de validación en al menos un 5 % respecto a completo) es una constante de módulo evaluada en código, no a ojo. El umbral del 5 % absorbe el ruido de rejilla y semilla, ya que cada subconjunto rehace su propia búsqueda de hiperparámetros.

Esto es un **estudio de retirada de bloques, no una descomposición**. Los bloques **no son disjuntos en información**: `fourier_annual` y la meteorología retardada codifican ambos el ciclo anual —la sección 5.9.2 lo midió, viendo colapsar las correlaciones al parcializar por los términos anuales—, y los retardos 7/14/21/28 de `total` codifican implícitamente el calendario semanal. Por tanto, `solo_temporal` sigue “sabiendo” el día de la semana y `solo_exogeno` sigue “sabiendo” la estación. Las columnas de la tabla E.7 no suman a nada y no deben leerse como contribuciones aditivas.

El resultado responde a Q2 de la forma prevista: `solo_temporal` y `solo_exogeno` **empeoran ambos por encima del umbral**, un 88,6 % y un 30,9 % de MAE de validación respectivamente, de modo que el modelo de una sola etapa **integra internamente los dos tipos de información**. Esto es la corroboración mecánica del resultado ya establecido en la sección 5.6 (no una prueba nueva; el capítulo ya demostró la mitad predictiva, que el híbrido pierde): la división en dos etapas hace, a través de etapas, lo que un único modelo tabular ya hace en una. Dos lecturas secundarias, reportadas sin veredicto de aprobado/suspenso: `solo_calendario` (17 variables) queda un 22,8 % por encima de completo, sustancial pero lejos de la paridad, así que la serie no está determinada solo por el calendario determinista; y `sin_meteo` rinde un 3,2 % *mejor* que completo, es decir, el bloque de 105 columnas meteorológicas no se gana su sitio y actúa como ruido neto leve, en línea con el hallazgo de la sección 5.9.2 de que la meteorología retardada es cuasi-ruido-blanco en el residuo. La figura 5.16 ordena los cinco subconjuntos por MAE de validación y superpone el MAE de test de cada uno, de modo que el orden de la ablación pueda leerse de un vistazo junto con su estabilidad entre particiones.

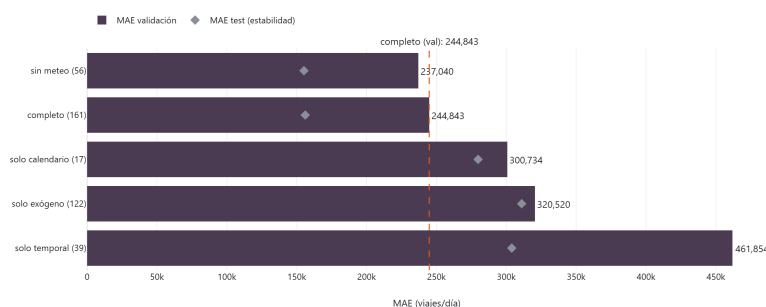


Figura 5.16: MAE de `xgboost_alone` reentrenado sobre subconjuntos restringidos de variables, mejor subconjunto arriba. La línea discontinua marca el MAE de validación de `completo` (244.843) y los rombos el MAE de test como comprobación de estabilidad. Retirar el bloque temporal o el exógeno empeora el modelo por encima del umbral del 5 %; retirar la meteorología no.

LSTM en paridad informativa

Para responder a Q1 se dota a la rama recurrente de **paridad informativa**: la ventana de 28 pasos del objetivo alimenta la LSTM y el vector de variables exógenas `feat_` del *propio día* t se concatena después ($\text{Input}(W, 1) \rightarrow \text{LSTM} \rightarrow \text{concat}(\text{Input}(k)) \rightarrow \text{Dropout} \rightarrow \text{Dense}(1)$). Es el cambio mínimo que logra paridad real: una ventana multivariante (W, k) daría el calendario de los 28 días previos, pero no el del día que se predice, porque la ventana abarca $[t - W, t)$ y excluye la fila t por construcción. Cada columna exógena es la misma fila limpia de fuga que ve XGBoost (guardas de las fases 2 y 3: sin operadores del mismo día, sin meteorología del mismo día), y una aserción reverifica el prefijo `feat_` en el punto de entrada.

La interpretación es asimétrica y se declara antes de los resultados (objeción O4): la LSTM en paridad ve una ventana cruda de 28 pasos *más* el vector del día t , es decir, estrictamente *más* información que el vector plano de XGBoost (que solo dispone de 7 retardos seleccionados y 4 estadísticos móviles). En consecuencia, **una derrota es una conclusión fuerte** (queda descartada la asimetría informativa como causa) y **una victoria es débil** (procede en parte de información extra, no solo de la arquitectura). Y un buen resultado aquí sería un argumento *contra* el híbrido, no a favor de uno mejor: una etapa 1 que ya ve las variables exógenas no deja a la etapa 2 nada que corregir que la etapa 1 no pudiera ver.

Base de comparación y número de semillas. El punto de referencia es la LSTM univariante de la fase 4. Se compara contra su mediana de cinco semillas (42, 1, 2, 3, 4) en la fracción 1,00 de la curva de aprendizaje de la sección 5.9.4, **416.324** de MAE de validación, y no contra su mejor semilla individual (411.142, la cifra citada en la sección 5.12 y en el capítulo 4). El veredicto de paridad se lee sobre una mediana de cinco semillas, así que la base tiene que ser también una mediana de cinco semillas (mismo arnés, mismas semillas, misma partición de validación, genuinamente comparable); enfrentar una mediana contra un mejor caso sesgaría el cierre del hueco a la baja, hacia la conclusión que el TFM ya defiende, que es justo donde el análisis debe ser más estricto. La mediana de cinco semillas resulta coincidir con la de tres (la semilla 1 es el valor central en ambos casos), de modo que el hueco a cerrar sigue siendo $416,324 - 244,843 = 171,481$ y los umbrales no cambian.

El paso de tres a cinco semillas se decidió por la regla preregistrada (secciones de método y objeción O6: subir a cinco solo si la sonda de tiempos muestra holgura, decidido *antes* de la ejecución); la sonda mostró amplia holgura (el ajuste más lento fue de 9,7 s frente a un techo de unos 6 min). La sonda también dejó visibles unos MAE de validación provisionales con semilla 42 (395.274 en `calendario`, 369.835 en `completo`), ambos dentro de la banda ambigua del 25–60 %; se hace constar explícitamente que esas cifras *eran* visibles antes de elegir el número de semillas y que la regla, basada en holgura de cómputo, gobierna con independencia de dónde cayeran.

Lecturas preregistradas. Con un hueco de 171.481, cerrar el ≥ 60 % (MAE de validación mediano $\leq 313,435$) se lee como *asimetría informativa*: la debilidad de la etapa

1 está dominada por lo que no se le dejó ver. Cerrar el $\leq 25\%$ (mediano $\geq 373,454$) se lee como *limitación arquitectónica*: un modelo recurrente simplemente no es competitivo en este problema a este tamaño de muestra, ni siquiera en paridad informativa. Entre ambos, *mixto/ambiguo*. La degeneración se evalúa primero: si la mayoría de semillas de un ámbito degenera, el veredicto es *fallo de capacidad/muestra*. Y si la banda mín-máx cruza un umbral, el veredicto se degrada a *mixto/ambiguo* con independencia de dónde caiga la mediana (objeción O6).

Resultado. Ninguno de los dos ámbitos se acerca a cerrar el hueco (tabla E.8, figura 5.17). Ninguna semilla degenera y ninguna queda infra-entrenada (`best_epoch` mediano 75 y 53, muy por encima del umbral de 20). El ámbito *calendario* ($k = 17$), el más nítido para la afirmación de ceguera al calendario, cierra solo un **16,2 %** del hueco (MAE de validación mediano 388.625; banda 382.695–395.877, sin cruce de umbral): veredicto *limitación arquitectónica*. Es decir, dada exactamente la información de calendario y Fourier donde la sección 5.9.1 localizó la estructura del residuo, un modelo recurrente en paridad informativa apenas recupera una sexta parte de la distancia hasta XGBoost. El ámbito *completo* ($k = 161$) lo hace mejor, cierra un **31,0 %** (mediano 363.133), pero su banda (354.235–391.630) cruza el umbral del 25 %, de modo que el veredicto se degrada a *mixto/ambiguo*; y por la asimetría O4 un cierre parcial es de todos modos evidencia débil. En conjunto, la respuesta a Q1 es que la etapa 1 pierde principalmente por ser *recurrente sobre esta serie y a este tamaño de muestra*, no por ser univariante: la paridad informativa no la rescata.



Figura 5.17: MAE de validación de la LSTM en paridad informativa por ámbito: mediana y banda mín-máx de 5 semillas, con las líneas de referencia de la LSTM univariante (mediana de 5 semillas), SARIMAX y `xgboost_alone` en validación. Ninguno de los dos ámbitos se acerca a la línea de `xgboost_alone`.

Lectura conjunta

Los dos controles apuntan en la misma dirección. La ablación muestra que `xgboost_alone` ya usa de forma demostrable tanto la información temporal como la exógena: no hay un componente redundante que la etapa recurrente del híbrido pudiera aportar por separado. El control de paridad muestra que, aun entregándole a una LSTM

exactamente las variables exógenas que ve XGBoost, e incluso más información en forma de ventana cruda, la vía recurrente no alcanza a los modelos de una sola etapa. La debilidad de la etapa 1, por tanto, no es un problema de acceso a la información que la etapa 2 compense: es la combinación de una arquitectura recurrente y unas 860 observaciones diarias. Esto refuerza, por una vía independiente, la conclusión del contraste central (5.6): para la demanda agregada diaria a este tamaño de muestra, un único modelo de *gradient boosting* sobre variables exógenas bien construidas supera a un híbrido residual secuencial, y el beneficio del híbrido se limita a rescatar una etapa temporal débil en lugar de a batir a modelos de una sola etapa fuertes.

La ablación permite además cerrar una tensión que arrastran las secciones anteriores y que un lector atento habrá advertido. Tres hechos sobre la meteorología retardada parecen incompatibles: la sección 5.5 reporta que ese bloque concentra el 51,6 % de la ganancia del modelo residual; la sección 5.9.2 no encuentra ninguna de sus 105 variables con correlación parcial significativa con el residuo; y la ablación de la sección 5.10.1 muestra que retirarlo por completo *mejora* el MAE de validación en un 3,2 % (237.040 frente a 244.843).

La reconciliación está en qué mide realmente la ganancia por impureza. Esa métrica acumula la reducción de impureza sobre *cada corte* en que interviene una variable y, por tanto, premia sistemáticamente a las variables continuas de alta cardinalidad: una columna meteorológica continua ofrece al algoritmo centenares de puntos de corte candidatos, mientras que una indicadora binaria de calendario ofrece exactamente uno. Con 105 columnas continuas frente a 17 indicadoras y solo 552 filas de entrenamiento OOF, el bloque meteorológico dispone de una enorme superficie de corte sobre la que ajustar ruido, y acumula cuota de ganancia sin aportar señal explotable. Leídos así, los tres hechos dejan de estar en conflicto y el 51,6 % no es una anomalía, sino **evidencia adicional a favor de la tesis de este capítulo**: la etapa 2 dedicó la mayor parte de su capacidad de modelado a un bloque de variables que dos pruebas independientes (una correlacional, otra de retirada) muestran que no aporta nada. Es coherente con ello que la variable individual de mayor ganancia del modelo residual siga siendo `feat_day_type_festivo` (0,125, sección 5.5): el mismo fenómeno visto por el otro lado, con la única variable genuinamente informativa siendo una indicadora de baja cardinalidad cuya cuota *de grupo* queda diluida. La conclusión metodológica, aplicable más allá de este TFM, es que una cuota de ganancia por impureza no es evidencia de utilidad predictiva y no debe leerse como tal sin un contraste de retirada que la respalde.

5.11. Desglose por subgrupos y periodos

El contraste central deja abierta una pregunta concreta: aunque el híbrido no gane globalmente, podría comportarse de forma distinta en festivos, puentes, fines de semana o periodos de demanda anómala. Esta sección la responde de forma sistemática y honesta. Bajo el mismo régimen de control diagnóstico declarado en la sección 5.9, es un reanálisis puro de `data/processed/*_predictions.parquet`: no se entrena, reajusta ni

vuelve a predecir nada, y todas las cifras proceden de `metrics.all_metrics`, la misma ruta que el resto del capítulo.

El riesgo dominante aquí no es la incompletitud, es el *p-hacking*: buscar subgrupos hasta encontrar uno que favorezca al híbrido es *p-hacking* con independencia de por qué se emprenda la búsqueda. Por eso el diseño se fijó **antes** de calcular ningún MAE, ranking o diferencia —solo se contaron los tamaños muestrales—, de modo que “el híbrido no gana en ningún subgrupo” sea tan publicable como cualquier hallazgo positivo.

Diseño preregistrado

Se fijaron **quince subgrupos** como constantes de módulo en `src/evaluation/subgroup_breakdown.py`, contruidos a partir de columnas ya presentes en `features_daily.parquet` (`day_type`, `feat_is_bridge_day`, `feat_is_weekend`, `holiday_name`): regímenes de calendario (laborable ordinario, laborable en puente, sábado, domingo, festivo y sus uniones), estación (julio y agosto frente al resto del año), dos ventanas ancladas en festivo (22 dic–6 ene; Jueves Santo ± 4 días) y demanda anómala. Un día es *anómalo* si $|total_t - total_{t-7}|$ supera 3,0 desviaciones robustas, con la escala (MAD) estimada **solo sobre train** para no ajustar la definición a los datos de evaluación. La regla alternativa de mediana móvil a 28 días se prototipó sobre train y se descartó: en ella domina la estacionalidad semanal, con lo que sería un detector de fines de semana que duplicaría en silencio el subgrupo *domingo*. Esa variante no se reporta.

El umbral de inferencia es $n \geq 20$ (MIN_N_INFERENTIAL), justificado por el apalancamiento de un solo día: a $n = 20$ un día pesa $\leq 5\%$ de la media del grupo; a $n = 5$, un 20 %; a $n = 3$, un 33 %. Cualquier subgrupo por debajo de 20 se calcula y se tabula con su n , se marca “no inferencial” y queda excluido de toda afirmación de victoria o derrota.

Test es la partición de veredicto; validación es el requisito de réplica. Un subgrupo cuenta como victoria del híbrido solo si este queda primero en **ambas** particiones. Este es el dispositivo anti-*p-hacking* más fuerte del diseño: convierte una búsqueda de nueve subgrupos en una que exige réplica independiente, y no cuesta nada porque las dos particiones ya están publicadas. La partición combinada `val_test` (393 días contiguos) es un ámbito **secundario** preregistrado, usado solo de forma descriptiva para los subgrupos que no alcanzan $n = 20$ en test; su coste es que el orden SARIMAX, los hiperparámetros de XGBoost y los pesos del ensamble se eligieron sobre validación, así que sus resultados se etiquetan como secundarios y nunca encabezan una conclusión.

Resolución del ámbito de veredicto para `demanda_anomala`. Este subgrupo, el único fijado por nombre en el diseño preregistrado, tiene $n = 16$ en test y $n = 34$ en validación, y su vista combinada `val_test` alcanza $n = 50$. Recibe el veredicto *no inferencial* en el ámbito de veredicto (test), exactamente igual que cualquier otro subgrupo por debajo del umbral, **sin excepción**. Su resultado combinado se reporta como ámbito secundario y descriptivo, con la salvedad completa ya indicada, y nunca porta un veredicto ni un titular. El ámbito combinado estaba disponible y **deliberadamente no** se promovió a ámbito de veredicto para este subgrupo: elegir por subgrupo qué partición

decide es un grado de libertad del investigador, y adoptarlo precisamente para el subgrupo fijado de antemano es donde menos defendible sería. La función de veredicto lee la constante `VERDICT_SCOPE` de forma uniforme para los quince subgrupos y ningún camino de código asigna un ámbito de veredicto por subgrupo; un test lo fija.

Navidad y Semana Santa no admiten conclusión. La ventana de evaluación de 393 días contiene exactamente una Navidad (solo en validación, $n = 16$; test $n = 0$) y una Semana Santa (solo en test, $n = 10$; validación $n = 0$). Ninguna definición arregla esto: es una propiedad de la partición, no del umbral. Ambos periodos se reportan de forma descriptiva, se marcan *no analizable* y no se emite ninguna afirmación de victoria o derrota sobre Navidad o Semana Santa en este TFM. La lista de subgrupos se fijó por adelantado y añadir uno tras ver los resultados invalidaría la corrección por comparaciones múltiples: es la restricción que prevalece sobre la exhaustividad.

Resultados por subgrupo

Las tablas E.9 y E.10 recogen, por subgrupo y para un conjunto reducido de modelos, el mejor competidor y su MAE, el mejor híbrido y su MAE, la diferencia entre ambos y el veredicto preregistrado. La figura 5.18 lo representa con barras huecas para los subgrupos no inferenciales.

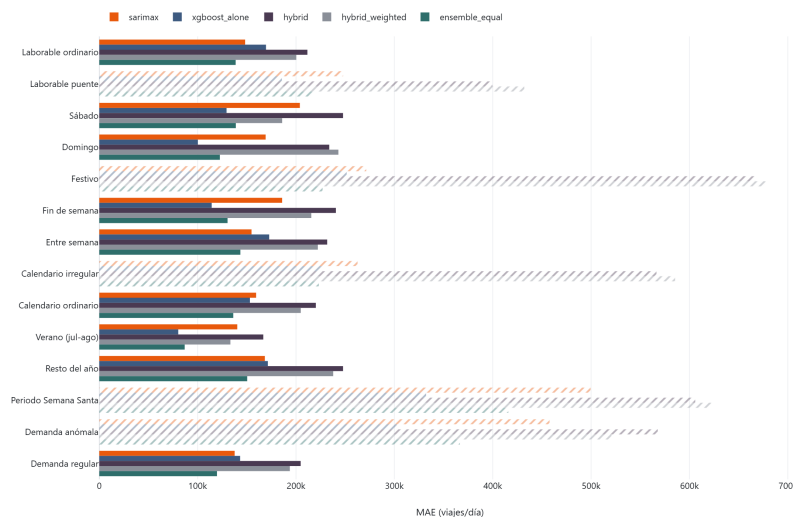


Figura 5.18: MAE por subgrupo en la partición test para el conjunto reducido de modelos. Las barras huecas y traslúcidas corresponden a subgrupos no inferenciales ($n < 20$).

El resultado es inequívoco. De los quince subgrupos preregistrados, nueve alcanzan $n \geq 20$ en test; cuatro quedan por debajo (laborable en puente $n = 3$, festivo $n = 5$, calendario irregular $n = 8$, demanda anómala $n = 16$) y dos están excluidos por falta de solape val/test. **En 0 de los 9 subgrupos con n suficiente gana el híbrido con evidencia**, y en 0 queda siquiera primero sin evidencia: en los nueve, el mejor híbrido es peor que el mejor competidor por entre 53.019 (verano) y 133.578 (domingo) viajes diarios de

MAE. En validación el único subgrupo donde un híbrido queda primero es festivo ($n = 9$), que es no inferencial y además no replica el hecho en test. La búsqueda está hecha y su resultado es negativo.

El único hallazgo positivo ya conocido se mantiene y se reitera: en festivos, la etapa 2 recorta el error de la etapa 1 de 1.585.388 a 666.572 (un -58%). El mecanismo de corrección residual es real; lo que es insuficiente es la etapa 1 de la que parte, y ese recorte no convierte a festivo en una victoria de subgrupo —sigue siendo peor que `xgboost_alone` y que el ensamble, y su $n = 5$ lo hace indicativo, no concluyente—.

La etiqueta `demanda_anomala` usa el total_t observado: es legítima para un reporte a posteriori, pero **ilegítima como regla de enrutamiento**. Ningún lector debe concluir “despléguese el híbrido en los días anómalos”: la anomalía no se conoce antes de observar la demanda del propio día. Su vista combinada `val_test` ($n = 50$, secundaria y descriptiva) tampoco cambia el signo: el mejor híbrido es peor que `xgboost_alone` por unos 123.000 de MAE.

Incertidumbre y comparaciones múltiples

La incertidumbre de cada diferencia se obtiene con un *bootstrap* pareado de bloques móviles. Como un subgrupo categórico está disperso por la partición, el remuestreo se hace sobre la serie de evaluación completa y contigua, en bloques de 7 días (preservan la dependencia semanal, que es la autocorrelación dominante y la razón por la que un test pareado ingenuo sobreestima la significación), con 2000 réplicas y semilla global. La pertenencia al subgrupo viaja con el día, y `best_model` y `best_hybrid` se **vuelven a seleccionar dentro de cada réplica** (*argmin* sobre competidores y sobre `{hybrid, hybrid_weighted}`), de modo que la doble selección se paga en lugar de ignorarse. Una réplica que aporta menos de 20 días del subgrupo se descarta; si la fracción de réplicas utilizables baja de 0,90, el intervalo de confianza se suprime. El cuadro E.11 recoge la diferencia, su intervalo y las dos p , bruta y ajustada, para cada subgrupo de la familia inferencial, y la figura 5.19 representa esas mismas diferencias como gráfico de bosque frente a la línea del cero.

La corrección de Benjamini-Hochberg se aplica con la misma llamada y el mismo $q = 0,10$ que el análisis meteorológico de la sección 5.9, sobre la familia de los nueve subgrupos inferenciales (una hipótesis por subgrupo, no una por subgrupo $\times$ modelo, porque el estadístico focal es una única diferencia). Los subgrupos se solapan fuertemente (`entre_semana` contiene a `laborable_ordinario`), así que los tests son positivamente dependientes —el régimen (PRDS) bajo el que `fdr_bh` sigue siendo válido—; las diferencias entre subgrupos **no descomponen**: es una familia de vistas solapadas, no una partición.

En los nueve subgrupos inferenciales la diferencia es positiva y significativa tras Benjamini-Hochberg: la evidencia es que el híbrido es significativamente **peor**, no que empate. Este es el desenlace más expuesto a sobrelectura, así que su tratamiento está preregistrado. Bajo la hipótesis nula los once modelos son intercambiables dentro de un

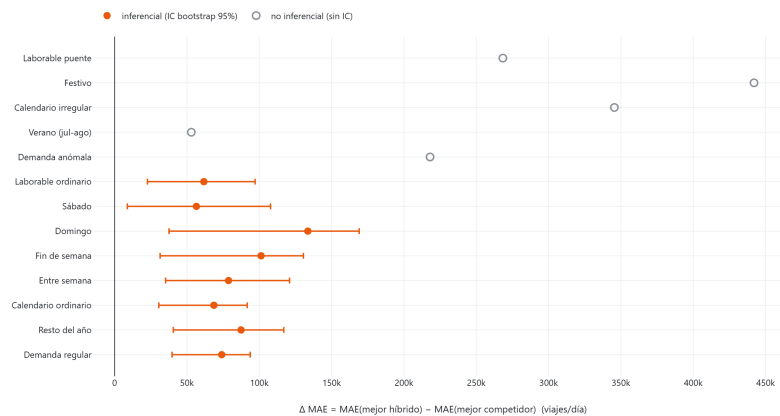


Figura 5.19: Gráfico de bosque de la diferencia de MAE por subgrupo con su IC *bootstrap* del 95 %. La línea del cero es la referencia: un IC íntegramente a su derecha es evidencia de que el híbrido pierde en ese subgrupo. Marcadores huecos y sin IC para los subgrupos no inferenciales.

subgrupo, de modo que la probabilidad de que el MAE más bajo corresponda a la familia híbrida de dos miembros es $2/11$; sobre los nueve subgrupos inferenciales eso da $9 \times 2/11 \approx 1,6$ subgrupos que quedarían primeros por puro azar. El recuento observado es 0, por debajo incluso de esa expectativa nula. Donde la inferencia es imposible —festivo ($n = 5$), laborable en puente ($n = 3$)— no se fabrica ningún valor p: se da un MAE puntual con su n y nada más, que es la respuesta honesta.

Vista temporal

La figura 5.20 traza, sobre los 393 días contiguos de validación y test, el MAE móvil a 28 días de cada modelo (panel superior) y la diferencia móvil entre el mejor híbrido y el mejor competidor (panel inferior), con la frontera val/test marcada. La ventana de 28 días se hereda de $W = 28$ de la LSTM y de la ventana de las variables móviles; no se ha ajustado. La diferencia móvil se mantiene por encima de cero en toda la serie: la posición relativa del híbrido no se invierte en ningún tramo. Es una figura descriptiva: ventanas consecutivas se solapan en 27 días, así que no se le aplica ningún test y las ventanas que cruzan la frontera mezclan las dos particiones.

Lectura

La búsqueda por subgrupos y periodos se ha completado con un resultado negativo bien instrumentado. En los nueve subgrupos con tamaño muestral suficiente el híbrido **no queda primero en ninguno**, ni con evidencia ni sin ella, y en los nueve la evidencia *bootstrap*, corregida por comparaciones múltiples, es que es significativamente peor que el mejor modelo de una sola etapa o el ensamble. En los cuatro subgrupos no inferenciales, incluido el de demanda anómala, no se emite veredicto. El único comportamiento diferencial real del híbrido, el recorte del 58 % del error de la etapa 1 en festivos, es un

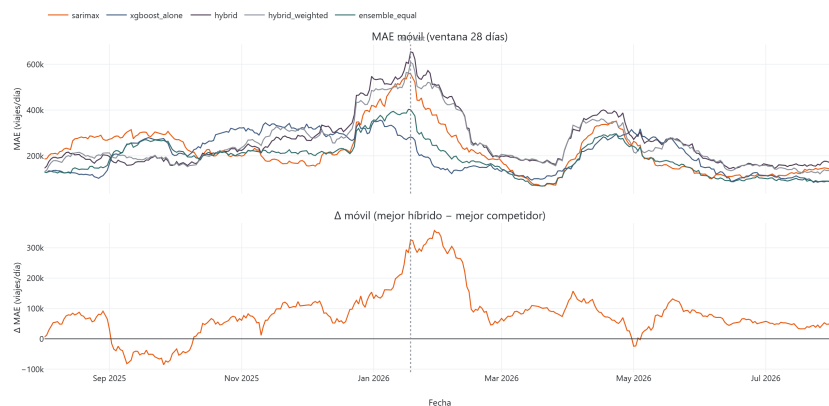


Figura 5.20: MAE móvil a 28 días por modelo (panel superior) y diferencia móvil mejor híbrido – mejor competidor (panel inferior) sobre la ventana contigua de validación y test. Regla vertical en la frontera val/test. Figura descriptiva: sin test, por el fuerte solapamiento de ventanas consecutivas.

mecanismo de corrección, no una victoria, y ya estaba documentado. Esta sección no abre preguntas de investigación nuevas: cierra la que quedaba, confirmando por una vía sistemática la respuesta del contraste central (5.6).

5.12. Respuesta a las preguntas de investigación

Se retoman aquí, en el mismo orden en que se plantearon en la sección 1.2, las cinco preguntas de investigación que estructuran este TFM, respondidas de forma explícita y remitiendo a la evidencia ya presentada en este capítulo.

PI1. ¿Bate el híbrido a SARIMAX en validación y en test simultáneamente?

No, en ninguna de las dos particiones: el MAE del híbrido es 7.270 viajes diarios peor en validación y 70.597 peor en test (tabla 5.9, sección 5.6).

PI2. ¿Bate el híbrido a `xgboost_alone`, entrenado sobre las mismas variables exógenas?

No, y por un margen todavía mayor que frente a SARIMAX: 35.895 en validación y 78.103 en test (tabla 5.9).

PI3. ¿Reduce el híbrido específicamente el error en festivos y puentes frente a SARIMAX y `xgboost_alone`?

No: es peor que ambos en prácticamente todos los grupos de calendario de la tabla 5.3, con la salvedad de que la comparación en festivos ($n=5$) y puentes ($n=3$) en la partición test debe leerse como indicativa, no concluyente, dado su reducido tamaño muestral (sección 5.4).

PI4. ¿Funciona la etapa 2 como mecanismo de corrección residual, aunque el híbrido completo no supere a los modelos de una sola etapa?

Sí: reduce el MAE de la etapa 1 en un 16,6 % en test y, de forma más marcada todavía, en un 58 % en festivos, de 1.585.388 a 666.572 (sección 5.4), y la variable de mayor

ganancia del modelo residual, `feat_day_type_festivo` (0,125), confirma a nivel de variable individual que esa corrección procede exactamente de la señal de calendario que la etapa 1 no puede ver (sección 5.5). El mecanismo de la arquitectura híbrida es real; lo que resulta insuficiente es la etapa 1 de la que parte.

PI5. ¿Existe una combinación más simple que supere a ambos componentes de una sola etapa?

Sí: el ensamble equiponderado 50/50 de SARIMAX y `xgboost_alone` (`ensemble_equal`), sin ninguna red LSTM ni reentrenamiento adicional, alcanza un MAE de test de 139.725; su variante ponderada por el inverso del MAE de validación (`ensemble_inverse_mae`, pesos 0,4724/0,5276) lo reduce ligeramente a 139.681, un 10,5 % por debajo del mejor de sus dos componentes por descorrelación de errores (sección 5.8).

Los dos controles diagnósticos de la sección 5.10 no abren preguntas de investigación nuevas, pero refuerzan las respuestas a PI1, PI2 y PI4 por una vía independiente: la ablación de bloques (5.10.1) confirma mecánicamente que `xgboost_alone` ya integra la información temporal y la exógena —lo que explica por qué separar el problema en dos etapas no añade poder predictivo—, y el control de paridad informativa (5.10.2) descarta que la etapa 1 pierda por ser univariante: sigue por detrás de los modelos de una sola etapa incluso viendo las mismas variables exógenas que XGBoost, con veredictos de *limitación arquitectónica* (`calendario`) y *mixto/ambiguo* (`completo`).

6. Conclusiones y trabajos futuros

Este capítulo de cierre no introduce ninguna cifra nueva: toda afirmación que sigue remite a una tabla o figura ya presentada y discutida en el capítulo 5, del que este capítulo es una síntesis argumental y no una nueva fuente de evidencia.

6.1. Conclusiones

El objetivo general —evaluar si una arquitectura híbrida residual LSTM-XGBoost mejora la predicción de la demanda diaria agregada de transporte público en Madrid frente a baselines estadísticos y a modelos de una sola etapa— se ha cumplido en su totalidad. OE2, OE4 y OE5 quedan satisfechos por las respuestas a las cinco preguntas de investigación de la sección 5.12, aunque no todos en el sentido que la motivación inicial podría sugerir; OE1 y OE3, objetivos de diseño e integridad estructural del pipeline, se verifican mediante la suite de tests del Anexo B: la matriz de 161 variables queda construida y verificada, y ninguna de las guardias anti-fuga ha fallado en ningún momento del proyecto. OE2 aporta además un resultado que gobierna todo lo que sigue: el SARIMAX seleccionado por rejilla AIC no es un baseline decorativo, sino uno de los dos modelos de una sola etapa más precisos del estudio.

El hallazgo empírico central aparece en la evaluación de la corrección residual: **el híbrido, tal como queda diseñado, no supera a SARIMAX ni a `xgboost_alone` en ninguna de las dos particiones** (PI1 y PI2, tabla 5.9), y tampoco reduce de forma diferencial el error en los días de calendario irregular que motivaron su diseño (PI3, tabla 5.3). No es un fallo de ejecución sino un resultado consistente con la teoría de combinación de pronósticos: Granger [9] exige que los componentes sean individualmente subóptimos y se apoyen en conjuntos de información distintos, y esta arquitectura incumple la segunda condición —ambas etapas observan, en última instancia, el mismo pasado de `total`— como se argumenta en la sección 5.6.1. Las competencias M4 y M5 [12, 13] documentan que el patrón —métodos simples y combinaciones bien fundamentadas superando a arquitecturas más sofisticadas, sobre todo en muestras moderadas como las 1.310 observaciones de este proyecto— es esperable y no una anomalía de esta implementación.

Tampoco debe leerse como una arquitectura sin mérito: PI4 confirma que la etapa 2 cumple su función, reduciendo el MAE de su propia etapa 1 en un 16,6 % en test y un 58 % en festivos, y que su variable de mayor ganancia, `feat_day_type_festivo`, es exactamente la señal de calendario que una LSTM univariante no puede ver por construcción (sección 5.5). El mecanismo que justifica la hibridación es real y medible; lo insuficiente es la etapa 1 de la que parte.

El modelo que mejor resuelve el problema no requiere ninguna red neuronal: PI5 confirma que un ensamble aritmético de SARIMAX y `xgboost_alone` —sin reentrena-

miento, sin etapa LSTM y sin ningún paso de ajuste— alcanza un MAE de test de 139.725 equiponderado (`ensemble_equal`) y de 139.681 ponderado por el inverso del MAE (`ensemble_inverse_mae`), en torno a un 10,5 % por debajo del mejor de sus componentes. No es un artefacto estadístico sino la identidad de Bates y Granger [3] aplicada a la correlación de 0,576 entre los residuos de ambos sobre test (sección 5.8): inferior a la unidad porque fallan en subconjuntos de días sistemáticamente distintos —el primero en laborables ordinarios, el segundo en fin de semana y festivos—, que es lo que hace rentable promediarlos.

La lectura conjunta de ambos resultados —el híbrido secuencial que no mejora porque comparte información entre etapas, y el ensamble paralelo que sí mejora porque combina información parcialmente descorrelacionada— es la aportación empírica central de este TFM: una auditoría con protocolo *out-of-fold* estricto en toda su cadena de las condiciones exactas bajo las cuales cada estrategia de combinación funciona y bajo cuáles no, sobre una serie real de demanda de transporte público agregada y multimodal.

6.2. Limitaciones

Agregación a nivel de red, sin corte espacial. Este proyecto modela la demanda total del sistema CRTM como una única serie diaria, con el desglose por operador solo como variable de apoyo (rezagos, EDA) y no como dimensión espacial del modelo. A diferencia de Cardozo et al. [6], que modelan las entradas estación por estación, esta memoria no ofrece resolución espacial alguna: de sus resultados no cabe concluir nada sobre una línea, una estación o un corredor concreto. La elección fue deliberada (sección 2.1), pero es una limitación de alcance.

Tamaño muestral reducido en los días de calendario más interesantes. Con 1.310 observaciones diarias en total, la muestra es modesta para un modelo LSTM, y se vuelve particularmente escasa precisamente en los subgrupos de calendario cuyo comportamiento resulta más interesante para este estudio: la partición de test contiene solo 5 días festivos y 3 días de laborable con puente (tabla 5.3). Todas las afirmaciones de esta memoria sobre el comportamiento de los modelos en esos dos grupos, incluida la mejora del 58 % de la etapa 2 sobre su etapa 1 en festivos, se han etiquetado consistentemente como indicativas y no concluyentes; la base más sólida para generalizar sobre el comportamiento estructural frente al calendario sigue siendo el diagnóstico exploratorio de la fase 4 sobre el periodo completo (48 festivos y 54 puentes; la detección de calendario identifica 55 días puente sobre los 1.310 días totales, de los que 54 caen fuera de los 28 días de calentamiento excluidos del entrenamiento), no las particiones reducidas de evaluación final.

Recorte de 28 días por calentamiento de las variables de retardo. El lag y la media móvil de mayor ventana (28 días) obligan a descartar los 28 primeros días de la serie de entrenamiento (2023-01-01 a 2023-01-28) antes de que la matriz de características quede libre de valores nulos (sección 3.5). Esta política, aplicada únicamente sobre el bloque de entrenamiento y verificada para no alcanzar nunca las particiones de vali-

dación o test, reduce el conjunto de entrenamiento efectivo de 917 a 889 días: un coste aceptado y documentado, pero un coste real sobre el volumen de datos disponible para el ajuste de los modelos.

Naturaleza provisional de los registros de demanda del CRTM. El propio fichero fuente de demanda (CRTM_Evolucion_demanda_diaria.xlsx) advierte explícitamente que sus cifras son provisionales y sujetas a revisión posterior por el consorcio. Este TFM no tiene control sobre esa advertencia ni forma de cuantificar la magnitud de una eventual revisión futura; toda cifra de demanda y todo error de modelo reportado en esta memoria hereda, por tanto, la incertidumbre de la fuente original.

Exclusión deliberada de la meteorología del mismo día. Se excluye por diseño la meteorología concurrente del día predicho, empleando solo sus rezagos de 1, 2, 3 y 7 días y su media móvil de 7 (sección 3.4.1), para no asumir en evaluación un pronóstico atmosférico perfecto que no existe en producción. La decisión es conservadora y correcta para el objetivo del trabajo, pero implica que ninguna cifra de esta memoria estima la mejora alcanzable con un pronóstico meteorológico operativo de calidad: esa cota queda fuera de alcance y se retoma como línea futura.

Veredictos parcialmente ambiguos en la anatomía del residuo. El diagnóstico del residuo de la etapa 1 (sección 5.9) evaluó cuatro criterios de aceptación fijados por adelantado, y solo dos cayeron con claridad en uno de los dos extremos: la concentración del error en el calendario irregular (“la estructura persiste”, factor 6,52 en festivos) y la ausencia de señal meteorológica retardada detectable (“ruido cuasi-blanco”, ninguna correlación parcial significativa tras Benjamini-Hochberg). Los otros dos, la memoria temporal general y la estacionalidad semanal, quedaron en la banda intermedia: la mayor autocorrelación del residuo es 0,166, por debajo del umbral de 0,20 fijado como evidencia de estructura, pero por encima del de 0,10 que indicaría ruido, y el pico espectral de periodo 7 días es débil por fuga espectral. En consecuencia, la afirmación de que la etapa 1 deja sin explicar estructura *semanal* descansa en el tamaño del efecto del perfil por día de la semana (dispersión de 0,60 desviaciones típicas) y no en los criterios de autocorrelación o espectrales, que son individualmente inconcluyentes. El hallazgo central de esa sección (que el residuo retiene estructura de *calendario*, no de clima) no se ve afectado, pero la caracterización de su componente temporal fino es más matizada de lo que un veredicto binario sugeriría.

Veredicto ambiguo en el control de paridad informativa con matriz completa. El control de paridad informativa (sección 5.10.2) evaluó dos ámbitos de variables exógenas con criterios de cierre de hueco fijados por adelantado. El ámbito *calendario* ($k = 17$) cayó con claridad en *limitación arquitectónica*: cierra solo un 16,2 % del hueco hasta *xgboost_alone*, con la banda mín-máx de cinco semillas por completo dentro de la zona de ese veredicto. Pero el ámbito *completo* ($k = 161$) cerró un 31,0 % y su banda (354.235–391.630) cruza el umbral del 25 %, de modo que su veredicto se degrada a *mixto/ambiguo* por la regla de cruce de banda (objeción O6). La conclusión de la sección —que la etapa 1 pierde por ser recurrente sobre esta serie y a este tamaño de muestra, y no por ser univariante— se apoya en el ámbito *calendario*, que es el más nítido para

la afirmación de ceguera al calendario, y en la asimetría de interpretación O4 (la LSTM en paridad ve estrictamente más información que XGBoost, así que un cierre parcial es evidencia débil). Aun así, no puede descartarse con la matriz completa que una fracción del hueco se explique por acceso a la información y no solo por la arquitectura; discriminar ambas causas con más semillas o un diseño que iguale también la representación de entrada queda fuera del alcance de este TFM.

6.3. Trabajos futuros

Las siguientes líneas de trabajo se enuncian como extensiones metodológicas concretas y justificadas por los propios resultados de este TFM; ninguna de ellas ha sido ejecutada ni evaluada en el marco de este proyecto.

Modelado desagregado por operador. Este TFM modela la demanda total del sistema, preservando metro, emt, carretera y cercanías únicamente como variables de apoyo. Una extensión natural sería replicar el mismo pipeline de ingeniería de características y la misma arquitectura de evaluación (baselines, LSTM *out-of-fold*, XGBoost residual, ensamble) de forma independiente para cada uno de los cuatro operadores, lo que permitiría dos preguntas que este TFM no puede responder: si la ventaja del ensamble sobre el híbrido se mantiene a una escala de demanda menor y con patrones de calendario específicos de cada modo —el perfil de Cercanías, más ligado a la movilidad laboral pendular, es previsiblemente distinto del perfil de la red de EMT—, y si un modelo conjunto multivariante que comparta representación entre operadores aporta algo que cuatro modelos independientes no capturan.

Integración de pronósticos meteorológicos operativos. Tal como se señala en la limitación correspondiente, este TFM emplea exclusivamente meteorología retardada para evitar asumir un pronóstico perfecto. Una extensión natural, y metodológicamente honesta si se ejecuta correctamente, sería sustituir esos rezagos por pronósticos meteorológicos reales de horizonte 1-7 días obtenidos de un proveedor operativo (por ejemplo, la propia API de pronóstico de Open-Meteo, complementaria a su API histórica ya empleada en este proyecto), evaluando el sistema bajo la incertidumbre de pronóstico real en lugar de bajo el valor observado retardado. Esto permitiría cuantificar, con evidencia y no por extrapolación, la brecha entre el límite superior bajo pronóstico perfecto y el desempeño alcanzable en un escenario de producción genuino.

Arquitecturas neuronales basadas en parches o transformers temporales. La etapa 1 de este TFM es deliberadamente una LSTM univariante simple, elegida para aislar con claridad qué puede aportar la dinámica temporal pura antes de introducir información exógena en la etapa 2. Arquitecturas más recientes de pronóstico de series temporales de largo alcance basadas en mecanismos de atención y en la segmentación de la serie en parches temporales han mostrado, en la literatura reciente de aprendizaje profundo para series temporales, capacidad de capturar dependencias estacionales de múltiples escalas, semanal y anual simultáneamente, de forma más directa que una red recurrente clásica. Sustituir la etapa 1 de este TFM por una arquitectura de este ti-

po, entrenada bajo el mismo protocolo estrictamente *out-of-fold* que gobierna todo este proyecto, permitiría comprobar si una etapa 1 más expresiva cierra la brecha frente a SARIMAX que este TFM documenta como su hallazgo central, sin renunciar en ningún momento al rigor de validación que hace de esa comprobación un resultado comparable y no una nueva fuente de sesgo optimista.

6.4. Implicaciones para la planificación

Sin introducir ninguna evidencia nueva, los resultados del capítulo 5 admiten una lectura orientada a la explotación operativa, que es el destino último que motivó este TFM. El ensamble seleccionado predice la demanda diaria agregada de la red con un MAPE de test del **3,13 %** (cuadro 5.1), un margen compatible con los usos de planificación en los que la unidad de decisión es la red completa y el horizonte es de días: dimensionamiento de la oferta de material y de turnos de personal, previsión de carga agregada para la programación de frecuencias, y detección temprana de desviaciones frente a lo esperado. Para esos usos, la recomendación que se desprende de este trabajo es doble y deliberadamente austera: emplear la combinación aritmética de SARIMAX y `xgboost_alone` descrita en la sección 5.7.2, y no una arquitectura recurrente —el capítulo 5 muestra que la etapa LSTM no solo no aporta precisión, sino que su coste de entrenamiento, ajuste y mantenimiento no se traduce en ninguna ventaja medible—; y fijar los pesos a partes iguales, porque la variante ponderada no mejora de forma apreciable y sí añade una dependencia de la partición de validación que habría que rehacer con cada reentrenamiento.

Esa recomendación tiene límites que deben acompañarla siempre, y que son exactamente los de la sección 6.2. El error no se reparte de forma uniforme: se concentra en los días de calendario irregular —festivos y puentes (cuadro 5.3)—, que son precisamente aquellos en los que una decisión de oferta equivocada resulta más costosa, de modo que las cifras de error agregadas no deben trasladarse sin más a la planificación de esos días. El modelo no ofrece resolución espacial alguna por debajo del agregado de red, ni predice la demanda de un operador concreto, por lo que no sustituye a ningún instrumento de planificación de línea, corredor o estación. Y, al excluir por diseño la meteorología del mismo día, su desempeño es el alcanzable sin pronóstico meteorológico: incorporarlo es la vía de mejora inmediata, y es la primera de las líneas enunciadas en la sección 6.3.

6.5. Contribución a los Objetivos de Desarrollo Sostenible

Este trabajo se vincula a la Agenda 2030 de Naciones Unidas a través de dos objetivos. Conviene precisar el alcance de esa vinculación antes de enunciarla, porque el hallazgo central de esta memoria es negativo y una contribución declarada sin esa salvedad sería engañosa: lo que sigue no describe una mejora desplegada, sino una aportación metodológica.

ODS 11, ciudades y comunidades sostenibles (meta 11.2). El objeto de estudio es la demanda agregada del sistema de transporte público de Madrid, entre 1,3 y 6,6 millones de viajes diarios, que es el modo de desplazamiento que la meta 11.2 busca hacer accesible, asequible y sostenible. La predicción a un día alimenta las decisiones de dimensionamiento de flota, turnos de personal y programación de frecuencias descritas en la sección 6.4: una previsión más ajustada permite sostener el nivel de servicio sin sobredimensionar la oferta, que es la forma en que un sistema de transporte público mejora su acceso sin aumentar su coste. La contribución es indirecta —el trabajo no despliega ningún sistema— y está acotada exactamente por la sección 6.2: no hay resolución espacial por debajo del agregado de red y el error se concentra en el calendario irregular, que son los días en los que una decisión de oferta equivocada resulta más costosa.

ODS 13, acción por el clima. La aportación específica de este trabajo a este objetivo no está en la precisión alcanzada, sino en su coste. La sección 5.10 documenta que la etapa recurrente no aporta precisión medible, y el mejor resultado del estudio lo obtiene una combinación aritmética de dos modelos ya entrenados, sin entrenamiento adicional ni aceleración por hardware especializado. En un campo cuya respuesta por defecto ante un problema de predicción es escalar la arquitectura, un resultado negativo bien documentado ahorra ese gasto computacional a quien venga después, y ese ahorro sí es directamente atribuible al trabajo. No se cuantifica aquí ninguna reducción de emisiones ni se afirma relación causal alguna entre la calidad de la predicción y la huella ambiental del sistema de transporte: sostenerlo excedería lo que permiten los datos empleados.

A. Catálogo de variables

Este anexo recoge, con fines de consulta autocontenida, la taxonomía completa de las 161 variables predictoras del modelo, obtenidas mecánicamente mediante `build_features.feature_columns()` sobre la matriz procesada `data/processed/features_daily.parquet` y organizadas en los mismos seis grupos temáticos empleados a lo largo de toda la memoria (en el análisis exploratorio del capítulo 3, en la ingeniería de características de la sección 3.4 y en el análisis de importancia de variables del capítulo 5), de modo que cualquier cifra de importancia de variable citada en esta memoria pueda contrastarse directamente contra el grupo y el recuento exacto de su familia sin necesidad de releer el código fuente.

Toda variable de este catálogo lleva el prefijo `feat_`, una convención que no es meramente estética: el propio código deriva el conjunto de columnas del modelo a partir de ese prefijo, de forma que la matriz de entrada nunca se define mediante una lista mantenida a mano que pudiera desincronizarse silenciosamente del código que realmente construye las variables. Los seis grupos son, en el orden en que aparecen en el cuadro A.1: los siete retardos de la demanda total (`feat_total_lag_{1,2,3,7,14,21,28}`); los 28 retardos de operador, siete por cada uno de los cuatro operadores del CRTM (`feat_{metro,emt,carretera,cercanias}_lag_{1,2,3,7,14,21,28}`), presentes en el modelo pese a que `total` sea el único objetivo de predicción, en cumplimiento de la directriz del contexto técnico del proyecto de no podar los operadores como “columnas sin uso”; las cuatro medias y desviaciones típicas móviles de la demanda total, ventanas de 7 y 28 días, todas construidas con desplazamiento previo estricto (`feat_total_roll_{mean,std}_{7,28}`); los seis términos armónicos de Fourier deterministas para la estacionalidad semanal de primer orden y la anual de primer y segundo orden (`feat_fourier_{weekly,annual}_k{1,2}_{sin,cos}`); las once variables de calendario, que codifican de forma exhaustiva y sin nivel de referencia implícito el tipo de día, el tipo de festividad y los indicadores de fin de semana y de día puente; y, con diferencia el grupo más numeroso, las 105 variables meteorológicas retardadas —21 variables físicas de Open-Meteo multiplicadas por cuatro rezagos (1, 2, 3 y 7 días) más una media móvil de 7 días—, sujetas a la misma prohibición de meteorología del mismo día que se documenta y verifica en el Anexo B.

El detalle de variable individual está en los cuadros de top-15 variables por ganancia de cada modelo XGBoost del capítulo 5, y la lista nominal completa de las 161 columnas la genera `src/features/build_features.py`, consultable en el repositorio del proyecto.

Grupo	N.º variables	Descripción
Retardos de la demanda total	7	lags 1, 2, 3, 7, 14, 21, 28
Retardos de operadores	28	4 operadores x 7 retardos
Medias/desv. móviles de la demanda	4	media y desv. típica, ventanas 7 y 28, shift-first
Términos de Fourier	6	semanal k=1, anual k=1 y k=2
Calendario	11	day_type (5), holiday_type (4), is_weekend, is_bridge_day
Meteorología retardada	105	21 variables x (lags 1/2/3/7 + media móvil 7)
Total	161	

Cuadro A.1: Resumen de los seis grupos de variables del modelo, 161 en total (repetición de la tabla 3.3 para consulta autocontenida del anexo).

B. Suite de tests y garantías anti-fuga

La afirmación central de esta memoria —que un resultado negativo bien instrumentado es una contribución metodológica válida— solo se sostiene si el pipeline que lo produce está libre de los errores de fuga de información que sistemáticamente producen resultados optimistas y no reproducibles en la literatura de predicción de series temporales. Este anexo documenta la arquitectura de verificación que respalda esa afirmación: **470 tests**, recolectados y ejecutados con `python -m pytest` desde la raíz del repositorio con el entorno `.\venv\Scripts\python.exe` activado, de los cuales 257 se acumulan a lo largo de las fases de modelado (0 a 6b, cuadro 1.1) y el resto se añaden durante la redacción de la memoria: auditan la capa de presentación (figuras y cuadros), las tres fases diagnósticas de las secciones 5.9, 5.10 y 5.11, el cuaderno principal de flujo, y la coherencia interna del propio documento: `test_prose_claims_match_artifacts.py` fija contra su artefacto de origen cada cifra de alta relevancia citada en la prosa —métricas de test, tamaños de partición, recuentos de filas y variables— junto con la lista completa de ficheros en que cada una debe aparecer, de modo que una cifra no pueda quedar obsoleta en un capítulo mientras se actualiza en otro. Tres pruebas más fijan mecánicamente que las tres fases diagnósticas, puramente explicativas, no introducen ningún modelo nuevo en la tabla maestra de comparación ni escriben artefacto alguno bajo `models/`.

Una incidencia de la última revisión merece registro porque ilustra el modo de fallo que esta suite existe para evitar. Las dos guardias anti-fuga del ensamblado eran **inalcanzables**: comparaban el bloque de variables contra el nombre *desnudo* de la columna (`metro`, `temperature_2m_mean`), que `feature_columns()` descarta por construcción al filtrar por prefijo, de modo que la lista de infractoras nunca podía tener elementos. Ninguna fuga llegó a producirse —la protección real es estructural: los constructores solo emiten formas retardadas y móviles—, pero una comprobación que promete una garantía que no puede dar es peor que no tenerla, porque el siguiente lector la da por buena. Ahora comparan contra el nombre *prefijado*, que es lo que produciría una regresión de constructor, y disparan.

La suite no se limita a comprobar que el código se ejecuta sin excepciones: una parte sustancial de ella está diseñada específicamente para hacer fallar el pipeline si se reintrodujera, ahora o en una modificación futura, alguna de las siguientes reglas anti-fuga, listadas de la más general a la más específica del dominio de este proyecto.

1. **Desplazamiento previo estricto en todo retardo o estadístico móvil.** Toda media o desviación típica móvil derivada de `total` o de un operador aplica `shift(1)` *antes* de la ventana (`s.shift(1).rolling(w).mean()`, nunca la corrección a posteriori ni una ventana sin desplazar), de forma que ninguna fila conozca su propio

valor dentro de su propia estadística. Verificado en `tests/test_features.py`, con una salvedad documentada: para una ventana *trailing* ambos órdenes son matemáticamente idénticos, y la regla se exige por corrección estructural.

2. **Particiones siempre cronológicas, nunca aleatorias.** Ninguna partición se genera mediante barajado; todas derivan de la única función `src.utils.splits.chronological_split`, con sus fronteras fijadas por `test_split_boundaries_are_exact_and_stable` en `tests/test_splits.py`, incluida la protección contra el error de redondeo que un `int()` ingenuo introduciría al calcular el 70 % de 1.310 filas.
3. **Escalado ajustado exclusivamente sobre entrenamiento.** Todo `MinMaxScaler` de la etapa LSTM se ajusta (`fit`) solo sobre el entrenamiento de su propio fold y se aplica (`transform`) sin reajuste sobre validación y test, nunca mediante un `fit_transform` sobre el conjunto completo; verificado en `tests/test_lstm.py`.
4. **Prohibición de ingeniería de características global antes de la partición**, salvo la única excepción sancionada: los términos de Fourier son una función determinista de la fecha, no una estadística estimada a partir del objetivo, de modo que calcularlos sobre la serie completa equivale a calcularlos por partición; fijado por `test_fourier_is_deterministic_and_split_invariant`.
5. **Predicciones de la etapa 1 siempre fuera de muestra (*out-of-fold*) para alimentar la etapa 2.** Ningún residuo que entrena el XGBoost de la etapa 2 procede de una predicción *in-sample* de la LSTM; el esquema *walk-forward* expansivo de 5 folds y la exclusión documentada del fold 1 degenerado se verifican en `tests/test_lstm.py` y `tests/test_hybrid_residual.py`. Es la decisión de diseño fijada por escrito como no negociable con anterioridad a cualquier resultado.
6. **Guardia de fuga *same-day* sobre la suma de operadores.** Puesto que `metro + emt + carretera + cercanias == total` de forma exacta, un operador del mismo día no es variable predictiva sino el objetivo expresado de otra forma; `lag_features._assert_no_same_day_leakage` hace fallar la construcción si alguno apareciera sin retardo, comprobado sobre la matriz ensamblada y no solo módulo a módulo.
7. **Guardia de fuga *same-day* sobre meteorología.** Ninguna de las 21 variables meteorológicas del día predicho entra en la matriz; solo sus versiones retardadas (1, 2, 3, 7 días y media móvil de 7), verificado por `weather_features._assert_no_same_day_weather` con el mismo criterio de doble comprobación que la guardia de operadores.
8. **El modelo de control `xgboost_alone` entrena sobre las 889 filas completas de entrenamiento, no sobre las 552 del conjunto residual.** No depende de los folds

out-of-fold de la LSTM, y limitar sus datos por una razón ajena a su diseño penalizaría artificialmente al control frente al híbrido; fijado por `test_xgboost_alone_trains_on_the_full_889_row_split`.

9. **El veredicto del experimento A de sensibilidad se evalúa en código, no a ojo.**

El criterio pre-registrado de la sección 5.7.1 (mejora en días irregulares y coste global acotado al 5 %) se calcula mediante `evaluate_criterion_a`, de forma que el veredicto “NO CUMPLIDO” no pueda derivar a posteriori hacia una lectura más favorable; verificado en `tests/test_sensitivity_analysis.py`.

10. **El tema visual institucional es un objeto compartido, no una paleta duplicada.**

El cuadro de mando interactivo y la exportación estática importan literalmente el mismo objeto `VIU_TEMPLATE`, no dos definiciones que pudieran divergir en silencio; verificado por `test_interactive_and_static_paths_share_one_template_object` en `tests/test_theme.py`. No es anti-fuga en sentido estricto, pero cumple para la capa de presentación el mismo papel de garantía estructural.

Cada una de estas diez comprobaciones incluye, además de su verificación sobre datos reales, al menos una prueba negativa que inyecta deliberadamente la condición de fuga correspondiente sobre un caso de prueba mínimo y comprueba que el código *sí* falla; una guardia que nunca se ha visto obligada a disparar en la práctica es indistinguible de una guardia que no existe, y este proyecto no se conforma con esa ambigüedad.

C. Reproducibilidad y entorno

Este anexo documenta lo estrictamente necesario para regenerar, desde una instalación limpia, cualquier cifra, tabla o figura de esta memoria, sin dejar ninguna decisión de entorno implícita en la memoria de quien ejecutó el pipeline originalmente.

Repositorio. Todo el código de este trabajo —ingesta, ingeniería de características, modelos, evaluación, cuadernos y las fuentes $\text{\LaTeX}$ de esta memoria— es de acceso público en <https://github.com/AdayCuestaCorrea/TFM>. Las menciones a “el repositorio” en los anexos de esta memoria se refieren siempre a esa dirección. Los datos de partida no se versionan allí: proceden de tres fuentes abiertas cuyo origen, formato y convenciones de lectura detalla el cuadro 3.1, y hay que descargarlos y colocarlos sin renombrar en `data/raw/` antes del primer comando de la secuencia siguiente, que es el único punto del pipeline que los lee.

Entorno de ejecución. Python 3.13.5, en un entorno virtual dedicado (`./venv`), con todas las dependencias fijadas por versión exacta, nunca por rango, en `requirements.txt`, entre ellas:

- `numpy==2.2.6`, `pandas==2.3.3`, `scikit-learn==1.7.2`
- `xgboost==3.0.5`, `tensorflow==2.20.0`, `statsmodels==0.14.6`
- `plotly==6.3.0`, `pyarrow==22.0.0`, `kaleido==1.3.0`

Dos pines documentan una incompatibilidad real y no una preferencia: `pyarrow`, porque `pandas` necesita un motor Parquet explícito, y `kaleido` en su rama 1.x y nunca en la 0.2.1, porque esta cuelga de forma indefinida y sin excepción al exportar figuras bajo `plotly 6.x` —un fallo silencioso, más costoso de diagnosticar que un error de instalación.

Fijación de semillas aleatorias. La semilla global `GLOBAL_SEED = 42` se define una sola vez, en `src/Utils/seed.py`, y desde ahí se aplica a `random`, `numpy` y `tensorflow` en todo punto de entrada que entrena; ningún módulo de `src/models/` la redefine localmente. Además de sembrar los generadores, `set_global_seed` exige a TensorFlow núcleos deterministas (`TF_DETERMINISTIC_OPS=1` y `tf.config.experimental.enable_op_determinism()`): sembrar por sí solo no garantiza resultados bit a bit idénticos si el motor conserva reducciones no deterministas, y es la reproducibilidad bit a bit la que este trabajo se fijó como convención.

Comandos de reproducción de principio a fin. Partiendo de un clon del repositorio, el entorno se crea con `python -m venv venv` y se puebla con `pip install -r requirements.txt`. Ejecutados después en este orden desde la raíz del repositorio, con el entorno virtual activado (`.\venv\Scripts\Activate.ps1` en PowerShell), regeneran de forma determinista la totalidad de los artefactos citados en esta memoria, sin reentrenar ni repredecir ningún modelo más allá de lo que cada paso indica explícitamente:

1. `python -m src.ingestion.unify` — unifica las tres fuentes crudas en `data/interim/unified_daily.parquet`.
2. `python -m src.features.build_features` — construye la matriz de 161 variables en `data/processed/features_daily.parquet`.

3. `python -m src.models.baselines.run_baselines` — baselines y SARIMAX seleccionados por AIC (6 min, dominados por la rejilla de búsqueda).
4. `python -m src.models.lstm.window_comparison` — ventanas $W \in \{14, 28, 60\}$ sobre validación; `lstm_window_comparison.parquet` (tabla 4.2).
5. `python -m src.models.lstm.oof` — *walk-forward* expansivo de 5 folds (sección 4.4); `lstm_oof_predictions.parquet` y `lstm_oof_fold_log.parquet`.
6. `python -m src.models.lstm.final_model` — etapa 1 final sobre las 889 filas; `lstm_val_test_predictions.parquet` y `models/lstm_stage1_final.keras`.
7. `python -m src.models.hybrid_residual.xgboost_residual` — etapa 2 sobre el conjunto residual de 552 filas.
8. `python -m src.models.hybrid_residual.xgboost_alone` — modelo de control sobre las 889 filas de entrenamiento.
9. `python -m src.models.hybrid_residual.combine` — combina las etapas 1 y 2 en la predicción híbrida final.
10. `python -m src.evaluation.full_comparison` — tabla maestra de comparación; `full_comparison.parquet`, contra la que se verifican la ablación de bloques y el cuaderno principal de flujo.
11. `python -m src.models.hybrid_residual.xgboost_residual_weighted` — experimento A (sección 5.7.1): peso triple en días de calendario irregular.
12. `python -m src.evaluation.ensemble_baseline` — experimento B (sección 5.8): ensamble SARIMAX + XGBoost-alone, sin reentrenamiento.
13. `python -m src.evaluation.sensitivity_report` — consolida ambos y evalúa en código el criterio pre-registrado del experimento A; `sensitivity_comparison.parquet`.
14. `python -m src.evaluation.lstm_learning_curve` — sección 5.9: 20 ajustes de la etapa 1 (6 fracciones x 3 semillas, más las semillas 3 y 4 en la fracción 1,00, base del control de la sección 5.10.2), solo validación; `lstm_learning_curve.parquet`.
15. `python -m src.evaluation.stage1_residual_anatomy` — tablas de la anatomía del residuo y veredictos de los cuatro criterios pre-registrados.
16. `python -m src.evaluation.feature_block_ablation` — sección 5.10: `xgboost_alone` sobre cuatro subconjuntos de variables; `feature_block_ablation.parquet`.
17. `python -m src.evaluation.subgroup_breakdown` — sección 5.11: reanálisis por quince subgrupos pre-registrados, con *bootstrap* de bloques móviles y corrección Benjamini-Hochberg; `subgroup_breakdown.parquet` y `subgroup_rolling_mae.parquet`.
18. `python -m src.evaluation.lstm_parity_control` — sección 5.10.2: LSTM en paridad informativa, 10 ajustes (2 ámbitos x 5 semillas), solo validación; `lstm_parity_control.parquet`.
19. `python -m src.evaluation.export_figures` — exporta las 34 figuras estáticas de `reports/figures/`, fuera de todo kernel de Jupyter.
20. `python -m src.evaluation.export_latex_tables` — vuelca las 32 tablas LaTeX de `docs/LaTeX/tables/`.
21. `jupyter nbconvert --to notebook --execute --inplace notebooks/07_flujo_completo.ipynb` — ejecuta el cuaderno principal de flujo (40 s) y **contrasta cada métrica recomputada contra el valor publicado** en `full_comparison.parquet`: si alguna se desvía más de 10^{-6} en términos relativos, levanta una excepción.
22. `python -m pytest` — ejecuta la suite completa de 470 tests (descrita en el Anexo B).
23. `latexmk -pdf -interaction=nonstopmode -halt-on-error TFT.tex`, ejecutado desde `docs/LaTeX/`, compila esta memoria hasta obtener `TFT.pdf`.

Cuadro de mando interactivo. La etapa de despliegue del ciclo KDD (sección 1.3) se materializa en el cuaderno `notebooks/06_dashboard.ipynb`, que se abre con `jupyter lab` una vez ejecutados los pasos anteriores y presenta, sobre los artefactos ya persistidos, los cinco elementos de visualización comprometidos en la propuesta inicial del trabajo: demanda real, demanda predicha, comparación entre modelos, error de predicción e importancia de variables del modelo XGBoost. El cuaderno lee de `data/processed/` y de los modelos ya entrenados de `models/`, y no entrena ni predice nada; la exportación de las figuras estáticas de esta memoria se realiza aparte, mediante `export_figures`, fuera de todo kernel de Jupyter.

Cuaderno principal de flujo. El cuaderno `notebooks/07_flujo_completo.ipynb` recorre el trabajo completo en las seis etapas del ciclo —carga y comprobación de datos, preparación de variables, partición temporal, ejecución de los modelos principales, comparación de resultados y gráficos finales—, importando de `src/` sin duplicar lógica. Se distingue del cuadro de mando anterior en propósito y en método: aquel visualiza artefactos ya persistidos; este **reejecuta el pipeline**, recomputando en vivo la ingesta y sus validaciones, la matriz de 161 variables, la partición cronológica, los cuatro baselines, el SARIMAX —del que solo se carga el orden $(2, 1, 2) \times (0, 1, 2, 7)$ ya seleccionado por AIC— y los dos modelos XGBoost. La etapa 1 LSTM es el único modelo que se carga de artefacto, por una razón metodológica y no de coste: la etapa 2 debe corregir los residuos del mismo modelo que produjo los resultados aquí reportados, y reentrenar la red los calcularía contra un modelo distinto del que documenta esta memoria. Se versiona *con sus salidas guardadas*, de forma que pueda leerse sin ejecutarlo, y no escribe nada en `data/` ni en `models/`.

Identidad visual institucional. El color naranja empleado en la memoria es una **aproximación** documentada del color corporativo de la universidad, no el valor oficial —no disponible durante el desarrollo del proyecto—, y se codifica literalmente en **exactamente dos puntos**, uno por capa: la capa de figuras en `VIU_COLORS["primary"] = "#E8590C"` (`src/evaluation/theme.py`) y la capa $\text{\LaTeX}$ en `docs/LaTeX/preamble.tex` (`\definecolor de naranja y slcolor`). Sustituirlo por el valor oficial es, por diseño, un cambio de dos líneas que se propaga a todas las figuras y cuadros sin tocar ningún otro fichero.

D. Declaración de uso de inteligencia artificial generativa

Este anexo declara el uso de herramientas de inteligencia artificial generativa en la realización de este Trabajo Fin de Máster, delimita las tareas en las que se emplearon y deja explícito qué decisiones permanecen, sin excepción, bajo la responsabilidad del autor.

D.1. Herramientas empleadas y alcance

En la realización de este Trabajo Fin de Máster se han utilizado herramientas de inteligencia artificial generativa, en concreto Claude (Anthropic), en su interfaz conversacional y en su versión para desarrollo de software (Claude Code), durante el periodo abril–septiembre de 2026 [1, 2].

Su uso se ha concentrado en cuatro tareas.

Implementación del código. La escritura de los módulos de Python del repositorio (ingesta, ingeniería de características, modelado, evaluación y generación de figuras y tablas), así como de la suite de 470 pruebas automáticas que verifica su comportamiento (Anexo B), se ha realizado con asistencia de IA a partir de especificaciones técnicas definidas por el autor. Las decisiones de arquitectura del pipeline, el protocolo *out-of-fold*, las reglas anti-fuga, la partición cronológica y el diseño de los controles diagnósticos fueron fijadas por el autor antes de su implementación y verificadas contra los resultados obtenidos.

Redacción de la memoria. El texto de este documento se ha redactado con apoyo de IA a partir de los resultados experimentales, las decisiones metodológicas y la estructura argumental establecidos por el autor. Los borradores generados han sido revisados y validados por el autor, que asume la responsabilidad sobre su contenido.

Revisión y auditoría documental. Se ha empleado asistencia de IA para verificar la correspondencia entre las cifras citadas en la memoria y los artefactos generados por el código, detectar inconsistencias internas y comprobar el cumplimiento de los requisitos formales.

Búsqueda bibliográfica. Al inicio del desarrollo de este TFM el autor empleó asistencia de IA para localizar de forma expeditiva las referencias del proyecto. Cada ficha fue revisada después de forma independiente: se comprobó su existencia, se consultó la fuente primaria y, cuando correspondía, se descargó el documento original. Las referencias de esta memoria no son citas inventadas ni alucinadas por el modelo; todas están verificadas contra su fuente.

D.2. Tareas no delegadas

La definición del problema y de los objetivos, la selección de las fuentes de datos, las decisiones metodológicas —incluidas las que fijaron criterios de aceptación antes de observar resultados—, la interpretación de los hallazgos y la formulación de las conclusiones son responsabilidad del autor. En particular, la decisión de mantener y documentar el resultado negativo de la arquitectura híbrida, en lugar de explorar configuraciones alternativas hasta obtener un resultado favorable, es una decisión metodológica del autor y constituye la aportación central del trabajo.

El autor asume la responsabilidad plena sobre la metodología, los datos, los resultados, las referencias y el contenido final de esta memoria.

D.3. Referencia de las herramientas

Las fichas de las herramientas empleadas, recogidas también en la bibliografía general, son las siguientes:

- Anthropic. (2026). *Claude* [modelo de lenguaje de gran tamaño]. <https://claude.ai>.
- Anthropic. (2026). *Claude Code* [herramienta de desarrollo de software asistido por IA]. <https://claude.com/claude-code>.

E. Cuadros diagnósticos extendidos

Este anexo recoge los cuadros de detalle de las secciones 5.9, 5.10 y 5.11. El argumento, las cifras titulares y los veredictos preregistrados permanecen íntegros en el cuerpo del capítulo 5; aquí solo se traslada el detalle tabular, al que el texto sigue remitiendo por número de cuadro.

Retardo (días)	ACF	PACF	Fuera de banda	Ljung-Box Q	p-valor (LB)
1	0,088	0,088	No	—	—
2	0,066	0,059	No	—	—
3	0,020	0,009	No	—	—
7	0,162	0,153	Sí	19,48	0,0068
14	0,119	0,084	Sí	35,42	0,0013
21	0,030	0,024	No	—	—
28	0,166	0,214	Sí	68,77	0,0000

Cuadro E.1: Autocorrelación (ACF) y autocorrelación parcial (PACF) del residuo de la etapa 1 en los retardos semanales, con las columnas de Ljung-Box (descriptivas; ver el texto). Serie primaria: modelo final sobre 393 días contiguos de validación y test.

Día	n	Residuo medio	Mediana	Desv. típica	Residuo medio	Media / sigma
Lunes	56	142.660	283.700	663.544	479.203	0,257
Martes	56	-42.271	44.308	563.530	346.908	-0,076
Miércoles	56	45.764	154.731	517.848	313.560	0,082
Jueves	56	-173.984	-76.090	618.420	279.800	-0,313
Viernes	56	125.534	276.219	690.421	489.159	0,226
Sábado	56	-190.065	-227.190	289.830	287.858	-0,342
Domingo	57	48.428	106.565	341.577	226.810	0,087

Cuadro E.2: Perfil del residuo de la etapa 1 por día de la semana: media, mediana, desviación típica, error absoluto medio y media relativa a la σ global del residuo. Serie primaria (393 días).

Dimensión	Grupo	n	Residuo medio	Error abs. medio (%)
day_type	domingo	57	226.810	8,29
day_type	festivo	14	1.818.580	78,98
day_type	laborable	269	309.658	6,08
day_type	sabado	53	268.777	7,69
day_type	laborable, bridge day	15	826.986	21,34
day_type	laborable, ordinary	254	279.107	5,17
holiday_type	festivo local de la ciudad de madrid	2	1.960.171	54,10
holiday_type	festivo nacional	11	1.931.389	89,82
holiday_type	festivo de la comunidad de madrid	1	294.503	9,58
holiday_type	none	379	291.481	6,63

Cuadro E.3: Magnitud del residuo de la etapa 1 por régimen de calendario (tipo de día y tipo de festivo). El residuo se concentra 6,52x en festivos y 2,96x en puentes frente a los laborables ordinarios. La fila “laborable” ($n = 269$) es la suma de “laborable, ordinary” (254) y “laborable, bridge day” (15); no es un grupo adicional independiente.

Variable meteorológica	Retardo	Rho bruto	Rho parcial	p parcial	p ajustado (BH)	Significativa
snowfall_sum	lag_3	-0,109	-0,132	0,0090	0,8149	No
snowfall_sum	lag_1	-0,096	-0,118	0,0196	0,8149	No
snowfall_sum	lag_7	0,112	0,114	0,0244	0,8149	No
relative_humidity_2m_max	lag_1	0,283	0,106	0,0372	0,8149	No
weather_code	lag_7	0,189	0,100	0,0489	0,8149	No
wind_speed_10m_min	lag_7	0,080	0,097	0,0567	0,8149	No
wind_speed_10m_max	lag_2	-0,147	-0,094	0,0630	0,8149	No
wind_speed_10m_mean	lag_7	0,030	0,092	0,0699	0,8149	No
rain_sum	lag_1	0,169	0,090	0,0774	0,8149	No
temperature_2m_max	lag_2	-0,280	-0,090	0,0776	0,8149	No

Cuadro E.4: Diez variables meteorológicas retardadas con mayor correlación parcial (en valor absoluto) con el residuo de la etapa 1. “Rho parcial” controla por los cuatro términos anuales de Fourier; ninguna es significativa tras Benjamini-Hochberg ($q=0,10$).

Partición	Modelo	MAE	RMSE	MAPE (%)	R ²	Delta MAE	Delta MAE (%)	Cuota del hueco (%)
test	lstm_alone	280.952	431.714	6,62	0,8948	0	0,0	0,0
test	hybrid	234.223	328.131	5,49	0,9392	-46.729	-16,6	37,4
test	xgboost_alone	156.121	234.154	3,30	0,9690	-78.103	-33,3	62,6
val	lstm_alone	411.142	655.479	11,81	0,8023	0	0,0	0,0
val	hybrid	280.738	415.562	7,09	0,9205	-130.405	-31,7	78,4
val	xgboost_alone	244.843	355.027	5,68	0,9420	-35.895	-12,8	21,6

Cuadro E.5: Cadena de error lstm_alone → hybrid → xgboost_alone por partición. “Delta MAE” es el paso respecto a la fila anterior (negativo = reduce el MAE); “Cuota del hueco” expresa ese paso como fracción de la distancia total lstm_alone-xgboost_alone. No es una descomposición.

Fracción	Filas train	Secuencias	MAE val (mediana)	MAE val (mín)	MAE val (máx)	R ² val (mediana)	Fits degenerados
0.25	222	194	1.167.187	1.167.111	1.181.916	0,1990	3
0.40	356	328	515.603	492.361	517.640	0,7580	0
0.55	489	461	502.449	492.995	512.022	0,7565	0
0.70	622	594	490.773	449.494	495.200	0,7762	0
0.85	756	728	411.457	401.565	430.084	0,8033	0
1.00	889	861	416.324	379.074	433.829	0,7985	0

Cuadro E.6: Curva de aprendizaje de la LSTM de etapa 1: MAE de validación mediano (mín-máx sobre 3 semillas; 5 en la fracción 1,00, que es la base de comparación del control de paridad de la sección 5.10.2) por fracción del bloque de entrenamiento. “Fits degenerados” cuenta las semillas cuyo ajuste colapsó a una predicción cuasi-constante. La referencia xgboost_alone (val) es 244.843.

Subconjunto	N variables	prof.	n_est.	lr	MAE val	MAE test	Delta MAE val (%)
completo	161	5	100	0,05	244.843	156.121	0,0
solo temporal	39	4	100	0,05	461.854	303.927	88,6
solo exogeno	122	4	300	0,05	320.520	311.023	30,9
sin meteo	56	5	100	0,05	237.040	155.196	-3,2
solo calendario	17	5	300	0,10	300.734	279.830	22,8

Cuadro E.7: Ablación de bloques de variables sobre xgboost_alone (control diagnóstico). MAE de validación y test por subconjunto, con los hiperparámetros seleccionados en cada búsqueda y el cambio relativo de MAE de validación frente a completo. Estudio de retirada de bloques, NO una descomposición: los bloques no son disjuntos en información y las columnas no suman a nada.

Ámbito	k	MAE val (mediana)	MAE val (mín)	MAE val (máx)	Hueco cerrado (%)	Fits dege- nerados	best epoch (mediana)
calendario	17	388.625	382.695	395.877	16,2	0	75,0
completo	161	363.133	354.235	391.630	31,0	0	53,0

Cuadro E.8: LSTM en paridad informativa por ámbito de variables exógenas (control diagnóstico, 5 semillas, solo validación). MAE de validación mediano con banda mín-máx, porcentaje del hueco univariante→xgboost_alone cerrado, número de ajustes degenerados y best_epoch mediano. Base univariante: mediana de 5 semillas 416.324; mejor semilla 411.142.

Subgrupo	n	Mejor competidor	MAE com- petidor	Mejor híbrido	MAE híbrido	Delta MAE	Veredicto
Laborable ordinario	133	ensemble_equal	138.628	hybrid_weighted	200.326	61.699	no gana
Laborable puente	3	persistence	131.251	hybrid	399.637	268.386	no inferencial (n insuficiente)
Sábado	27	xgboost_alone	129.381	hybrid_weighted	185.918	56.537	no gana
Domingo	29	xgboost_alone	100.200	hybrid	233.778	133.578	no gana
Festivo	5	ensemble_inverse_mae	224.587	hybrid	666.572	441.986	no inferencial (n insuficiente)
Fin de semana	56	xgboost_alone	114.269	hybrid_weighted	215.528	101.259	no gana
Entre semana	141	ensemble_equal	143.426	hybrid_weighted	222.176	78.750	no gana
Calendario irregular	8	ensemble_inverse_mae	221.011	hybrid	566.472	345.461	no inferencial (n insuficiente)
Calendario ordinario	189	ensemble_equal	136.193	hybrid_weighted	204.831	68.638	no gana
Verano (jul-ago)	33	xgboost_alone	80.274	hybrid_weighted	133.293	53.019	no gana
Resto del año	164	ensemble_equal	150.371	hybrid_weighted	237.791	87.420	no gana
Periodo Semana Santa	10	xgboost_alone	332.267	hybrid	605.909	273.642	no analizable (sin solape val-test)
Demanda anómala	16	xgboost_alone	302.679	hybrid_weighted	520.736	218.057	no inferencial (n insuficiente)
Demanda regular	181	ensemble_equal	119.662	hybrid_weighted	193.727	74.065	no gana

Cuadro E.9: Desglose por subgrupo, partición test (ámbito de veredicto). “Delta MAE” es MAE del mejor híbrido menos MAE del mejor competidor; positivo significa que el híbrido es peor. El veredicto es único por subgrupo y se lee sobre esta partición.

Subgrupo	n	Mejor competidor	MAE competidor	Mejor híbrido	MAE híbrido	Delta MAE	Veredicto
Laborable ordinario	121	ensemble_equal	205.597	hybrid_weighted	240.264	34.667	no gana
Laborable puente	12	ensemble_inverse_mae	486.552	hybrid_weighted	690.105	203.553	no inferencial (n insuficiente)
Sábado	26	xgboost_alone	132.334	hybrid_weighted	245.120	112.786	no gana
Domingo	28	xgboost_alone	128.658	hybrid	245.450	116.792	no gana
Festivo	9	ensemble_inverse_mae	322.149	hybrid	306.409	-15.740	no inferencial (n insuficiente)
Fin de semana	54	xgboost_alone	130.428	hybrid_weighted	249.490	119.062	no gana
Entre semana	142	ensemble_equal	236.953	hybrid_weighted	286.561	49.608	no gana
Calendario irregular	21	ensemble_inverse_mae	416.094	hybrid	547.929	131.835	no inferencial (n insuficiente)
Calendario ordinario	175	ensemble_inverse_mae	203.657	hybrid_weighted	243.111	39.454	no gana
Verano (jul-ago)	57	xgboost_alone	130.465	hybrid_weighted	174.647	44.183	no gana
Resto del año	139	ensemble_inverse_mae	262.686	hybrid_weighted	318.052	55.366	no gana
Periodo Navidad	16	xgboost_alone	394.273	hybrid_weighted	603.473	209.200	no analizable (sin solape val-test)
Demanda anómala	34	xgboost_alone	343.987	hybrid	400.652	56.665	no inferencial (n insuficiente)
Demanda regular	162	ensemble_inverse_mae	190.627	hybrid_weighted	241.797	51.170	no gana

Cuadro E.10: Desglose por subgrupo, partición validación (requisito de réplica). La columna “Veredicto” repite el veredicto preregistrado único por subgrupo; esta tabla solo difiere de la E.9 en las columnas de MAE y diferencia.

Subgrupo	n	Delta MAE	IC 95 % inf.	IC 95 % sup.	p	p (BH)	Veredicto
Laborable ordinario	133	61.699	22.714	97.147	0,005	0,006	no gana
Sábado	27	56.537	8.811	107.801	0,021	0,021	no gana
Domingo	29	133.578	37.602	169.095	0,003	0,005	no gana
Fin de semana	56	101.259	31.456	130.511	0,004	0,005	no gana
Entre semana	141	78.750	35.267	120.969	0,000	0,000	no gana
Calendario ordinario	189	68.638	30.543	91.683	0,000	0,000	no gana
Verano (jul-ago)	33	53.019	—	—	0,000	0,000	no gana
Resto del año	164	87.420	40.550	116.936	0,000	0,000	no gana
Demanda regular	181	74.065	39.705	93.746	0,000	0,000	no gana

Cuadro E.11: Diferencia de MAE (mejor híbrido – mejor competidor) con intervalo de confianza *bootstrap* del 95 %, p bruto y p ajustado por Benjamini-Hochberg ($q = 0,10$), para la familia inferencial en la partición test. En verano el IC se suprime porque la fracción de réplicas utilizables cae a 0,81; la p se conserva.

Bibliografía

- [1] Anthropic. Claude. Modelo de lenguaje de gran tamaño. <https://claude.ai>, 2026.
- [2] Anthropic. Claude Code. Herramienta de desarrollo de software asistido por IA. <https://claude.com/claude-code>, 2026.
- [3] John M. Bates and Clive W. J. Granger. The combination of forecasts. *Operational Research Quarterly*, 20(4):451–468, 1969.
- [4] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012.
- [5] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, 1970.
- [6] O. D. Cardozo, J. C. García-Palomares, and J. Gutiérrez. Application of geographically weighted regression to the direct forecasting of transit ridership at station-level. *Applied Geography*, 34:548–558, 2012.
- [7] Vitor Cerqueira, Luis Torgo, and Igor Mozetič. Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning*, 109:1997–2028, 2020.
- [8] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [9] Clive W. J. Granger. Invited review: Combining forecasts—twenty years later. *Journal of Forecasting*, 8(3):167–173, 1989.
- [10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [11] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition, 2021.
- [12] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808, 2018.
- [13] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, 38(4):1325–1336, 2022.

- [14] Laura Monje, Ricardo A. Carrasco, Carlos Rosado, and Manuel Sánchez-Montaños. Deep learning XAI for bus passenger forecasting: A use case in Spain. *Mathematics*, 10(9):1428, 2022.
- [15] Michael P. Perrone and Leon N. Cooper. When networks disagree: Ensemble methods for hybrid neural networks. In Richard J. Mammone, editor, *Neural Networks for Speech and Image Processing*, pages 126–142. Chapman-Hall, 1993.
- [16] C. Surribas-Sayago et al. Hybrid CNN-LSTM-MLP architecture for diffuse solar radiation forecasting. Trabajo del dominio de la predicción de radiación solar difusa, citado únicamente como inspiración de la ingeniería de características de este TFM (retardos y términos de Fourier), no como arquitectura replicada (ver Capítulo 2).
- [17] Florian Toqué, Mostepha Khouadja, Etienne Côme, Martin Trépanier, and Latifa Oukhellou. Short and long term forecasting of multimodal transport passenger flows with machine learning methods. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, pages 560–566. IEEE, 2017.
- [18] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.
- [19] G. Peter Zhang. Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50:159–175, 2003.